

**Optimización de la gestión de memoria en simuladores cuánticos mediante la  
compresión de datos sin pérdida de precisión**

Daniel Adrián González Buendía

Javier Andres Sarmiento Salazar

**Trabajo de grado para optar el título de Ingeniero de Sistemas**

**Director**

Luis Alberto Nuñez Martínez

PhD en Ciencias de la Computación

**Codirector**

Gilberto Javier Díaz Toro

PhD en Ciencias de la Computación

**Universidad Industrial de Santander**

**Facultad De Ingenierías Fisicomecánicas**

**Escuela De Ingeniería De Sistemas e Informática**

**Pregrado en Ingeniería de Sistemas e Informática Bucaramanga**

**2025**

**Dedicatoria**

Este proyecto va dedicado a todas las personas que me apoyaron y aportaron a mi desarrollo como persona, y como profesional. A todas las personas que quiero.

A mis padres.

A mi hermana.

A mis amigos.

A Daniel.

## Contenido

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>Introducción</b>                                                             | <b>3</b>  |
| <b>1. Planteamiento del problema</b>                                            | <b>4</b>  |
| <b>2. Objetivos</b>                                                             | <b>6</b>  |
| 2.1. Objetivo General . . . . .                                                 | 6         |
| 2.2. Objetivos Específicos . . . . .                                            | 6         |
| <b>3. Marco Teórico</b>                                                         | <b>7</b>  |
| 3.1. Qubit y Representación del Estado Cuántico . . . . .                       | 7         |
| 3.2. Superposición Cuántica . . . . .                                           | 8         |
| 3.3. Entrelazamiento Cuántico . . . . .                                         | 9         |
| 3.4. Operadores Unitarios y Evolución del Estado Cuántico . . . . .             | 10        |
| 3.4.1. Puertas cuánticas como operadores unitarios. . . . .                     | 11        |
| 3.4.2. Transformaciones en sistemas de múltiples qubits. . . . .                | 12        |
| 3.5. Circuitos Cuánticos: Profundidad, Reversibilidad e Interferencia . . . . . | 12        |
| 3.5.1. Estructura de un circuito cuántico. . . . .                              | 13        |
| 3.5.2. Profundidad de un circuito cuántico. . . . .                             | 13        |
| 3.5.3. Reversibilidad de los circuitos cuánticos. . . . .                       | 13        |
| 3.5.4. Interferencia cuántica. . . . .                                          | 14        |
| 3.6. Simulación tipo Schrödinger . . . . .                                      | 14        |
| 3.7. Partición de Circuitos y Caminos de Feynman . . . . .                      | 15        |
| 3.8. Diagramas de Decisión Cuánticos . . . . .                                  | 15        |
| 3.9. Fidelidad entre Estados Cuánticos . . . . .                                | 16        |
| 3.10. Bibliotecas de Compresión con Pérdida: SZ y ZFP . . . . .                 | 17        |
| <b>4. Estado del Arte</b>                                                       | <b>18</b> |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| 4.1. Simuladores Cuánticos . . . . .                             | 18        |
| 4.2. Compresión de Memoria con Pérdida de Precisión . . . . .    | 19        |
| 4.3. Compresión de Memoria sin Pérdida de Precisión . . . . .    | 20        |
| 4.4. Algoritmos de compresión sin pérdida de precisión . . . . . | 22        |
| 4.4.1. FPC (Burtscher & Ratanaworabhan) . . . . .                | 22        |
| 4.4.2. FPZIP (Lindstrom & Isenburg) . . . . .                    | 23        |
| 4.4.3. Filtro Bitshuffle con LZ4 (Masui et al.) . . . . .        | 24        |
| 4.4.4. Transformación de Datos Tipados (TDT) . . . . .           | 25        |
| 4.5. Otros Métodos de Optimización de Memoria . . . . .          | 26        |
| <b>5. Metodología de la Investigación</b>                        | <b>27</b> |
| 5.1. Diseño . . . . .                                            | 28        |
| 5.2. Implementación y optimización del modelo . . . . .          | 29        |
| 5.3. Evaluación experimental . . . . .                           | 29        |
| <b>6. Lenguajes de Descripción de Arquitectura</b>               | <b>30</b> |
| 6.1. La necesidad de una arquitectura de referencia . . . . .    | 30        |
| 6.2. Criterios de selección . . . . .                            | 31        |
| 6.3. Búsqueda de Alternativas . . . . .                          | 32        |
| 6.4. Un nuevo modelo para Smart Campus . . . . .                 | 35        |
| 6.5. Sintaxis de la notación . . . . .                           | 39        |
| 6.6. Implementando Una Validación . . . . .                      | 42        |
| <b>7. Estado Actual Frente al de Referencia</b>                  | <b>44</b> |
| 7.1. Conociendo el estado actual . . . . .                       | 44        |
| 7.2. Centralizando los Datos . . . . .                           | 47        |
| 7.3. Implementado un Event-Handler . . . . .                     | 48        |
| 7.4. Comparando Arquitecturas . . . . .                          | 51        |

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>8. Adaptando la Arquitectura</b>                                | <b>53</b> |
| 8.1. Identificando Los Problemas, Estableciendo Acciones . . . . . | 53        |
| 8.2. De Acciones, a Instrucciones . . . . .                        | 56        |
| 8.3. De Instrucciones a Acciones . . . . .                         | 58        |
| <b>9. Validación de la Implementación</b>                          | <b>62</b> |
| 9.1. Escenario Experimental A . . . . .                            | 63        |
| 9.2. Escenario Experimental B . . . . .                            | 64        |
| <b>10. Análisis de Resultados</b>                                  | <b>65</b> |
| 10.1. Resultados De Escenario Experimental A . . . . .             | 65        |
| 10.2. Resultados De Escenario Experimental B . . . . .             | 70        |
| <b>11. Conclusiones y Trabajo Futuro</b>                           | <b>75</b> |
| <b>Referencias</b>                                                 | <b>77</b> |

### Lista de figuras

|     |                                                                                                                                                                                               |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.  | Representación de un estado cuántico sobre la esfera de Bloch. Los polos corresponden a los estados $ 0\rangle$ y $ 1\rangle$ , y los puntos intermedios representan superposiciones. . . . . | 9  |
| 2.  | Metodología de investigación . . . . .                                                                                                                                                        | 28 |
| 3.  | Versión 2 del modelo concepto planteado por Jiménez Herrera . . . . .                                                                                                                         | 36 |
| 4.  | Mensaje JSON enviado por los dispositivos en Smart Campus UIS . . . . .                                                                                                                       | 37 |
| 5.  | Versión 1 del metamodelo planteado para SCampusADL . . . . .                                                                                                                                  | 38 |
| 6.  | Diagrama de rail de la sintaxis definida para la notación de SmartCampusADL                                                                                                                   | 39 |
| 7.  | Diagrama de rail de la sintaxis de declaración de la aplicación . . . . .                                                                                                                     | 40 |
| 8.  | Diagrama de rail de la sintaxis definida para la notación del contexto geográfico de la aplicación . . . . .                                                                                  | 40 |
| 9.  | Diagrama de rail de la sintaxis definida para la notación de los componentes de la aplicación . . . . .                                                                                       | 41 |
| 10. | YAML de una posible aplicación de monitoreo . . . . .                                                                                                                                         | 42 |
| 11. | Diagrama de flujo del proceso realizado por el módulo <i>Lexical</i> . . . . .                                                                                                                | 43 |
| 12. | Diagrama de flujo para validación y procesamiento de las locaciones . . . . .                                                                                                                 | 43 |
| 13. | Diagrama de flujo del procesamiento de los requerimientos de datos . . . . .                                                                                                                  | 44 |
| 14. | Arquitectura del prototipo de Smart Campus definido por . . . . .                                                                                                                             | 46 |
| 15. | Arquitectura actual del proyecto con el agregador . . . . .                                                                                                                                   | 48 |
| 16. | Diagrama de flujo del proceso realizado por Lexical actualizado . . . . .                                                                                                                     | 49 |
| 17. | Primera propuesta del proceso a realizar por <i>Looker</i> . . . . .                                                                                                                          | 49 |
| 18. | Proceso realizado por el observador durante el consumo de los mensajes enviados por los dispositivos . . . . .                                                                                | 50 |
| 19. | Proceso reloj realizado por el observador para la actualización del estado de la aplicación . . . . .                                                                                         | 52 |

|     |                                                                                                      |    |
|-----|------------------------------------------------------------------------------------------------------|----|
| 20. | Arquitectura actual del proyecto . . . . .                                                           | 54 |
| 21. | Proceso para la identificación de problemas de los requerimientos de datos . . . . .                 | 55 |
| 22. | UML de la estructura de directivas y ordenes . . . . .                                               | 57 |
| 23. | Proceso de decisión de acciones base realizado por Bran . . . . .                                    | 58 |
| 24. | Proceso base de DoThing . . . . .                                                                    | 59 |
| 25. | Proceso para la ejecución de ordenes de adición . . . . .                                            | 60 |
| 26. | Proceso para la ejecución de ordenes de reinicio . . . . .                                           | 60 |
| 27. | Proceso para la ejecución de ordenes de reconfiguración . . . . .                                    | 61 |
| 28. | Arquitectura completa del proyecto . . . . .                                                         | 62 |
| 29. | Diagrama del escenario experimental A . . . . .                                                      | 63 |
| 30. | Validación de la declaración de referencia realizada para el escenario experi-<br>mental A . . . . . | 63 |
| 31. | Aplicaciones definidas para el desarrollo del escenario experimental B . . . . .                     | 64 |
| 32. | Aplicación y directivas registradas en Bran . . . . .                                                | 66 |
| 33. | Looker evalúa el estado de la aplicación . . . . .                                                   | 66 |
| 34. | Bran identifica los problemas y establece las acciones . . . . .                                     | 67 |
| 35. | DoThing ejecuta las acciones establecidas por Bran . . . . .                                         | 67 |
| 36. | Contenedores creados por DoThing en respuesta a las órdenes recibidas . . . . .                      | 68 |
| 37. | Bran identifica otros problemas en la aplicación . . . . .                                           | 69 |
| 38. | Bran establece acciones de reconfiguración para adaptar la aplicación . . . . .                      | 69 |
| 39. | Aplicaciones y directivas registradas en Bran . . . . .                                              | 71 |
| 40. | Dos instancias de looker monitoreando las aplicaciones declaradas . . . . .                          | 72 |
| 41. | Se matan los contenedores con los servicios iniciales . . . . .                                      | 72 |
| 42. | Bran intenta reiniciar los contenedores . . . . .                                                    | 73 |
| 43. | DoThing no puede encontrar los contenedores . . . . .                                                | 73 |
| 44. | Bran crea órdenes de adicción . . . . .                                                              | 74 |

**Lista de tablas**

1. Criterios usados para la determinación de la notación a utilizar . . . . . 32
2. Evaluación de las alternativas en función de los criterios establecidos . . . . . 35



**Lista de apéndices**

|    |                                                      |    |
|----|------------------------------------------------------|----|
| 1. | Directivas declaradas la aplicación Malaga . . . . . | 82 |
| 2. | Directivas de la aplicación Malaga . . . . .         | 83 |
| 3. | YAML de la aplicación Patio . . . . .                | 85 |
| 4. | YAML la aplicación House . . . . .                   | 85 |
| 5. | Directivas de la aplicación Patio . . . . .          | 86 |
| 6. | Directivas de la aplicación House . . . . .          | 87 |

## Glosario

**IoT:** Internet of Things

**LDA:** Lenguaje de Descripción de Arquitectura (Architecture Description Language)

## Resumen

**Título:** Optimización de la gestión de memoria en simuladores cuánticos mediante la compresión de datos sin pérdida de precisión<sup>1</sup>

**Autores:** Daniel Adrián González Buendía  
Javier Andres Sarmiento Salazar<sup>2</sup>

**Palabras clave:** Computación cuántica, Simulación cuántica, Gestión de memoria, Compresión de datos, Computación de Alto Rendimiento (HPC)

### Descripción:

La simulación de sistemas cuánticos en hardware clásico enfrenta una limitación severa debido al crecimiento exponencial en la memoria requerida. A medida que aumenta el número de qubits, la cantidad de datos a almacenar supera rápidamente las capacidades de las computadoras convencionales y supercomputadoras. Esta restricción impide la exploración de algoritmos cuánticos avanzados y dificulta la validación de experimentos en entornos clásicos antes de su ejecución en hardware cuántico. Por ello, es fundamental desarrollar estrategias eficientes de gestión de memoria que permitan extender el alcance de estas simulaciones sin comprometer la fidelidad de los cálculos. A pesar de los avances en simuladores cuánticos y técnicas de optimización, la gestión de memoria sigue siendo un cuello de botella significativo. En particular, muchas soluciones actuales sacrifican fidelidad a cambio de eficiencia, lo que limita su aplicabilidad en contextos donde la precisión es fundamental. Por ello, es necesario explorar estrategias que reduzcan el consumo de memoria sin comprometer la exactitud de los resultados. Este trabajo propone el diseño e implementación de una estrategia de compresión de memoria sin pérdida de precisión en simuladores cuánticos, con el objetivo de encontrar un balance óptimo entre uso de memoria, rendimiento y fidelidad. Al final, se espera evaluar la viabilidad de la propuesta y compararla con otros enfoques existentes para determinar su impacto en la optimización de simulaciones cuánticas en hardware clásico.

---

<sup>1</sup>Trabajo de Investigación

<sup>2</sup>Facultad De Ingenierías Fisicomecánicas. Escuela De Ingeniería De Sistemas e Informática.  
Director: PhD. Luis Alberto Nuñez Martínez, Codirector: PhD. Gilberto Javier Díaz Toro

## Abstract

**Title:** Optimization of memory management in quantum simulators through data compression without loss of precision<sup>3</sup>

**Authors:** Daniel Adrián González Buendía  
Javier Andres Sarmiento Salazar<sup>4</sup>

**Key words:** Quantum Computing, Quantum Simulation, Memory Management, Data Compression, High-Performance Computing (HPC)

### Description:

The simulation of quantum systems on classical hardware faces a severe limitation due to the exponential growth in memory requirements. As the number of qubits increases, the amount of data to be stored quickly exceeds the capabilities of conventional computers and supercomputers. This restriction hinders the exploration of advanced quantum algorithms and complicates the validation of experiments in classical environments before their execution on quantum hardware. Therefore, it is crucial to develop efficient memory management strategies that extend the scope of these simulations without compromising computational fidelity. Despite advances in quantum simulators and optimization techniques, memory management remains a significant bottleneck. In particular, many current solutions sacrifice fidelity in exchange for efficiency, limiting their applicability in contexts where precision is crucial. Therefore, it is necessary to explore strategies that reduce memory consumption without compromising the accuracy of the results. This work proposes the design and implementation of a lossless memory compression strategy for quantum simulators, aiming to find an optimal balance between memory usage, performance, and fidelity. Ultimately, the feasibility of the proposal will be evaluated and compared with existing approaches to determine its impact on optimizing quantum simulations on classical hardware.

---

<sup>3</sup>Research Work

<sup>4</sup>Faculty of Physics-Mechanics Engineering. School of Systems Engineering and Computer Science.  
Advisor: PhD. Luis Alberto Nuñez Martínez, Co-Advisor: PhD. Gilberto Javier Díaz Toro

## Introducción

La simulación cuántica en computadoras clásicas enfrenta un desafío fundamental debido al crecimiento exponencial del espacio de estados conforme aumenta el número de qubits. Por ejemplo, la representación de un sistema cuántico de 40 qubits requiere aproximadamente  $8 \text{ bytes (double)} \times 2 \text{ (real, imag)} \times 2^{40} = 16 \text{ TB}$  de memoria RAM, mientras que para 50 qubits la demanda se dispara a petabytes (PB), superando la capacidad de las computadoras convencionales (Wu y cols., 2019). Esta limitación de memoria restringe el estudio de algoritmos cuánticos avanzados y su validación en hardware clásico antes de su implementación en procesadores cuánticos (Zhang y cols., 2024a). Ante esta problemática, es necesario desarrollar estrategias eficientes de gestión de memoria que permitan extender el alcance de las simulaciones cuánticas sin comprometer la viabilidad computacional.

Considerando la creciente importancia de la computación cuántica, optimizar la memoria en simuladores cuánticos no solo permite ampliar el rango de algoritmos simulables en hardware clásico, sino que también reduce la dependencia exclusiva de procesadores cuánticos experimentales. Además, una gestión eficiente de memoria podría reducir significativamente los costos computacionales y el tiempo de simulación en supercomputadoras, lo que resulta en un beneficio directo tanto para la investigación en computación cuántica como para aplicaciones prácticas en la industria y la ciencia.

Para abordar este problema, se han explorado diversas estrategias que pueden agruparse en varias familias de métodos. Entre ellas, la compresión de datos ha sido ampliamente estudiada con enfoques como la compresión con pérdida controlada (e.g., SZ, ZFP), que permite reducir la carga de almacenamiento de vectores de estado sin afectar significativamente la fidelidad de los cálculos (Wu y cols., 2018). Otra línea de investigación relevante es la de los métodos de podado de estados cuánticos, los cuales eliminan amplitudes irrelevantes para disminuir el consumo de memoria, aunque con un impacto variable en la precisión del resultado (Song, Li, y Wu, 2023). Asimismo, la computación de alto rendimiento (HPC o High

Performance Computing) ha demostrado ser una solución clave, empleando distribución de datos entre nodos y optimizaciones en el acceso a memoria para maximizar la eficiencia de la simulación en infraestructuras de supercomputadoras (Díaz, Steffenel, Barrios, y Couturier, 2024).

A pesar de los avances en estas áreas, muchas de estas estrategias introducen errores acumulativos en la simulación, lo que puede comprometer la validez de los resultados. En este contexto, esta investigación propone una estrategia basada en compresión de datos sin pérdida de precisión, con el objetivo de optimizar el almacenamiento de vectores de estado sin alterar las amplitudes cuánticas originales. Para ello, el proyecto se desarrollará en varias etapas. Inicialmente, se llevará a cabo una revisión del estado del arte para identificar las técnicas de compresión más relevantes. Luego, se seleccionará un simulador cuántico adecuado que permita modificaciones en su gestión de memoria para evaluar dichas técnicas. Posteriormente, se diseñará una estrategia para seleccionar e integrar técnicas de compresión sin pérdida de precisión existentes, seguida de su implementación y optimización en el simulador elegido. Finalmente, se realizarán experimentos controlados para evaluar el impacto de la compresión en términos de ahorro de memoria, tiempo de simulación y fidelidad del resultado, comparándolo con enfoques tradicionales. En las siguientes secciones, se detallarán cada una de estas etapas y su contribución al objetivo final del proyecto.

## 1 Planteamiento del problema

La simulación de sistemas cuánticos en computadoras clásicas se enfrenta a una barrera casi insuperable: el crecimiento exponencial de la memoria requerida. Representar un estado cuántico de  $n$  qubits implica almacenar  $2^n$  números complejos, lo que rápidamente supera la capacidad de cualquier sistema de memoria convencional. Las técnicas actuales, como la poda de estados, reducen drásticamente el consumo de recursos pero a costa de la precisión, mientras que la compresión con pérdida, aunque ofrece mayores índices de reducción, sacrifica irremediabilmente la fidelidad de los cálculos. Este dilema —la imposibilidad de simular

algoritmos cuánticos complejos sin perder información crítica— constituye el núcleo del problema, amenazando la validez y aplicabilidad de las simulaciones en el desarrollo de nuevas estrategias cuánticas.

Este desafío se agrava al considerar que la simulación precisa es esencial para la validación y desarrollo de algoritmos cuánticos en entornos clásicos. La degradación de la precisión debido a la pérdida de datos no solo impide obtener resultados confiables, sino que también limita la escalabilidad y el rendimiento de los simuladores cuánticos. Por ello, es crucial explorar alternativas que permitan optimizar el uso de recursos computacionales sin comprometer la exactitud de los cálculos, destacando la urgente necesidad de investigar métodos de compresión sin pérdida que ofrezcan un equilibrio entre eficiencia y fidelidad en la simulación cuántica.

Con este planteamiento se proponen los siguientes objetivos para abordar esta problemática.

## 2 Objetivos

### 2.1 Objetivo General

- Diseñar e implementar una estrategia eficiente de gestión de memoria en simuladores cuánticos basada en técnicas de compresión sin pérdida de precisión, con el fin de optimizar el uso de recursos computacionales sin comprometer la fidelidad de las simulaciones.

### 2.2 Objetivos Específicos

- Analizar el impacto del uso de herramientas de compresión de datos sin pérdida de precisión en el consumo de memoria y precisión de los resultados en simuladores cuánticos.
- Diseñar una estrategia integral de gestión de memoria basada en compresión sin pérdida de precisión, definiendo el algoritmo, los parámetros de compresión-descompresión y los puntos de integración con el simulador cuántico seleccionado.
- Desarrollar e integrar la estrategia diseñada en el simulador elegido, optimizando la sobrecarga computacional para garantizar un ahorro significativo de memoria sin afectar la fidelidad de la simulación.
- Implementar dos algoritmos cuánticos distintos en un simulador cuántico para evaluar el uso de memoria, velocidad y precisión con y sin compresión de datos.
- Comparar los resultados obtenidos con estrategias tradicionales y métodos que emplean compresión con pérdida de precisión, identificando ventajas y desventajas de cada enfoque.



### 3 Marco Teórico

La computación cuántica representa un nuevo paradigma en el procesamiento de información, basado en fenómenos propios de la mecánica cuántica como la superposición, el entrelazamiento y la interferencia. Comprender estos conceptos es esencial para abordar los retos que plantea su simulación en computadoras clásicas y para justificar el uso de técnicas avanzadas como la compresión de datos sin pérdida. A continuación, se presentan los fundamentos teóricos necesarios para contextualizar el problema abordado en este trabajo.

#### 3.1 Qubit y Representación del Estado Cuántico

Un *qubit* o *bit cuántico* es la unidad básica de información en computación cuántica. A diferencia del bit clásico, que solo puede representar los valores 0 o 1, un qubit puede encontrarse en una superposición lineal de ambos estados. Matemáticamente, su estado se expresa como:

$$|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$$

donde  $\alpha_1, \alpha_2 \in \mathbb{C}$  y  $|\alpha_1|^2 + |\alpha_2|^2 = 1$ . Esta condición garantiza que la probabilidad de encontrar al qubit en el estado  $|0\rangle$  o en el estado  $|1\rangle$  sea del 100%. Cuando se tienen  $n$  qubits, el sistema completo se representa como un vector de  $2^n$  amplitudes complejas en un espacio de Hilbert de dimensión  $2^n$ . Para  $n = 3$  hay  $2^3 = 8$  estados computacionales. El estado general de 3 qubits puede escribirse como:

$$|\psi\rangle = \sum_{i=0}^7 \alpha_i |i\rangle, \quad \text{con } |i\rangle \in \{|000\rangle, |001\rangle, \dots, |111\rangle\}$$

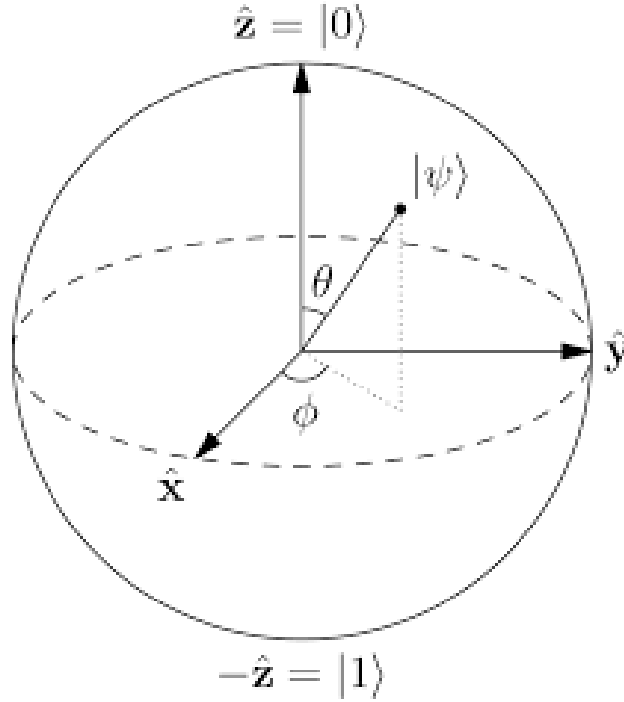
Aquí,  $\alpha_i \in \mathbb{C}$  es la amplitud asociada al estado base  $|i\rangle$  y satisface la condición de normalización  $\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1$ . Cada amplitud compleja en doble precisión ocupa 16 bytes (8 bytes para la parte real y 8 bytes para la imaginaria), lo que impone una gran carga de memoria cuando  $n$  crece.

### 3.2 Superposición Cuántica

La *superposición* es una de las propiedades fundamentales de la mecánica cuántica. Consiste en la capacidad de un sistema cuántico de existir simultáneamente en múltiples estados posibles antes de ser observado o medido. En el caso de un qubit, esta propiedad se expresa formalmente como:

$$|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle, \quad \text{con } \alpha_1, \alpha_2 \in \mathbb{C} \text{ y } |\alpha_1|^2 + |\alpha_2|^2 = 1$$

Aquí,  $|0\rangle$  y  $|1\rangle$  representan los estados clásicos posibles del qubit, mientras que  $\alpha_1$  y  $\alpha_2$  son las amplitudes de probabilidad asociadas a cada estado. Aunque el sistema ‘está’ en ambos estados al mismo tiempo, una vez se realiza una medición, colapsa a uno solo de ellos con probabilidad  $|\alpha_1|^2$  o  $|\alpha_2|^2$ , respectivamente. Desde una perspectiva geométrica, el estado de un qubit puede representarse como un punto en la esfera de Bloch, donde cada combinación válida de  $\alpha_1$  y  $\alpha_2$  corresponde a una ubicación en la superficie de la esfera. Esta representación visual resulta útil para entender cómo las puertas cuánticas rotan el estado en dicho espacio.



**Figura 1:** Representación de un estado cuántico sobre la esfera de Bloch. Los polos corresponden a los estados  $|0\rangle$  y  $|1\rangle$ , y los puntos intermedios representan superposiciones.

En términos computacionales, la superposición permite que un sistema de  $n$  qubits represente, de manera simultánea, todos los  $2^n$  posibles estados clásicos, lo cual es el fundamento del *paralelismo cuántico*. Este fenómeno, aunque no equivale directamente a realizar  $2^n$  cálculos a la vez, permite evaluar de forma concurrente múltiples configuraciones de entrada, y es lo que hace potencialmente tan poderosos a los algoritmos cuánticos.

### 3.3 Entrelazamiento Cuántico

El *entrelazamiento* cuántico es un fenómeno mediante el cual dos o más qubits quedan conectados de tal manera que el estado de uno de ellos no puede describirse independientemente del estado del otro, sin importar cuán lejos se encuentren en el espacio. Esta correlación es más fuerte que cualquier correlación clásica y desafía las intuiciones tradicionales sobre causalidad y localidad. Formalmente, un sistema de dos qubits está entrelazado si su estado no puede expresarse como el producto tensorial de dos estados individuales. Por ejemplo, el

siguiente estado de Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

En esta expresión, el factor  $1/\sqrt{2}$  garantiza la normalización del estado, ya que  $\left|\frac{1}{\sqrt{2}}\right|^2 + \left|\frac{1}{\sqrt{2}}\right|^2 = 1$ . El signo “+” indica una superposición lineal (suma vectorial en el espacio de Hilbert) de los estados base  $|00\rangle$  y  $|11\rangle$ ; no corresponde a concatenación ni a una operación lógica clásica.

Este estado de Bell no puede escribirse como  $|\psi_1\rangle \otimes |\psi_2\rangle$ . Esto significa que no existe una descripción completa del sistema que consista simplemente en describir el estado de cada qubit por separado. Este tipo de correlación permite que la medición de uno de los qubits determine instantáneamente el estado del otro, incluso si están separados por distancias arbitrariamente grandes. Sin embargo, este efecto no viola la relatividad ni permite transmitir información más rápido que la luz, ya que el resultado de cada medición individual es aleatorio. El entrelazamiento es un recurso clave en muchos algoritmos cuánticos avanzados, en protocolos de teleportación cuántica, y en esquemas de criptografía cuántica como el intercambio de claves cuánticas (Quantum Key Distribution, QKD). Además, es una de las principales fuentes de complejidad en la simulación de sistemas cuánticos en hardware clásico, pues aumenta de forma significativa el entrecruzamiento de información entre los qubits.

### 3.4 Operadores Unitarios y Evolución del Estado Cuántico

La evolución de un sistema cuántico aislado se describe mediante la mecánica cuántica a través de la ecuación de Schrödinger. Esta evolución es determinista, reversible y está gobernada por operadores matemáticos denominados *operadores unitarios*. En términos generales, si el estado inicial de un sistema es  $|\psi(0)\rangle$ , su evolución en el tiempo  $t$  viene dada por la aplicación de un operador unitario  $U(t)$ :

$$|\psi(t)\rangle = U(t) |\psi(0)\rangle$$

Donde  $U(t)$  es una matriz unitaria que cumple:

$$U^\dagger U = U U^\dagger = I$$

Aquí,  $U^\dagger$  representa el conjugado transpuesto (o adjunto) de  $U$ , e  $I$  es la matriz identidad. Esta condición garantiza que la norma del estado cuántico se conserva durante la evolución, lo que implica que la probabilidad total sigue siendo 1. Además, asegura la *reversibilidad* de las operaciones, una característica fundamental de la dinámica cuántica (con excepción del proceso de medición).

### 3.4.1 Puertas cuánticas como operadores unitarios.

En computación cuántica, las operaciones que transforman los estados de los qubits se conocen como *puertas cuánticas*. Estas puertas son implementaciones físicas y algorítmicas de operadores unitarios y constituyen los bloques fundamentales para construir circuitos cuánticos. Cada puerta cuántica puede representarse mediante una matriz unitaria. Por ejemplo, la puerta Hadamard ( $H$ ), que genera una superposición equitativa, se define como:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Aplicar  $H$  al estado base  $|0\rangle$  produce:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Este resultado es una superposición con amplitudes iguales de los estados  $|0\rangle$  y  $|1\rangle$ , un recurso clave en algoritmos como el de Grover o Deutsch-Jozsa.

### 3.4.2 Transformaciones en sistemas de múltiples qubits.

En sistemas con varios qubits, los operadores unitarios pueden actuar sobre uno o más qubits. Por ejemplo, la puerta CNOT (control-NOT) es una operación de dos qubits que invierte el segundo qubit (qubit objetivo) si el primero (qubit de control) está en el estado  $|1\rangle$ . Esta operación es esencial para generar entrelazamiento entre qubits. Para aplicar una puerta a un sistema completo de  $n$  qubits, se utiliza el producto tensorial de matrices. Por ejemplo, para aplicar una puerta  $G$  al tercer qubit en un sistema de 5 qubits, se construye:

$$U = I \otimes I \otimes G \otimes I \otimes I$$

La dimensión total de la matriz  $U$  será  $2^5 \times 2^5 = 32 \times 32$ , lo que ilustra el rápido crecimiento de la complejidad computacional al aumentar el número de qubits.

## 3.5 Circuitos Cuánticos: Profundidad, Reversibilidad e Interferencia

La computación cuántica, en su forma más común, se modela a través del llamado *modelo de circuitos cuánticos*. En este enfoque, un algoritmo cuántico se representa como una secuencia de operaciones —puertas cuánticas— aplicadas a un conjunto inicial de qubits. El circuito cuántico se construye de manera análoga a los circuitos digitales clásicos, pero con componentes que manipulan estados cuánticos en lugar de bits clásicos.

### 3.5.1 Estructura de un circuito cuántico.

Formalmente, un circuito cuántico está compuesto por tres fases:

1. **Inicialización:** Se prepara el sistema en un estado base, usualmente  $|0\rangle^{\otimes n}$ .
2. **Aplicación de puertas cuánticas:** Se aplica una secuencia de operaciones unitarias sobre los qubits.
3. **Medición:** Se colapsa el sistema a un resultado clásico mediante la observación.

Cada puerta cuántica actúa sobre uno o varios qubits, y las operaciones se disponen en capas, donde una capa puede contener varias puertas que se aplican en paralelo a qubits distintos.

### 3.5.2 Profundidad de un circuito cuántico.

La *profundidad* de un circuito se refiere al número de capas secuenciales de operaciones que deben ejecutarse en orden, considerando que algunas pueden realizarse en paralelo. Un circuito de menor profundidad implica que el algoritmo se ejecuta en menos pasos cuánticos, lo cual es deseable por dos razones principales:

- Reduce la duración total del algoritmo, minimizando la exposición del sistema a errores por decoherencia.
- Mejora la viabilidad de implementación en dispositivos cuánticos actuales, que están limitados en fidelidad y tiempo de coherencia.

Por ejemplo, un circuito que aplica  $n$  puertas de una sola vez en paralelo tiene una profundidad de 1, mientras que si cada puerta debe aplicarse una tras otra, la profundidad será  $n$ . Así, la profundidad es una medida directa de la complejidad temporal del algoritmo.

### 3.5.3 Reversibilidad de los circuitos cuánticos.

Toda operación cuántica (exceptuando la medición) debe ser *reversible*, lo que significa que debe existir una operación inversa que recupere el estado anterior sin pérdida de in-

formación. Esto se garantiza matemáticamente porque las puertas cuánticas se representan mediante operadores unitarios, los cuales por definición son invertibles. Esta propiedad distingue a los circuitos cuánticos de los circuitos clásicos, donde muchas operaciones (como AND u OR) son irreversibles. En el mundo cuántico, la única operación no reversible es la medición, que colapsa el estado superpuesto a uno de sus posibles resultados y destruye la información contenida en las amplitudes de probabilidad. La reversibilidad también es esencial para ciertos algoritmos cuánticos que requieren ‘deshacer’ operaciones intermedias sin borrar información, como ocurre en la técnica de *descomputación* (uncomputation), utilizada para limpiar registros auxiliares antes de la medición.

#### 3.5.4 Interferencia cuántica.

La *interferencia* cuántica es un fenómeno exclusivo de los sistemas que evolucionan bajo la mecánica cuántica. En este contexto, las amplitudes de probabilidad asociadas a distintos caminos de evolución del sistema pueden combinarse de forma constructiva (reforzando la probabilidad de ciertos resultados) o destructiva (cancelando otros resultados). Esta propiedad es esencial para el funcionamiento de muchos algoritmos cuánticos. Por ejemplo, en el algoritmo de Grover para búsqueda no estructurada, las soluciones correctas se refuerzan mediante interferencia constructiva, mientras que las opciones incorrectas se atenúan mediante interferencia destructiva. A diferencia de la superposición, que permite explorar múltiples estados al mismo tiempo, la interferencia actúa como un mecanismo de filtrado que permite extraer los resultados deseados al final del proceso computacional. Es la combinación de ambas —superposición e interferencia— lo que da lugar a la aceleración cuántica.

### 3.6 Simulación tipo Schrödinger

La simulación tipo Schrödinger es una de las aproximaciones más utilizadas en la computación cuántica para modelar la evolución de un sistema de qubits. Este método consiste en almacenar el vector de estado completo del sistema cuántico, que contiene  $2^n$  amplitudes



complejas para un sistema de  $n$  qubits. En cada paso del circuito, se aplican directamente las puertas cuánticas como transformaciones matriciales sobre este vector. Este enfoque ofrece una representación exacta del sistema y permite la simulación precisa de cualquier circuito cuántico, independientemente del grado de entrelazamiento o la profundidad del circuito. Sin embargo, su principal limitación es el consumo de memoria, que crece exponencialmente con el número de qubits lo cual es impracticable para la mayoría de los sistemas.

### 3.7 Partición de Circuitos y Caminos de Feynman

Una alternativa a la simulación tipo Schrödinger es el uso de técnicas basadas en *partición de circuitos* y *caminos de Feynman*. Estos métodos dividen el circuito cuántico en subcircuitos más pequeños o regiones menos entrelazadas, reduciendo así los requerimientos de memoria al costo de mayor tiempo de cómputo. La técnica de caminos de Feynman interpreta la evolución del sistema como una suma sobre todos los caminos computacionales posibles que conectan los estados iniciales y finales. Esta formulación, si bien elegante y más eficiente en términos de espacio, conlleva un crecimiento exponencial del número de caminos a evaluar, especialmente cuando la profundidad del circuito es alta. Estos enfoques han sido utilizados exitosamente para simular circuitos de hasta 54 qubits, principalmente adoptando soluciones híbridas con las simulaciones de Schrödinger, obteniendo de esta manera una reducción significativa en el uso de la memoria, sin sacrificar rendimiento (Arute y cols., 2019; Boixo, Isakov, Smelyanskiy, y Neven, 2017; Markov, Fatima, Isakov, y Boixo, 2018; Pednault y cols., 2020).

### 3.8 Diagramas de Decisión Cuánticos

Los *diagramas de decisión cuánticos* son estructuras de datos jerárquicas inspiradas en los diagramas de decisión binarios (BDDs) utilizados en verificación formal y diseño lógico. Estas estructuras permiten representar de forma comprimida vectores de estado y matrices unitarias aprovechando redundancias y patrones repetitivos.

- **QuIDD (Quantum Information Decision Diagrams)** codifican vectores y operadores mediante nodos reutilizables, lo que permite representar estructuras altamente simétricas de manera eficiente.
- **QMDD (Quantum Multiple-valued Decision Diagrams)** extienden los QuIDD utilizando aristas ponderadas con matrices, lo que los hace más aptos para representar operadores complejos y circuitos reversibles.
- **LIMDD (Local Invertible Map Decision Diagrams)** introducen transformaciones locales invertibles como aristas, lo cual permite representar incluso estados estabilizadores y circuitos con entrelazamiento moderado de manera eficiente.

Estas estructuras son altamente efectivas cuando el circuito presenta redundancia estructural, permitiendo pasar de una representación exponencial a una polinomial. Sin embargo, pierden eficacia en circuitos con alta interferencia o sin regularidad estructural, como ocurre en simulaciones aleatorias o algoritmos de propósito general.

### 3.9 Fidelidad entre Estados Cuánticos

Una métrica fundamental para evaluar la similitud entre dos estados cuánticos es la *fidelidad*. Generalmente, se emplea para cuantificar la diferencia entre dos estados cuánticos. Dado dos estados puros  $|\psi_r\rangle$  (referencia) y  $|\psi_c\rangle$  (estado simulado), la fidelidad entre ellos se define como:

$$F(|\psi_r\rangle, |\psi_c\rangle) = |\langle\psi_r|\psi_c\rangle|^2.$$

Esta expresión mide el cuadrado del valor absoluto del producto interno entre ambos vectores. La fidelidad toma valores entre 0 y 1, donde:

- $F = 1$  indica que los estados son idénticos.
- $F = 0$  indica que son ortogonales (completamente diferentes).
- Valores intermedios indican distintos grados de similitud.

Por ejemplo, si:

$$|\psi_r\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad |\psi_c\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 0,99 \\ 0 \\ 0 \\ 1,01 \end{bmatrix}$$

entonces, normalizando y aplicando la fórmula, se obtiene una fidelidad cercana a 1, lo que indica que el estado simulado es prácticamente indistinguible del original, a pesar de las pequeñas diferencias numéricas. Esta métrica es especialmente útil para evaluar estrategias de compresión: una fidelidad igual a 1 garantiza que el proceso de compresión y descompresión no ha alterado el estado cuántico original.

### 3.10 Bibliotecas de Compresión con Pérdida: SZ y ZFP

Las bibliotecas **SZ** y **ZFP** son herramientas de compresión científica ampliamente utilizadas para reducir el tamaño de datos en punto flotante sin necesidad de conservar todos los dígitos exactos.

- **SZ** aplica una estrategia de predicción adaptativa con control de error. Estima el valor de cada dato basado en sus vecinos, almacena solo la diferencia si esta cae dentro de un margen de error aceptable, y utiliza codificación entropica para comprimir los residuos. Es muy eficiente cuando los datos presentan correlaciones locales.
- **ZFP** divide los datos en bloques pequeños y los transforma a un dominio más compresible mediante una técnica similar a la transformada discreta del coseno. Posteriormente, se aplican técnicas de truncamiento y codificación por bloques. ZFP permite controlar la compresión por tasa fija, error absoluto o fidelidad relativa.

## 4 Estado del Arte

### 4.1 Simuladores Cuánticos

La simulación clásica de circuitos cuánticos es esencial para el desarrollo y validación de algoritmos cuánticos. Sin embargo, el principal desafío radica en la representación del estado cuántico, cuyo tamaño crece exponencialmente con el número de qubits.

Uno de los primeros enfoques para abordar este problema fue la representación mediante *Matrix Product States* (MPS), propuesta por Vidal (2003). Este método permite representar ciertos estados cuánticos de forma eficiente cuando el entrelazamiento es limitado, reduciendo los requisitos de almacenamiento y cómputo. Su principal ventaja es la escalabilidad en circuitos con baja profundidad, permitiendo simular cientos de qubits. No obstante, su limitación radica en que para sistemas altamente entrelazados el costo computacional crece exponencialmente, restringiendo su aplicabilidad a ciertos algoritmos.

A medida que se buscaban métodos más generales, surgió la simulación tipo *Schrödinger*, que almacena el vector de estado completo y lo actualiza en cada operación cuántica. Este enfoque, empleado en simuladores como QuEST y Qiskit Aer (Li, Kimura, Sato, Yu, y Fujita, 2023), garantiza alta precisión y permite simular cualquier circuito cuántico. Su desventaja principal es la extrema demanda de memoria, la cual crece exponencialmente con el número de qubits, alcanzando 0.5 petabytes para 45 qubits y varios exabytes para 50 qubits (Pednault y cols., 2020).

Para mitigar el problema de memoria, se desarrollaron métodos basados en partición del circuito y caminos de Feynman. Por ejemplo, Google e IBM utilizaron esta técnica para simular circuitos de hasta 56 qubits con profundidad limitada (Chen, Zhang, Huang, Newman, y Shi, 2018). Sin embargo, aunque reducen el almacenamiento requerido, estos métodos incrementan la complejidad computacional, ya que el número de caminos crece exponencialmente con la profundidad del circuito.

## 4.2 Compresión de Memoria con Pérdida de Precisión

Dado que la representación exacta de estados cuánticos es ineficiente en términos de memoria, se han explorado técnicas de compresión con pérdida de precisión. Estos enfoques sacrifican una pequeña cantidad de fidelidad numérica a cambio de una reducción significativa en el uso de memoria, permitiendo la simulación de circuitos más grandes.

Uno de los primeros enfoques en este ámbito fue el de Wu y cols. (2018, 2019), quienes introdujeron la compresión adaptativa con error acotado. Utilizando la biblioteca SZ, lograron reducir el tamaño del vector de estado hasta en un factor de 16, manteniendo fidelidades superiores al 99.9 %. Su principal ventaja es que permite ampliar el número de qubits simulados sin un incremento proporcional en los requisitos de memoria. Sin embargo, la compresión y descompresión recurrente introduce sobrecarga computacional, afectando la eficiencia temporal de la simulación.

En un esfuerzo por maximizar la escala de la simulación cuántica de estado completo, Zhang y cols. (2024a) presentaron *BMQSim*, un marco que extiende *SV-Sim* mediante el uso de **compresores con pérdida de precisión y control de error punto a punto** ejecutados directamente en GPU (por ejemplo, variantes de la familia SZ/cuSZ)(Zhang y cols., 2024b). Su diseño combina:

- Una estrategia de *circuit partitioning* que disminuye la frecuencia de (des)compresión.
- Un *pipeline* solapado que oculta en gran medida el coste de mover y comprimir datos.
- Una gestión de memoria jerárquica (VRAM + SSD mediante GPUDirect Storage) que asigna dinámicamente espacio a los bloques comprimidos.

Con estas técnicas, *BMQSim* reduce el consumo de memoria en más de un orden de magnitud y mantiene fidelidades superiores al 99 % sin penalizar de forma apreciable el tiempo de simulación. Su eficacia, sin embargo, está condicionada a la disponibilidad de hardware acelerado con GPUs de alto rendimiento y unidades NVMe rápidas, lo que puede restringir su adopción en plataformas de menor capacidad computacional.

Más recientemente, Huffman, Pavlichin, y Weissman (2024) exploraron la cuantización de amplitudes como un mecanismo para reducir la precisión numérica sin afectar la fidelidad global del sistema. Demostraron que representaciones con tan solo 7 bits por amplitud permiten mantener una fidelidad superior al 99 %, lo que sugiere que la precisión completa de punto flotante puede ser innecesaria en muchas simulaciones. Aunque esta técnica ofrece un enfoque simple y eficiente para reducir el uso de memoria, su aplicabilidad a circuitos de gran escala aún requiere validación empírica.

Díaz Toro (2025) propone un esquema de **vector de estado comprimido** que emplea compresores con pérdida de precisión—principalmente *ZFP* en modo *fixed-rate*, complementado por *SZ*—para reducir la huella de memoria del simulador. El autor reporta reducciones del **10–20 %** en casos base y, en configuraciones de mayor escala, ahorros que alcanzan hasta un **75 %** de la memoria requerida. *ZFP* opera sobre bloques independientes de  $4^d$  valores, de modo que la (des)compresión de cada bloque se realiza en completo aislamiento del resto; esto permite solapar dichas operaciones con el cómputo principal y amortiguar la sobrecarga introducida. Aunque la compresión incrementa el tiempo de ejecución, el impacto se compensa al transmitir los fragmentos del vector en formato comprimido dentro de una arquitectura distribuida basada en *MPI*, reduciendo el volumen de comunicación y habilitando simulaciones de hasta 30 qubits en los experimentos reportados. Finalmente, el estudio concluye que esta estrategia de *estado completo comprimido distribuido* resulta más fiable y escalable que la *poda de estados de baja probabilidad*, la cual presenta descensos de fidelidad y sobrecarga adicional que la hacen desaconsejable.

### 4.3 Compresión de Memoria sin Pérdida de Precisión

La simulación cuántica eficiente requiere reducir el consumo de memoria sin comprometer la fidelidad del estado cuántico. Para ello, se han desarrollado técnicas de compresión sin pérdida de precisión que buscan representar estados y operadores cuánticos de manera más compacta sin alterar sus valores.

Uno de los primeros enfoques en esta dirección fue el uso de diagramas de decisión cuánticos (*Quantum Information Decision Diagrams*, QuIDD), introducidos por Viamontes, Markov, y Hayes (2005). Este método permite representar vectores de estado y operadores unitarios aprovechando redundancias estructurales en sus coeficientes, lo que permite una representación más eficiente en memoria. Su ventaja radica en que, para circuitos con alta redundancia, el crecimiento en memoria puede reducirse de exponencial a polinomial. Sin embargo, su eficacia se limita a circuitos con estructura repetitiva, fallando en casos de entrelazamiento complejo.

En un desarrollo posterior, Miller y Thornton (2006) propusieron los *Quantum Multiple-valued Decision Diagrams* (QMDD), los cuales extienden la representación de QuIDD mediante el uso de aristas ponderadas con matrices unitarias. Esto permitió manejar de manera más eficiente circuitos reversibles y operadores cuánticos de mayor tamaño. Aunque estos diagramas optimizan la representación de puertas lógicas y matrices densas, su eficiencia sigue dependiendo de la estructura interna del circuito.

Más recientemente, Jaques y Häner (2021) exploraron la esparsidad del estado cuántico para almacenar solo las amplitudes no nulas, reduciendo significativamente el almacenamiento requerido en algoritmos con exploración limitada del espacio de Hilbert. Su enfoque demostró ser altamente eficiente en problemas de factorización y aritmética cuántica, pero pierde efectividad en circuitos con alta interferencia y entrelazamiento.

En 2023, Vinkhuijzen, Coopmans, Elkouss, Dunjko, y Laarman (2023) introdujeron los *Local Invertible Map Decision Diagrams* (LIMDD), una evolución de los diagramas de decisión que incorpora transformaciones locales para mejorar la representación compacta del estado cuántico. Esta técnica permite manejar estados estabilizadores y ciertas estructuras altamente entrelazadas que eran difíciles de representar con diagramas de decisión tradicionales. No obstante, su aplicabilidad sigue limitada a casos donde las transformaciones locales puedan ser explotadas para reducir la complejidad estructural.

Estos avances en compresión sin pérdida han permitido extender la capacidad de simu-

lación de circuitos cuánticos en hardware clásico. Sin embargo, aún existen limitaciones en la representación de estados arbitrarios, lo que ha llevado a la exploración de otros métodos para optimizar el uso de memoria.

#### 4.4 Algoritmos de compresión sin pérdida de precisión

En simulaciones científicas y cuánticas es común manejar vectores de estado representados como grandes arreglos de números de punto flotante. Cuando estos datos carecen de correlación temporal o patrones estructurales (es decir, presentan alta entropía o *aleatoriedad*), la compresión sin pérdida se vuelve especialmente desafiante. Muchos algoritmos de compresión de punto flotante aprovechan transiciones suaves o redundancias entre valores consecutivos; por ejemplo, técnicas de serie temporal almacenan solo los bits que difieren entre un valor y el siguiente, explotando que a menudo los valores sucesivos comparten prefijos de bits iguales. Sin embargo, en datos altamente aleatorios, estas suposiciones se rompen y tales métodos tienden a no lograr compresión significativa. Por lo tanto, es crucial identificar y seleccionar algoritmos de compresión sin pérdida que sean efectivos para datos de punto flotante con estas características. A continuación, se describen los principales algoritmos que han sido concebidos para este tipo de datos, analizando su funcionamiento, rendimiento y pertinencia para ser implementados y evaluados en el contexto de este trabajo de grado, con el fin de encontrar un balance óptimo entre el ahorro de memoria y la eficiencia computacional.

##### 4.4.1 *FPC (Burtscher & Ratanaworabhan)*

El algoritmo *FPC* (*Floating Point Compressor*)(Burtscher y Ratanaworabhan, 2009) fue diseñado para la compresión sin pérdida de flujos lineales de datos en doble precisión (64 bits). Internamente, FPC emplea un esquema de *predicción* dual junto con codificación de ceros líderes. Concretamente, mantiene dos predictores (derivados de técnicas FCM/DFCM) almacenados en tablas hash, que generan un valor estimado para el siguiente número en la secuencia. Para cada nuevo valor de entrada, FPC selecciona la predicción que más coincide



en sus bits más significativos con el valor real, y calcula el **XOR** entre el valor real y el predicho. Este **XOR** resalta únicamente los bits diferentes, convirtiendo los bits coincidentes en *cero*. A continuación, el algoritmo cuenta cuántos bytes iniciales del resultado **XOR** son cero (ceros líderes) y codifica esa cantidad en 3 bits, junto con 1 bit adicional que indica cuál predictor fue usado. Estos 4 bits de código, seguidos de los *bytes residuales* no nulos (almacenados sin más codificación), forman la salida comprimida.

Esta estrategia implica que, si el valor predicho es cercano al real, la diferencia tendrá muchos ceros líderes y, por tanto, pocos bytes residuales que almacenar. En escenarios de datos altamente aleatorios (donde las predicciones rara vez aciertan), FPC tiende a obtener pocos ceros líderes y la tasa de compresión se aproxima a 1:1 (sin reducción notable). No obstante, su diseño minimiza la penalización en estos casos: los bloques comprimidos incluyen encabezados mínimos y códigos de apenas 4 bits por valor, garantizando sobrecarga reducida y un rendimiento consistente. Así, aunque FPC no logre compresión sustancial en vectores de estado cuántico aleatorios, ofrece una solución de alta velocidad con mínima sobrecarga, capaz de detectar rápidamente si no hay redundancia aprovechable y manejar dichos datos sin degradar el rendimiento.

#### 4.4.2 **FPZIP** (*Lindstrom & Isenburg*)

*FPZIP* (Lindstrom y Isenburg, 2006) es un compresor de propósito específico para arreglos de punto flotante. Su enfoque se basa en la *predicción adaptativa* y la *codificación entrópica* eficiente para lograr compresión sin pérdida en múltiples contextos (mallas no estructuradas, volúmenes 3D, imágenes, etc.). Internamente, FPZIP aplica esquemas de predicción multi-dimensional (por ejemplo, predictores que aprovechan la continuidad espacial de los datos científicos) y luego comprime únicamente el error de predicción. A diferencia de métodos que cuantizan los flotantes a enteros, FPZIP opera directamente sobre las representaciones IEEE 754, manteniendo exactitud bit a bit.

Su arquitectura soporta predictores ajustables al tipo de datos: por ejemplo, en un campo

volumétrico 3D puede usar un predictor basado en valores vecinos, mientras que en una serie temporal podría usar extrapolación simple. Tras la predicción, FPZIP emplea un modelo de codificación entrópica optimizado para velocidad, que comprime los residuos de manera eficaz. Esta combinación le permite alcanzar tasas de compresión y velocidades competitivas, acelerando la E/S de simulaciones científicas al reducir el volumen de datos sin sacrificar precisión. En contextos de datos altamente aleatorios, donde las predicciones en su mayoría fallan, FPZIP tiende a comportarse como un compresor genérico pero con ventajas sutiles: incluso en ausencia de correlación temporal, puede explotar patrones locales dentro de la representación en coma flotante (p. ej., regularidades en campos de exponentes o signos comunes) que los algoritmos genéricos pasan por alto al tratar los datos solo como bytes crudos. Para vectores de estado cuántico, si bien los amplitudes pueden lucir aleatorios, FPZIP podría aprovechar distribuciones no uniformes de ciertos bits (por ejemplo, exponentes repetidos), logrando compresiones modestas con un rendimiento robusto.

#### 4.4.3 Filtro *Bitshuffle* con LZ4 (Masui et al.)

Una estrategia distinta para afrontar datos de punto flotante sin patrones evidentes es emplear transformaciones previas que revelen posibles regularidades a nivel de bits. *Bitshuffle* (Masui y cols., 2015) es un algoritmo de preprocesamiento sin pérdida que extiende el filtro *byte-shuffle* al nivel de *bit*. En lugar de reordenar solo los bytes de cada número (como hace el filtro *shuffle* de HDF5), Bitshuffle realiza una *transposición bit a bit* de una matriz de valores: considera la representación binaria de  $N$  números de  $m$  bits (por ejemplo,  $m=32$  para flotantes en simple precisión) como una matriz de  $N \times m$ , y reorganiza la data tomando columnas de bits completas. Tras esta permuta, aplica una compresión LZ (usualmente LZ4) sobre la secuencia resultante.

El fundamento es que, incluso si los valores originales parecen aleatorios, puede haber *correlaciones ocultas* dentro de ciertos planos de bits. Por ejemplo, en datos científicos muchos valores comparten el mismo exponente o presentan bits altos similares, lo que tras Bitshuffle

se manifiesta como secuencias repetitivas (*runs* de ceros o unos) más fáciles de comprimir mediante LZ4. A diferencia de métodos predictivos, Bitshuffle no asume continuidad entre elementos adyacentes; su efectividad radica en convertir cualquier correlación intra-byte (dentro de la representación binaria de cada valor) en redundancias horizontales a lo largo de la secuencia transformada. En datos de alta entropía pura (bits totalmente aleatorios), Bitshuffle no puede crear patrones donde no los hay; con todo, en vectores de estado cuántico puede explotar pequeñas irregularidades: si los qubits están normalizados, muchos amplitudes comparten signos o exponentes comunes, generando columnas de bits constantes que LZ4 comprime muy bien. En la práctica, Bitshuffle+LZ4 logra relaciones de compresión superiores a compresores genéricos en escenarios científicos sin reducción de precisión, a la vez que mantiene velocidades de descompresión del orden de gigabytes por segundo por núcleo.

#### 4.4.4 Transformación de Datos Tipados (TDT)

Una innovación reciente en compresión de flotantes es la *Transformación de Datos Tipados* (TDT) (Jamalidinan y Cheshmi, 2025), concebida para cerrar la brecha entre compresores genéricos y especializados en punto flotante. TDT no es un compresor en sí, sino una etapa de *preprocesamiento* adaptable que reorganiza los bytes de los datos antes de aplicar un compresor convencional (p. ej., Zstd o LZ4). La idea clave es agrupar y separar los bytes según su *nivel de entropía* y patrones estadísticos, en lugar de tratarlos uniformemente. En una representación IEEE 754, distintos campos (signo, exponente, mantisa) pueden poseer distribuciones diferentes; por ejemplo, en muchas series científicas los bytes de exponente tienden a repetirse más que los bytes de mantisa. TDT analiza el conjunto de datos (en bloques) extrayendo características estadísticas de cada posición de byte (entropía media, desviación, frecuencias de valores) y aplica *clustering* para decidir cuáles bytes se comportan de forma similar. Luego *reordena* la matriz de datos colocando juntos los bytes de comportamientos afines y separando los de entropía distinta. Esto produce flujos de datos más homogéneos que, al ser alimentados a un compresor genérico, permiten una codificación más eficiente.

Importante: TDT no altera ningún bit de la información (es totalmente reversible), solo cambia su disposición para exponer patrones que antes estaban enmascarados. En el contexto de vectores de estado cuántico —que suelen ser casos límite de alta aleatoriedad— TDT buscaría, por ejemplo, si ciertos bytes de mantisa comparten distribuciones o si el byte de exponente tiene menor entropía debido a normalizaciones. Incluso si los datos son mayormente aleatorios, esta técnica puede detectar *no uniformidades* sutiles entre componentes y agruparlas para facilitar su compresión. Resulta particularmente atractiva como capa previa porque ofrece un equilibrio: en *datasets* de baja entropía conserva beneficios de modelos flotantes (predicciones locales), y en *datasets* de alta entropía se comporta como un reordenamiento que potencia a los compresores genéricos, habilitando ganancias modestas de compresión y mejoras de rendimiento en los *throughputs* de compresión y descompresión.

#### 4.5 Otros Métodos de Optimización de Memoria

Además de la compresión sin pérdida, se han desarrollado enfoques alternativos para reducir el consumo de memoria en simuladores cuánticos, incluyendo técnicas basadas en redes tensoriales, poda de estados y optimización HPC.

Uno de los primeros enfoques en esta dirección fue propuesto por Markov y Shi (2008), quienes introdujeron el uso de redes tensoriales para simular circuitos cuánticos mediante contracción optimizada de tensores. En este método, el circuito se representa como un grafo tensorial, y su simulación se optimiza contrayendo secuencias de tensores en un orden eficiente. Esto permite representar ciertos circuitos con memoria subexponencial, aunque la efectividad del método depende de la estructura del entrelazamiento.

Posteriormente, Boixo y cols. (2017) propusieron un modelo gráfico para circuitos cuánticos de baja profundidad basado en la eliminación de variables en grafos probabilísticos. Su técnica permitió simular circuitos aleatorios cercanos al régimen de supremacía cuántica en hardware clásico, extendiendo el tamaño de circuitos simulables sin necesidad de almacenar el estado cuántico completo. Sin embargo, este método pierde eficacia conforme aumenta la

profundidad del circuito.

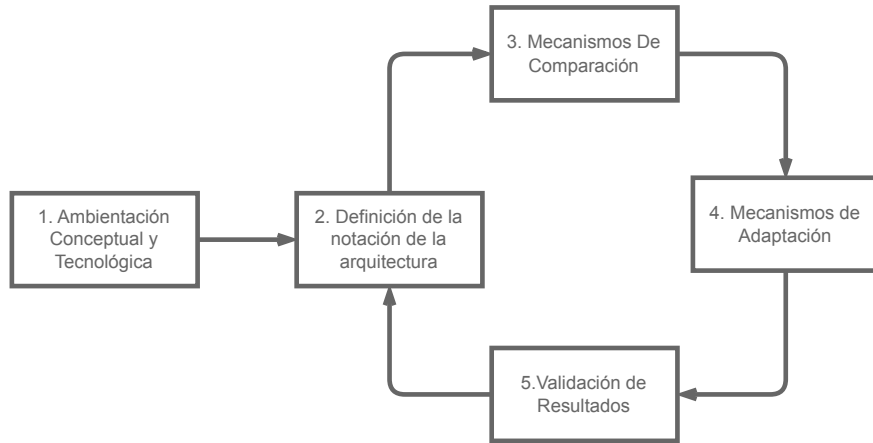
En paralelo, Pednault y cols. (2020) introdujeron técnicas de diferimiento de contracciones tensoriales en simulaciones HPC. Su propuesta permite manejar circuitos más grandes al intercambiar memoria por cómputo adicional, almacenando resultados intermedios en memoria secundaria y evitando la expansión total del estado cuántico en RAM. Esta estrategia ha sido clave en la simulación de circuitos con estructuras altamente no triviales, aunque su implementación requiere acceso a infraestructuras de supercomputación.

Finalmente, como se analizó previamente, la tesis doctoral de Díaz Toro (2025) introduce técnicas de compresión con pérdida de precisión en esquemas distribuidos de simulación cuántica. Si bien su propuesta constituye un hito en la reducción del consumo de memoria mediante bibliotecas como ZFP y SZ, el presente trabajo se orienta en una dirección distinta: se retoman fundamentos como el uso de vectores de estado y paralelismo distribuido con *MPI*, pero se propone como contribución original la exploración de técnicas de compresión **sin pérdida de precisión**. Esta línea de trabajo, no abordada en dicha tesis, busca preservar la fidelidad numérica total durante la simulación, evaluando el impacto empírico de algoritmos de compresión científica sin pérdida en el contexto de computación cuántica.

## 5 Metodología de la Investigación

Para este estudio, se ha seleccionado una metodología basada en investigación aplicada y evaluación experimental debido a la necesidad de diseñar, implementar y validar una estrategia de compresión sin pérdida de precisión en simuladores cuánticos. Dado que el problema a abordar implica la optimización del uso de memoria en simulaciones de alto consumo computacional, es fundamental seguir un enfoque sistemático que permita comparar la efectividad de la propuesta con otras estrategias existentes. Además, la metodología elegida permite realizar pruebas controladas sobre diferentes simuladores cuánticos y evaluar el impacto de la compresión en términos de memoria, rendimiento y fidelidad, garantizando así la reproducibilidad y validez de los resultados obtenidos.

**Figura 2:**  
Metodología de investigación



### 5.1 Diseño

Comprende el estudio del estado del arte, la definición de requisitos y supuestos, la selección del simulador, y la planificación experimental y de integración técnica.

- **Revisión de literatura y estado del arte.** Levantamiento sistemático de técnicas de compresión sin pérdida de precisión para arreglos de punto flotante y su aplicabilidad en simulación cuántica de *vectores de estado*. Se identifican ventajas, limitaciones y brechas; se sintetizan los fundamentos teóricos y se establecen hipótesis de trabajo.
- **Criterios y procedimiento de selección del simulador cuántico.** Evaluación de alternativas (p. ej., QuEST, Intel-QS, qsim, Quantum++, TMFQS) con base en: (i) *apertura y modificabilidad* del código, (ii) *facilidad de integración* de estructuras/arrays personalizados para el vector de estado, (iii) *soporte a optimizaciones de memoria* y paralelismo (OpenMP/MPI/GPU), y (iv) *capacidad de instrumentación* (medición de memoria y tiempo). Se aplica una matriz de decisión ponderada y se documenta la trazabilidad de la elección.
- **Arquitectura de la solución y diseño de la estrategia de compresión.** Especificación de las estructuras de datos y puntos de inserción en el simulador (capa de

almacenamiento del vector de amplitudes y rutas críticas de acceso). Selección del/los algoritmo(s) de compresión sin pérdida de precisión o equivalentes numéricos que no alteren la fidelidad numérica de los estados cuánticos (p. ej., representaciones compactas o transformaciones reversibles), así como políticas de (des)compresión y su impacto esperado en memoria, tiempo y paralelismo.

## 5.2 Implementación y optimización del modelo

La estrategia de compresión seleccionada se implementará en el simulador cuántico elegido, modificando su sistema de almacenamiento y gestión de memoria. Se realizarán optimizaciones para garantizar un desempeño eficiente, evitando costos computacionales excesivos debido a la aplicación de la compresión. Durante esta etapa, se llevarán a cabo las siguientes actividades:

- Modificación del código fuente del simulador para integrar la compresión sin pérdida.
- Optimización del sistema de almacenamiento para reducir el uso de memoria sin afectar el rendimiento.
- Validación inicial de la implementación mediante pruebas con circuitos de prueba.

## 5.3 Evaluación experimental

Se diseñará un conjunto de experimentos para evaluar el impacto de la estrategia de compresión en diferentes circuitos cuánticos. Los experimentos permitirán determinar si se logra un balance adecuado entre eficiencia computacional y fidelidad. En esta fase se realizarán las siguientes actividades:

- Definición de un conjunto de circuitos de prueba para evaluar la estrategia de compresión:
  - Transformada de Fourier Cuántica (QFT)

- Algoritmo de búsqueda de Grover
- Ejecución de simulaciones con y sin compresión para medir su impacto en el rendimiento y el uso de memoria:
  - Tiempo de ejecución total (segundos)
  - Uso máximo de memoria (MB)
  - Fidelidad del estado final:

$$F(|\psi_r\rangle, |\psi_c\rangle) = |\langle\psi_r|\psi_c\rangle|^2.$$

- Análisis de los resultados obtenidos y comparación con enfoques tradicionales.

## 6 Lenguajes de Descripción de Arquitectura

### 6.1 La necesidad de una arquitectura de referencia

La necesidad de una arquitectura de referencia, o arquitectura objetivo; es una parte esencial dentro de la computación autónoma. Esta forma parte de la base de conocimiento (K), y, de manera indirecta, de los objetivos del administrador del sistema (Lalanda, Diaconescu, y McCann, 2014, p. 24).

Con el fin de establecer un objetivo para el sistema autónomo, fue necesario determinar una manera de realizar la declaración de dicha arquitectura, que estableciera un estado de referencia. De esta manera, podría evaluarse el estado del sistema en tiempo de ejecución, y así tomar las acciones necesarias para adaptarlo hacia el estado de referencia.

Ahora, fue también necesario establecer el punto desde el cual se lleva a cabo la comparación entre los estados del sistema. Esto guió la búsqueda para determinar el como se realizó la declaración de los estados objetivo de Smart Campus UIS, al igual que los datos a recolectar.



Siendo así, y dados los objetivos que buscan cumplir los ecosistemas inteligentes para la toma de decisiones (Anagnostopoulos, 2023), se enfocó no sobre la arquitectura sobre la cual trabaja la plataforma, sino sobre las aplicaciones desarrollándose sobre esta. Es decir, los datos requeridos. Son las necesidades definidas por las aplicaciones, las descritas y usadas para evaluar el estado del sistema. Siendo así, que el enfoque debía estar orientado a cumplir con las necesidades de los datos establecidas por Smart Campus. Esto se expondrá de manera más clara durante las fases de comparación y adaptación.

## **6.2 Criterios de selección**

Con el fin de establecer la notación a usar para la declaración de las arquitecturas objetivo, se establecieron unos lineamientos con los cuales se realizaría la evaluación de las diferentes notaciones ya desarrolladas anteriormente. De esta manera se podría escoger la manera a representar los modelos, o en el caso de ser necesario, establecer los criterios por el cual se podría desarrollar uno.

**Tabla 1:**

Criterios usados para la determinación de la notación a utilizar

|    | Criterio                                                            | Explicación                                                                                                                                                                                                                                                                                                                                                        |
|----|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| C1 | Describir la arquitectura de un sistema IoT                         | Este criterio es una base a establecer con el fin de descartar aquellos lenguajes de notación generales o no necesariamente usados para la descripción de arquitecturas IoT.                                                                                                                                                                                       |
| C2 | Permitir la especificación de la ubicación del componente           | La especificación de la ubicación de los componentes es importante en los sistemas IoT, especialmente dentro del contexto del proyecto en el cual se está trabajando con un Smart Campus; ya que los componentes pueden estar distribuidos en diferentes ubicaciones físicas y la evaluación de su integridad puede depender de su presencia en un lugar dado.     |
| C3 | Habilitar el modelado del componente a nivel de sus entradas        | Es importante poder describir las entradas de los componentes, específicamente los datos que manejan, así como su rol dentro del sistema.                                                                                                                                                                                                                          |
| C4 | Modelar el comportamiento de los componentes                        | La notación debe permitir modelar el comportamiento de los componentes, de manera que se puedan entender sus interacciones y su función en el sistema IoT. Dentro del contexto del proyecto, no es tan relevante, ya que no se está evaluando la funcionalidad del sistema, sin embargo, para futuros trabajos, podría facilitar la extensión de Smart Campus UIS. |
| C5 | Posibilitar el establecer los estados de los componentes            | La notación debe poder definir los estados de los componentes. Estos estados pueden ser tanto de comportamiento o operacionales.                                                                                                                                                                                                                                   |
| C6 | Permitir de exportar el modelo descrito a gráficas u otros formatos | Es importante que la herramienta permita exportar el modelo descrito en diferentes formatos para facilitar su integración con otras herramientas y sistemas, y para permitir su visualización en diferentes formatos.                                                                                                                                              |

### 6.3 Búsqueda de Alternativas

Una vez establecidos los criterios de selección, se realizó una exhaustiva búsqueda de alternativas disponibles en la literatura y en la industria para describir arquitecturas de sistemas IoT. Este proceso se realizó a partir de la revisión en diferentes bases de datos, como *Scopus*; al igual que algunas de las revistas especializadas en el tema, y el internet en

general.

Durante la búsqueda, se identificaron una gran variedad de opciones. Sin embargo, la gran mayoría de estas se filtraron, o descartaron; a partir de los criterios de selección establecidos. Esto se debió a que los LDAs usados tanto en la industria y academia, como AADL (Architecture Analysis and Design Language), tienen un enfoque a los campos de aviónica, equipos médicos y aeronáutica (lo que complicaría su implementación hacia sistemas de software IoT) (Carnegie Mellon University, 2022, 2017); o SysML, que son bastante genéricos y abarcan hardware, software e incluso personas y procesos (Object Management Group, 2015).

Es así como, de las posibles opciones de notación, se seleccionaron cinco, las cuales fueron evaluadas con el fin de determinar si alguna de estas hubiera podido ser usada, o si era necesario desarrollar una notación propia para el proyecto. A continuación, se presentan las alternativas evaluadas:

- **MontiThings:** Basado en MontiArc, otro LDA más general; está diseñado para el modelado y prototipado de aplicaciones de IoT. Este realiza las descripciones de sus arquitecturas en un modelo componente-conector, donde los componentes están compuestos por otros componentes; y los conectores definen la manera en la que se comunican dichos componentes a nivel de los datos y la dirección de los mismos. (Kirchhof, Rumpe, Schmalzing, y Wortmann, 2022b, 2022a)
- **Eclipse Mita:** Mita, creado por la Eclipse Foundation; es un lenguaje de programación orientado al facilitar la programación de sistemas IoT. Aunque como tal no es un LDA, está orientada a la descripción de los componentes y el comportamiento del sistema establecido, de esta manera, puede generar el código que debe correr en los dispositivos embebidos (Eclipse Foundation, 2018).
- **SysML4IoT:** Es un perfil de SysML<sup>5</sup> en el cual se usan estereotipos de UML con el

---

<sup>5</sup>Los perfiles se refieren a extensiones a UML, en este caso es una extensión de SysML en sí (Andre, 2007).

fin de abstraer las diferentes partes de los sistemas de IoT. Al igual que SysML, este permite el modelado más allá de dispositivos incluso llegando a personas y procesos, con la diferencia del enfoque dado al *dominio de IoT*<sup>6</sup> (Costa, Pires, y Delicato, 2016).

- **ThingML:** Similar a Mita, ThingML, es un lenguaje de modelado el cual tiene capacidades de generar el código requerido por los dispositivos embebidos. En términos del proyecto, este permite el modelado de los sistemas de IoT a partir de *state machine models*<sup>7</sup> los cuales permiten describir los componentes del sistema al igual que el comportamiento de estos (Harrand, Fleurey, Morin, y Husa, 2016).
- **IoT-DDL:** Iot-DDL es un LDA, implementado en XML, que describe objetos dentro de los ecosistemas IoT con base en sus componentes, identidad y servicios entre otros. Este tiene la capacidad de describir parte de la base de conocimiento que tienen los diferentes componentes (Principalmente relaciones y asociaciones entre componentes) (Khaled, Helal, Lindquist, y Lee, 2018).

Una vez seleccionadas las alternativas a evaluar, se utilizó la matriz de evaluación en la tabla 2 para determinar la solución ideal para el desarrollo del proyecto.

---

<sup>6</sup>El *dominio del IoT*, hace referencia al *Architecture Reference Model* establecido por IoT-A, un consorcio Europeo el cual buscaba el establecer un modelo para la interoperatividad de dispositivos IoT. (IoT-A, 2014)

<sup>7</sup>Los *state machine model*, también conocidas como Autómatas Finitos; son modelos matemáticos que describen todos los posibles estados de un sistema a partir de unas entradas dadas (Mozilla Foundation, 2023).

**Tabla 2:**

Evaluación de las alternativas en función de los criterios establecidos

|    | MontiThings | Eclipse Mita | SysML4IoT | ThingML | IoT-DDL |
|----|-------------|--------------|-----------|---------|---------|
| C1 | ✓           | ✓            | ✓         | ✓       | ✓       |
| C2 | ✗           | ✗            | ✗         | ✗       | ✗       |
| C3 | ✓           | ✓            | ✓         | ✓       | ✓       |
| C4 | ✓           | ✓            | ✓         | ✓       | ✗       |
| C5 | ✓           | ✓            | ✗         | ✓       | ✗       |
| C6 | ✗           | ✗            | ✗         | ✓       | ✗       |

Partiendo de los resultados de la evaluación, se puede apreciar que ninguno de estos LDA cumple con los criterios establecidos para el proyecto. Aunque estos están orientados hacia la descripción y desarrollo de sistemas IoT, no están enfocados hacia su contexto en términos de aplicación más allá de la definición de comportamiento. Esto se debe a los objetivos de cada una de estas notaciones, de una u otra manera; buscan el representar como tal el sistema IoT en términos de su funcionalidad técnica y no la aplicación en sí.

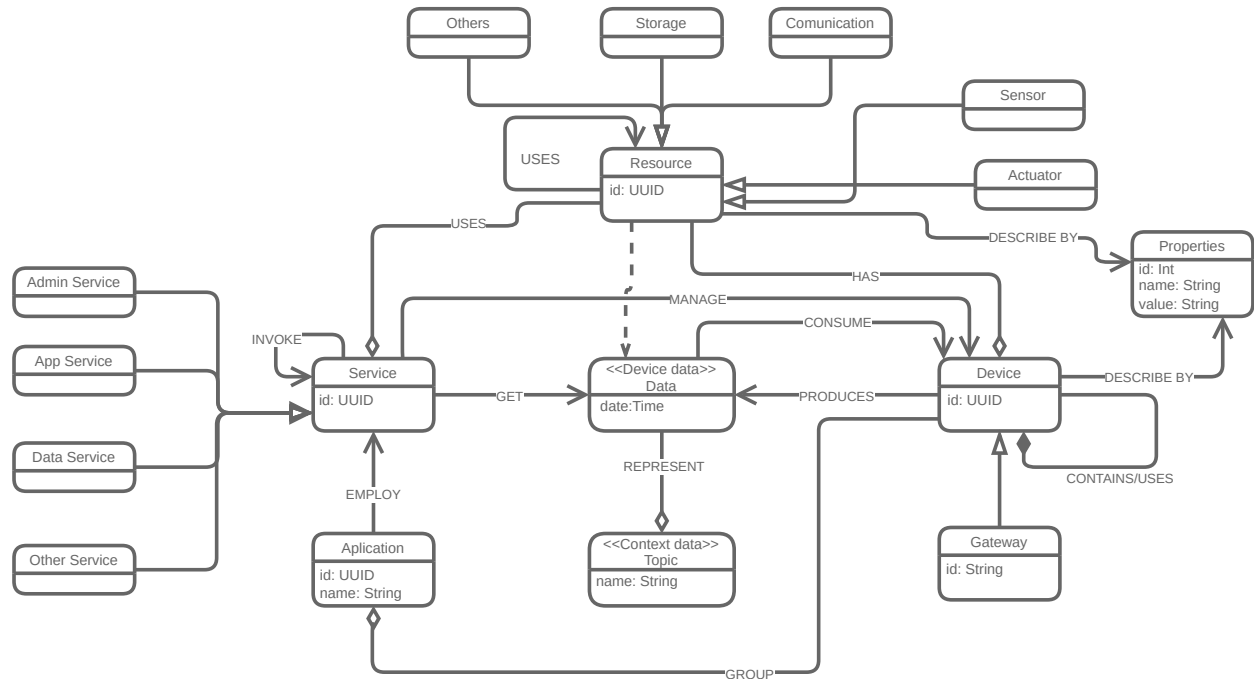
#### 6.4 Un nuevo modelo para Smart Campus

Dado que no hay una alternativa que se adapte completamente a las necesidades del proyecto, se tuvo que definir una notación propia, la cual permita modelar las arquitecturas a nivel de aplicación bajo el contexto de un Smart Campus, tomando en cuenta los criterios definidos como guía para el desarrollo.

Primeramente, fue necesario definir un metamodelo que permitiera establecer la manera en la que se ven estas arquitecturas. Para ello, y partiendo de la implementación a realizar, se apoyó parcialmente en el modelo establecido por Jiménez Herrera (2022, p. 63).

**Figura 3:**

Versión 2 del modelo concepto planteado por Jiménez Herrera



Basarse en el modelo de la figura 3, da la capacidad de describir a un nivel técnico un sistema IoT. Ahora, aunque se podría usar para el desarrollo del proyecto, fue necesario modificarlo, con el fin de acercarnos más hacia la descripción de un sistema IoT a nivel de aplicación.

El primer paso fue establecer el contexto de los dispositivos. Esto específicamente se refiere al criterio *C2* de la tabla 1, en donde, dada la necesidad de establecer la ubicación geográfica en algunas de las aplicaciones de los Smart Campus, era necesario poder describir los lugares pertenecientes a la aplicación.

Así mismo, se cubre el criterio *C3* cambiando las propiedades del dispositivos de una clase, externa a los dispositivos; a un atributo, interno, el cual le permite a los componentes manejar su propia información en cuanto a los datos que estos manejan. Estas propiedades pueden referirse a las entradas que tienen, en el caso de ser actuadores o procesadores de la información; o a los valores que reportan al sistema en el caso de ser sensores.

Ahora, algo importante a tener en cuenta, es la manera en la que se están reportando los

datos desde los sensores hacia Smart Campus UIS. Esto se debe a la necesidad de tener en cuenta los datos a los cuales tenemos realmente acceso desde cada uno de los dispositivos.

Cada uno de los dispositivos de Smart Campus, reporta un ‘JSON’ similar al presente en la figura 4. Este mensaje contiene información que permite identificar al dispositivo, gracias al *UUID*; al igual que el momento y los datos que este reporta.

**Figura 4:**

Mensaje JSON enviado por los dispositivos en Smart Campus UIS

(Jiménez Herrera, 2023)

```
{
  "deviceUUID": "1",
  "topic": "temperatura",
  "timeStamp": "06-01-2024 10:39:02",
  "values": {
    "temperature": 10.0,
    "co2": 15.0,
    "location": "AP2"
  },
  "status": "OK",
  "alert": false
}
```

Es importante destacar que, aunque contamos con suficiente información para identificar los dispositivos, en estos mensajes no está presente información crucial para el mapeo físico de los dispositivos. Por lo tanto, será necesario proporcionar manualmente estos datos durante la etapa de adaptación para llevar a cabo los ajustes necesarios en la arquitectura.

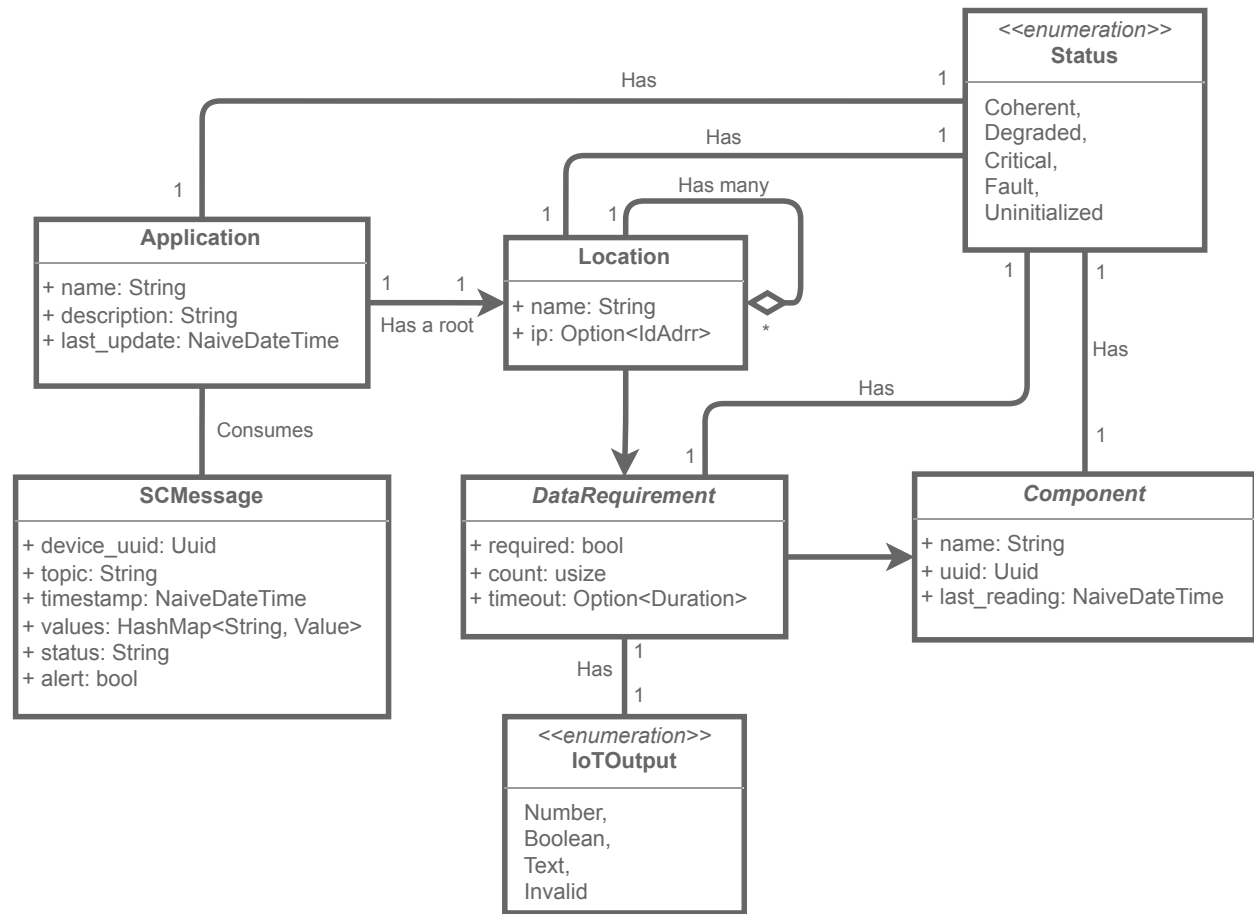
Partiendo de esto, se desarrolló el UML presente en la figura 5. La base de este es la aplicación (*Application*). Este contiene, a partir de una ubicación raíz, todas las demás locaciones (*location*) pertinentes para el correcto funcionamiento de esta.

Las locaciones, tienen una serie de requerimientos de datos (*DataRequirements*), las cuales son las representaciones lógicas, o de software, de las necesidades de la aplicación. Estos pueden ser de 2 tipos: requeridos, que son aquellos con los cuales de no tenerlos, la aplicación no podría funcionar correctamente; y los opcionales, los cuales, aunque buenos de tener, no impiden el funcionamiento.

1

**Figura 5:**

Versión 1 del metamodelo planteado para SCampusADL



Cada uno de estos requerimientos de datos, contiene la cantidad de dispositivos, y a su vez datos, requeridos por la locación, al igual que el tipo de datos esperado (*IoTOutput*), y el que el tiempo máximo permitido entre reportes de datos.

Los requerimientos de datos, llevan un registro de los componentes (*Component*) que han reportado datos en la locación. Esto permite evaluar cada uno de estos, y en la misma medida, determinar el estado (*Status*) de cada una de las locaciones registradas en la aplicación. Estos estados buscan describir qué tan diferente es lo observado de lo esperado al igual que la validez de cada una de las partes.

Ahora, las aplicaciones, en cierta medida, con el fin de llevar el registro de los datos enviados, consumen el mensaje JSON visto en la figura 4. Este ha sido deserializado y mapeado



en *SCMessage*. De esta manera, se podrá trabajar de manera sencilla en el procesamiento y actualización del estado de las aplicaciones.

De este modelo, se desarrolló e implementó una librería, apodada *StarDuck*<sup>8</sup>. Esta librería contiene todas las estructuras de datos definidas en el modelo, las cuales serán usadas por el resto de los servicios y aplicativos necesarios para la declaración, evaluación y adaptación de las arquitecturas de software en el proyecto.

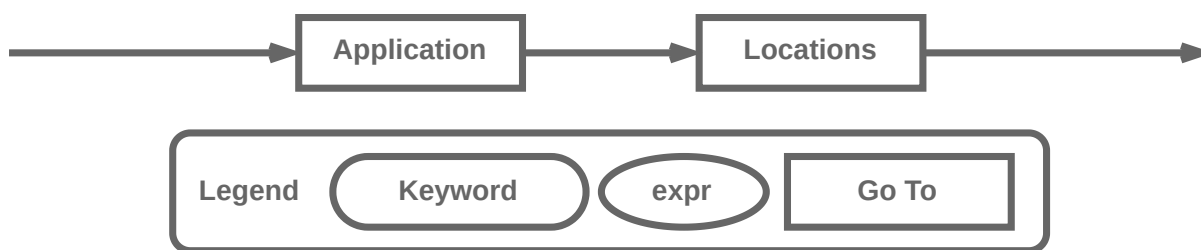
## 6.5 Sintaxis de la notación

Partiendo de esto, lo siguiente que se realizó fue la definición de la sintaxis de la notación a usar, basados en lo definido por el metamodelo. Para esto, se decidió usar **YAML**, un lenguaje de serialización de datos orientado a la legibilidad, reconocido y usado principalmente para la creación de archivos de configuración (Red Hat Foundation, 2023).

La sintaxis, como puede observarse en la figura 6, se compone de 2 partes: **Application**, donde se declara la aplicación; y **Locations**, en donde se define el contexto geográfico al igual que las necesidades de datos de estas.

**Figura 6:**

Diagrama de rail de la sintaxis definida para la notación de SmartCampusADL



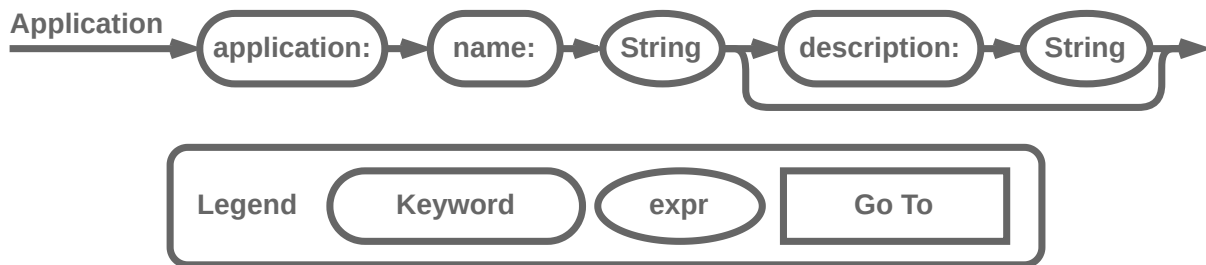
Esta primera parte, como puede observarse en la figura 7 se refiere al nombre que se usará para referirse a la aplicación al rededor de todos los servicios, y una descripción opcional con fines de documentación.

---

<sup>8</sup>El código fuente de la librería puede encontrarse en <https://github.com/ChipDepot/StarDuck>, y su respectivo paquete en crates.io, en <https://crates.io/crates/starduck>.

**Figura 7:**

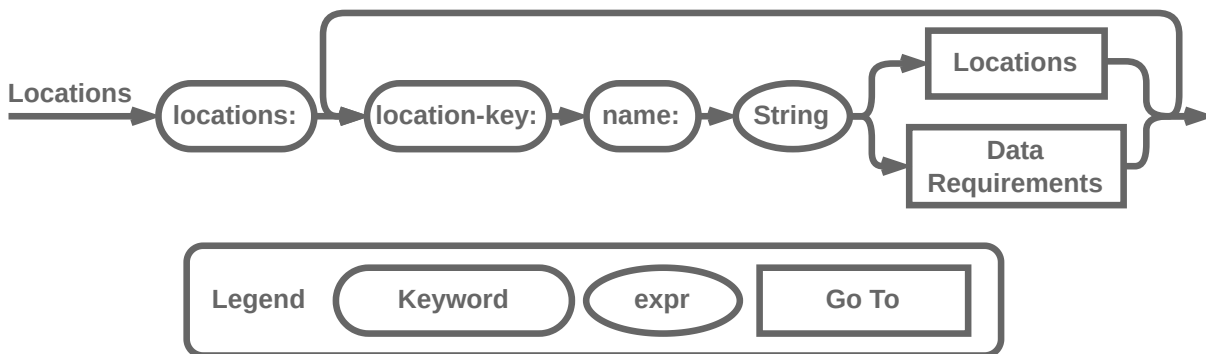
Diagrama de rail de la sintaxis de declaración de la aplicación



Para especificar las **locaciones** dentro del contexto de la aplicación, se definió la sintaxis que puede observarse en la figura 8. Partiendo de la locación raíz de la aplicación, podrán declararse  $n$  cantidad de locaciones, cada una con un nombre diferente.

**Figura 8:**

Diagrama de rail de la sintaxis definida para la notación del contexto geográfico de la aplicación



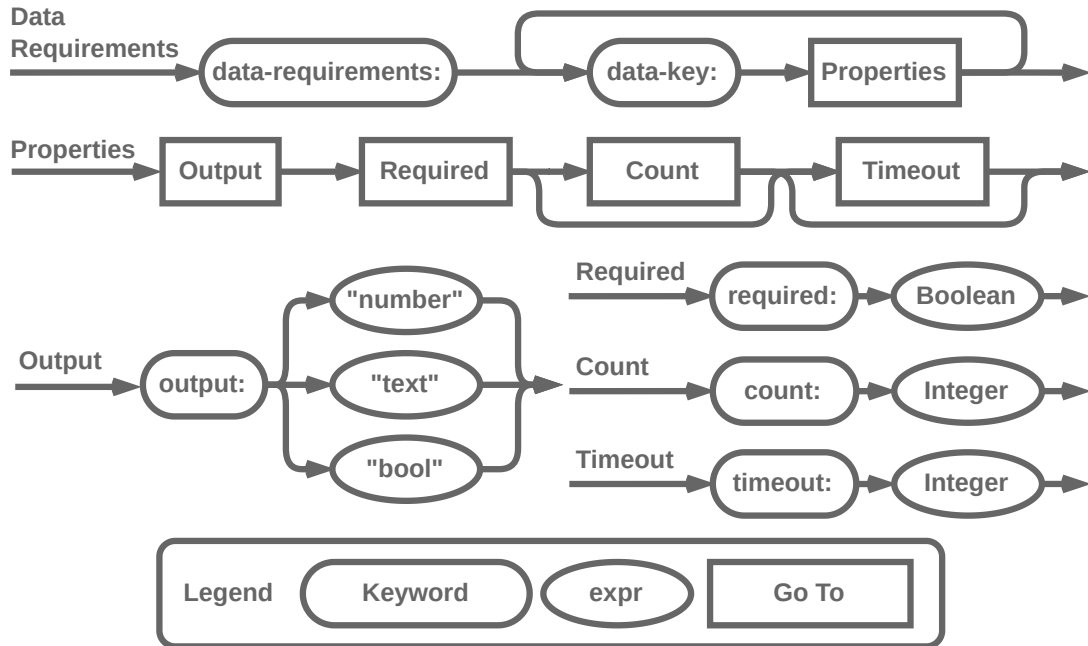
Las locaciones pueden funcionar de una de dos maneras. La primera es como agrupadores de otras locaciones, que puede verse como varios edificios de un campus o como varios pisos de un edificio, dependiendo de la granularidad que requiera la aplicación. Y, la segunda son puntos de origen de datos requeridos por la aplicación, reportados por diversas clases de dispositivos presentes.

Los requerimientos de datos, como se puede ver en la figura 9, están compuestos de varias partes. Las **data-key** se refieren al nombre del requerimiento de dato de la locación. Se espera que estos sean los mismos usados en la parte de **values** vista en los mensajes de

la plataforma Smart Campus de la figura 4.

**Figura 9:**

Diagrama de rail de la sintaxis definida para la notación de los componentes de la aplicación



A continuación, se procedería a definir las propiedades de dicho requerimiento de datos. Para esta primera versión de la notación, se han establecido las 4 cuatro propiedades vistas en el modelo (ver figura 5), dos son obligatorias para la declaración. La primera es **output**, que hace referencia al tipo de dato esperado <sup>9</sup>, y la segunda es **required**, que especifica si el requerimiento es obligatorio u opcional. Las otras dos propiedades opcionales: **count**, que representa la cantidad de fuentes de datos necesarias en la ubicación, y **timeout**, que indica el tiempo máximo entre los informes de datos. En ambos casos, de no declararlas, se tomaría como valor por defecto 0.

El resultante de esta notación serían archivos de declaración de arquitecturas, en formato YAML, similares a la que se puede observar en la figura 10.

Con esto hemos creado un marco sólido para representar las aplicaciones de Smart Campus UIS. Este enfoque bien definido, permitirá avanzar en el desarrollo de las funcionalidades

<sup>9</sup>Este output, en el modelo es el *enum IoTOutput*, que representa los tipos de datos.

**Figura 10:**

YAML de una posible aplicación de monitoreo

```
application:
  name: Chip
  description: "Test App"

locations:
  campus-central:
    name: "Campus Central"
    locations:
      laboratorios-pesados:
        name: "Laboratorios Pesados"
        data-requirements:
          temperature:
            output: number
            required: true
            count: 1
            timeout: 30
          co2:
            output: number
            required: false
            count: 1
```

para comparación de los modelos, y la implementación de los mecanismos de adaptación a usar.

## 6.6 Implementando Una Validación

Partiendo de la notación definida para la declaración de las arquitecturas de referencia, se realizó la implementación de un cliente que permitiera realizar la validación de los archivos de configuración.

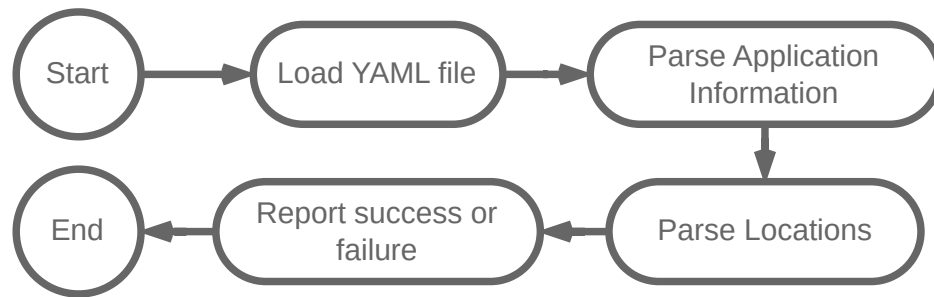
Este cliente, apodado *Lexical*, tiene como objetivo el tomar dichos archivos, y deserializarlos en las diferentes estructuras de datos definidas en el modelo, validando que estas cumplan con la sintaxis definida y que contengan los valores esperados. El grueso de este proceso se puede ver en la figura 11.

Las 2 primeras partes del proceso son bastante directas. Lo primero es cargar el archivo indicado, validando que sea de tipo YAML; y tomar los datos del manifiesto de la aplicación, en este caso el nombre y la descripción.

Seguido de esto, para la deserialización de las locaciones, se tendrán que recorrer cada

**Figura 11:**

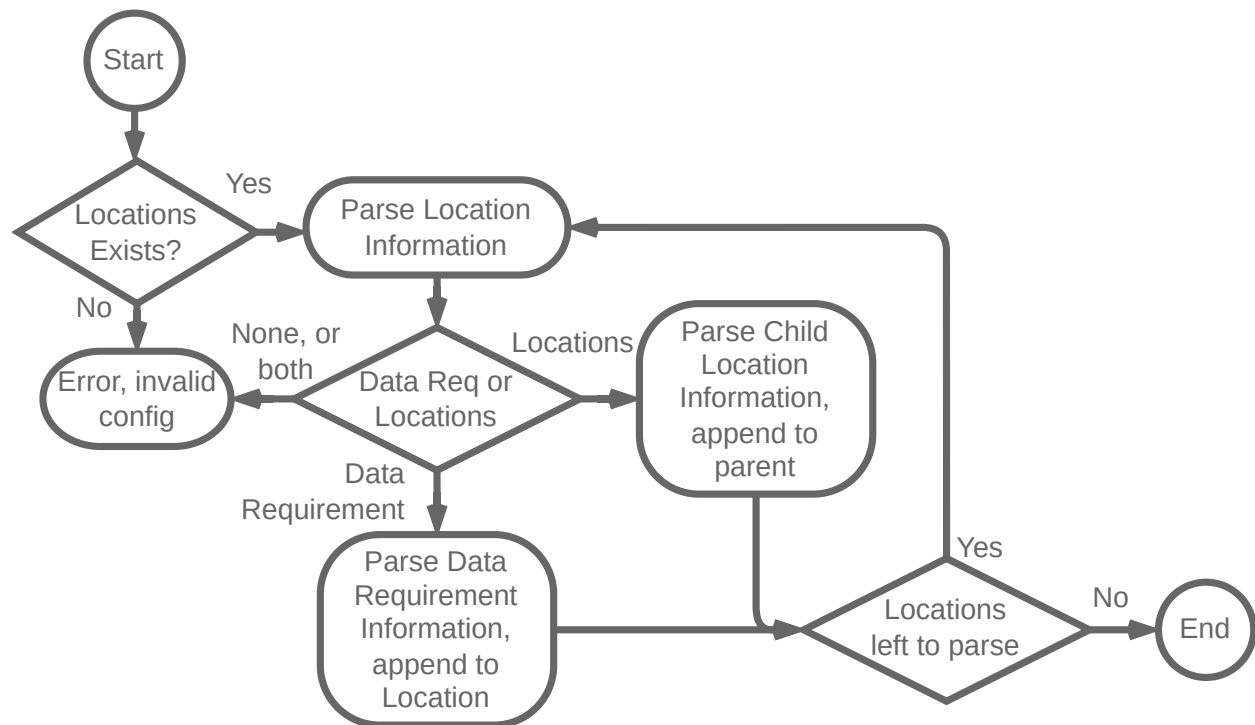
Diagrama de flujo del proceso realizado por el módulo *Lexical*



una de las llaves presentes, procesando los valores internos de la locación. Como se estableció anteriormente, estas pueden ser, o más locaciones, hijas de la locación superior; o requerimientos de datos. El diagrama de dicho proceso se puede ver en la figura 12.

**Figura 12:**

Diagrama de flujo para validación y procesamiento de las locaciones

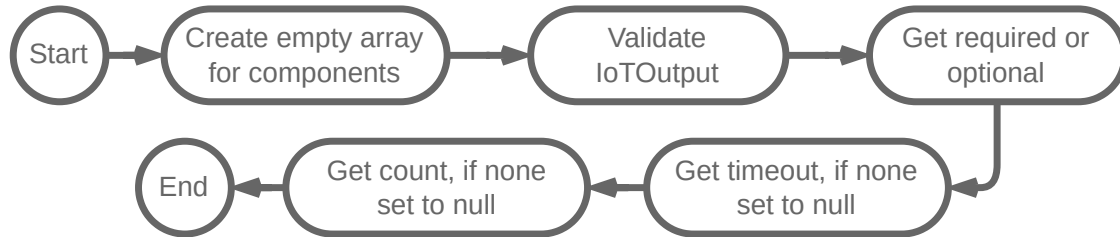


Este proceso se realiza para cada una de las locaciones registradas, lo que da como resultado el contexto geográfico en el que trabajaría la aplicación. Ahora, en cuanto al proceso de la validación de los requerimientos de datos, como se ve en la figura 13, es bastante directo

en cuanto se toman las propiedades, definidas en el metamodelo; con la particularidad de crear un array vacío para los componentes. Esto se debe a que, en el momento de declarar la arquitectura, no conocemos ninguno de los datos necesarios para crearlos. Siendo así, estos se crearan durante la ejecución, a partir de los mensajes de los dispositivos.

**Figura 13:**

Diagrama de flujo del procesamiento de los requerimientos de datos



El desarrollo de este módulos, se realizó en Rust. Escogido debido a su capacidad para garantizar la integridad de los datos y prevenir errores comunes, lo que es esencial en un entorno donde la precisión y la confiabilidad son críticas. Además, su ecosistema de herramientas y bibliotecas facilita la implementación eficiente de los módulos de validación y construcción de modelos.<sup>10</sup>

Con la implementación de este módulo de validación, hemos completado la primera fase del desarrollo de una notación propia para describir y una herramienta para validar arquitecturas de aplicaciones de Smart Campus. Ahora podemos avanzar en la implementación de las funcionalidades de comparación y adaptación de modelos, que son parte fundamental del enfoque a computación autónoma.

## 7 Estado Actual Frente al de Referencia

### 7.1 Conociendo el estado actual

Con la notación de las arquitecturas objetivo establecidas, al igual que el desarrollo del módulo encargado de la construcción y validación de los archivos de configuración; lo si-

<sup>10</sup>El código fuente de Lexical puede encontrarse en el repositorio de GitHub: <https://github.com/ChipDepot/Lexical>

guiente era la definición del proceso de la comparación entre la arquitectura actual y la arquitectura de referencia. Este proceso, permitirá evaluar el estado del sistema y, por consiguiente, establecer las acciones a tomar con el fin de adaptar la arquitectura hacia el estado objetivo establecido.

Esto requiere conocer el estado actual del sistema, al igual que el conocer el estado objetivo. Siendo así, y ya teniendo la posibilidad de declarar las necesidades de la aplicación, lo siguiente es establecer una manera de determinar el estado del sistema. Dado el enfoque hacia los datos recolectados, es necesario el precisar la manera en la que conoceríamos en qué estado se encuentra el sistema.

Lo primero era el identificar los puntos de acceso por los cuales podríamos acceder a los datos. En el caso de Smart Campus UIS, y teniendo en cuenta que el modelo que se definió depende de los mensajes enviados por los dispositivos, como se observa en la figura 14, existen dos opciones para procesar estos mensajes.

La primera, es consultar los mensajes enviados usando uno de los endpoints del servicio `data_microservice`, lo que da acceso a todos los mensajes registrados. Y, la segunda, usando el mismo bus de datos, empleado por los servicios de la plataforma, lo que permitiría el acceso, incluso de manera más directa, a los mensajes a medida que estos son enviados.

Ahora, cada una de las maneras de acceder a los datos es viable, y podría permitir la implementación correspondiente para determinar el estado del sistema. Sin embargo, de entre las dos opciones, se escogió el procesamiento de los mensajes enviados por los dispositivos, enviados por MQTT.

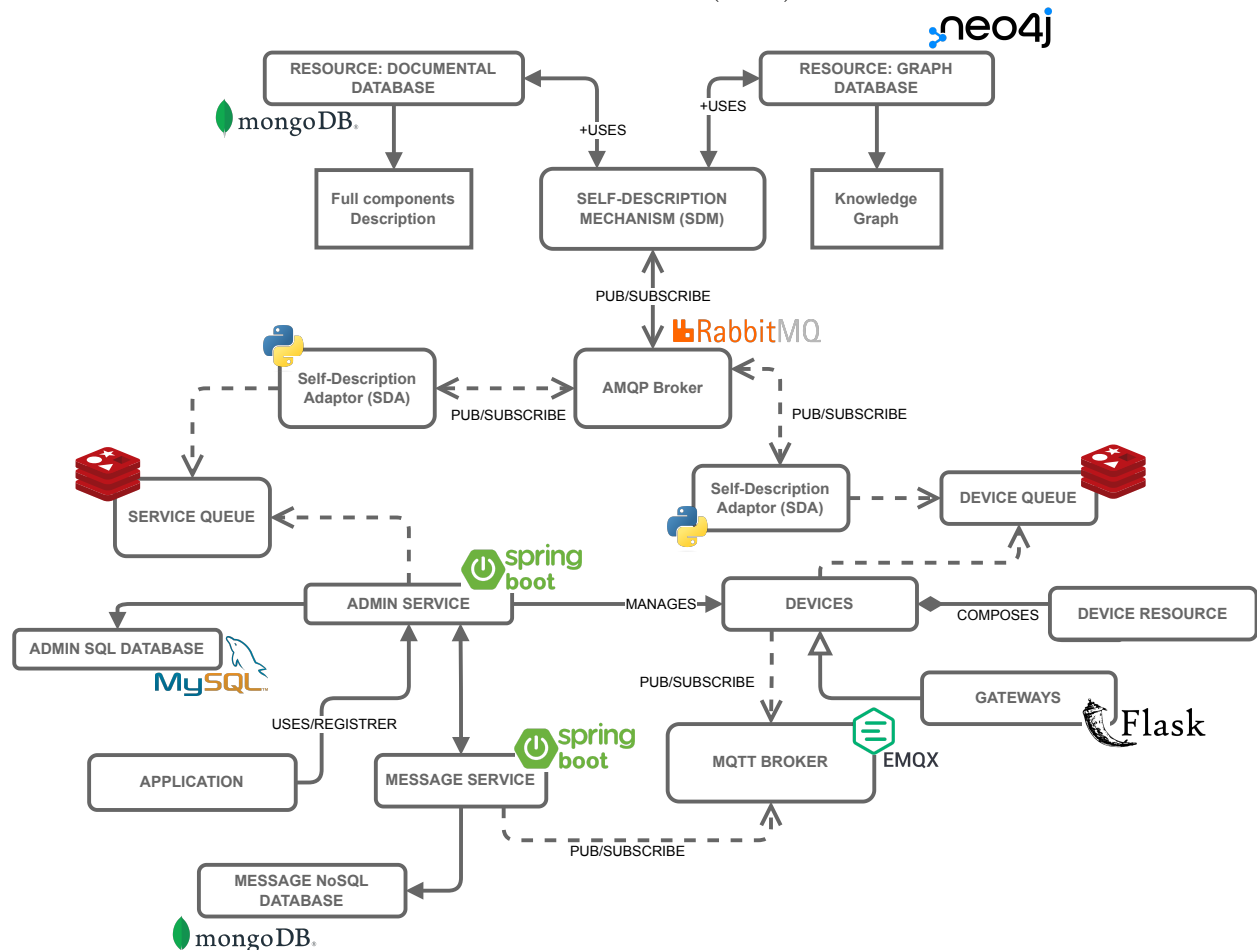
Esto se debe a varios factores, principalmente, debido a cómo se consultan los mensajes desde el endpoint. Para realizar esta consulta, es necesario conocer el *UUID* del dispositivo a consultar lo que, como se evidenció durante la validación de los requerimientos de datos en la sección 6.6, no es posible.

Aunque sería posible el realizar una modificación al servicio para poder ver todos los mensajes de los dispositivos, esto también podría presentar problemas por la manera en la

**Figura 14:**

Arquitectura del prototipo de Smart Campus definido por

Jiménez Herrera (2022)



que estos se listan. Ya que los mensajes se muestran todos en simultaneo, y no poseen una manera diferenciar 2 mensajes, más allá de su contenido (que no es necesariamente único); tendríamos que guardar todos los mensajes ya procesados y descartarlos cada vez que se consulte el endpoint, lo que generaría un *overhead* el uso de los recursos tanto de memoria como de procesamiento.

Siendo así, queda como opción el broker MQTT para la consulta de los mensajes. Este acercamiento permite el procesamiento, en tiempo real, de los mensajes enviados por los dispositivos, sin la necesidad de conocerlos; al igual que un aprovechamiento de los recursos ya que, una vez procesados, podemos librar el espacio usado. Esto le da una característica al



proyecto en forma de una especie de *plug-in*, puesto que, al no requerir modificaciones sobre la plataforma, esta puede funcionar sin ninguna afectación en su estado actual.

Partiendo de lo anterior, se realizó el diseño e implementación de un servicio encargado de procesar los mensajes que están siendo enviados por, y a, cada uno de los dispositivos registrados en Smart Campus UIS.

## 7.2 Centralizando los Datos

Lo primero para la implementación del observador, era determinar como este tendría acceso al estado de referencia. Hasta el momento, únicamente *Lexical* tiene el contexto definido por el usuario, por lo que era necesario definir una manera mediante la cual otros servicios puedan acceder a este.

Partiendo de la división de responsabilidades, se estableció el implementar un agregador, que contenga el estado definido por el usuario; desde el cual otros servicios puedan acceder a estos datos.

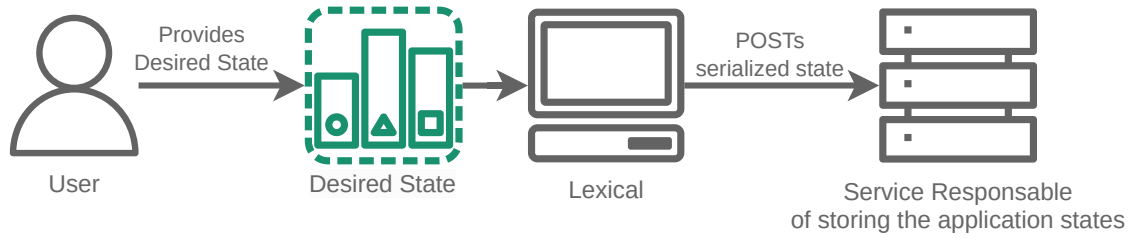
Esto tiene dos ramificaciones adicionales. La primera, es que abre a la posibilidad de declarar más de una aplicación en simultaneo, usando el agregador como un *hub* que permita guardar y consultar los estados de las aplicaciones que se registren, teniendo una instancia de observador por cada una de ellas.

La segunda, posibilita el usar este agregador para almacenar los estados actuales reportados por el observador. Esto se debe a que, las condiciones para evaluar el estado de las aplicaciones, se encuentran en las propiedades definidas por el usuario durante la declaración del estado de referencia. Siendo así, es posible ver el estado objetivo a partir de las propiedades; y el estado actual basado en la evaluación de los componentes registrados en los requerimientos de datos.

De lo anterior, podemos planear un diagrama, visto en la figura 15, que refleje el estado actual de la arquitectura del proyecto.

**Figura 15:**

Arquitectura actual del proyecto con el agregador



La implementación de este agregador, apodado *Bran*<sup>11</sup>, fue relativamente directa. Este tiene dos requerimientos, el primero, es la capacidad de recibir, guardar y actualizar los estados de las aplicaciones; y segundo, el poder enviar los estados almacenados cada vez que estos se soliciten.

Ambas de estas funcionalidades se implementaron usando el framework *Axum*<sup>12</sup>. Usando como base la librería *StarDuck*, se definió un único endpoint que toma, `apps/:app_name`, que acepta peticiones de tipo `GET`, para consultar el estado de la aplicación; `POST` y `PUT`, para el registro inicial de la aplicación, y posterior actualización de esta.

Ya con el servicio implementado se agregó, a *Lexical*, un cliente HTTP con el cual, tras validar la arquitectura objetivo proveída por el usuario, y como se ve en la figura 15, este realice una petición de tipo `POST` a la una instancia de *Bran* indicada con el fin de registrarla. Una versión actualizada del proceso realizado *Lexical* puede verse en la figura 16.

### 7.3 Implementado un Event-Handler

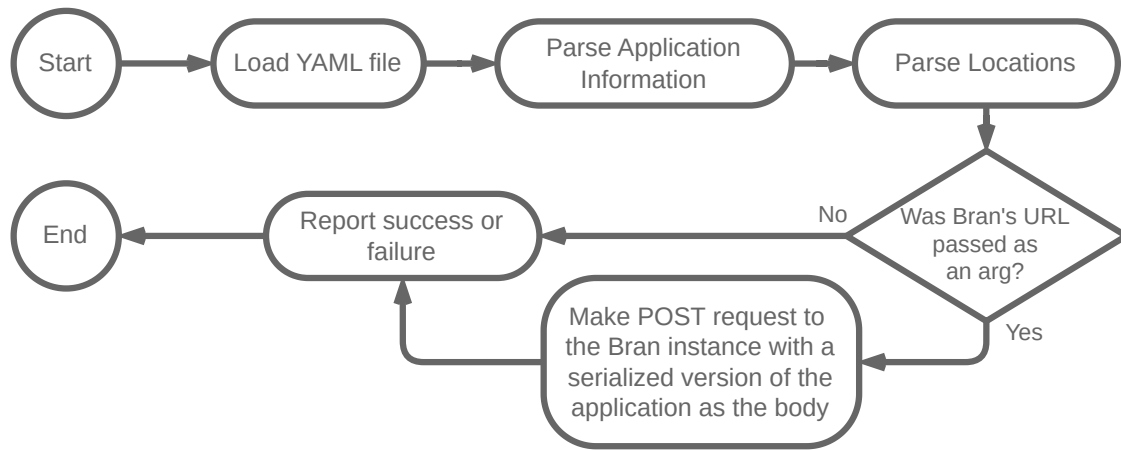
En el contexto de Smart Campus UIS, cada uno de los mensajes enviados por los dispositivos es manejado como un evento. Es decir, cada mensaje recibido debe ser registrado y procesado por los microservicios de administración para lograr un objetivo final. Este procesamiento, recuerda al patrón de diseño *Observer* en el cual se busca el realizar una acción cada vez que el estado, de lo que se está observando, cambia (Shvets, 2019).

<sup>11</sup>El código fuente de *Bran* puede encontrarse en el repositorio <https://github.com/ChipDepot/Bran>

<sup>12</sup>*Axum* es un framework modular para aplicativos web Open Source desarrollado en Rust. Más información sobre este puede encontrarse en su repositorio <https://github.com/tokio-rs/axum>

**Figura 16:**

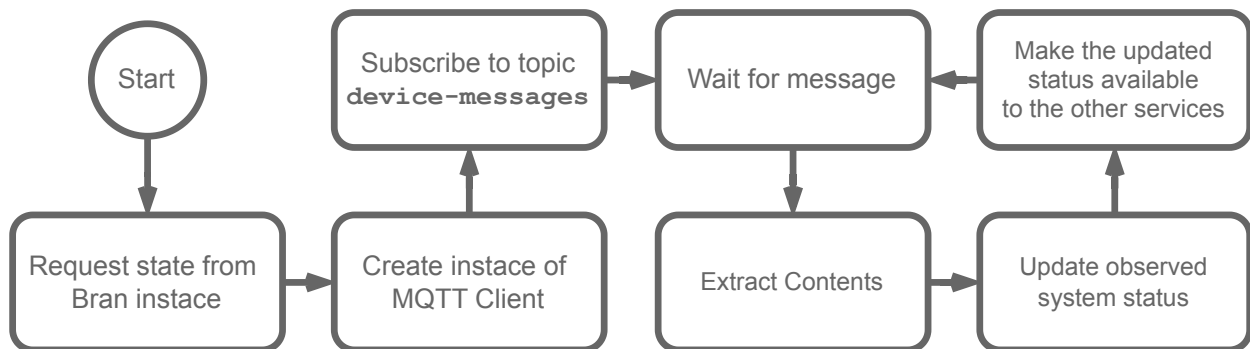
Diagrama de flujo del proceso realizado por Lexical actualizado



Siendo así, se estableció el realizar la implementación de un observador el cual se hará responsable de la captura de los mensajes enviados por los dispositivos y establecer el estado del sistema. De esto, como se puede ver en la figura 17 se propuso un proceso que debía realizar el observador, apodado *Looker*, a implementar.

**Figura 17:**

Primera propuesta del proceso a realizar por *Looker*

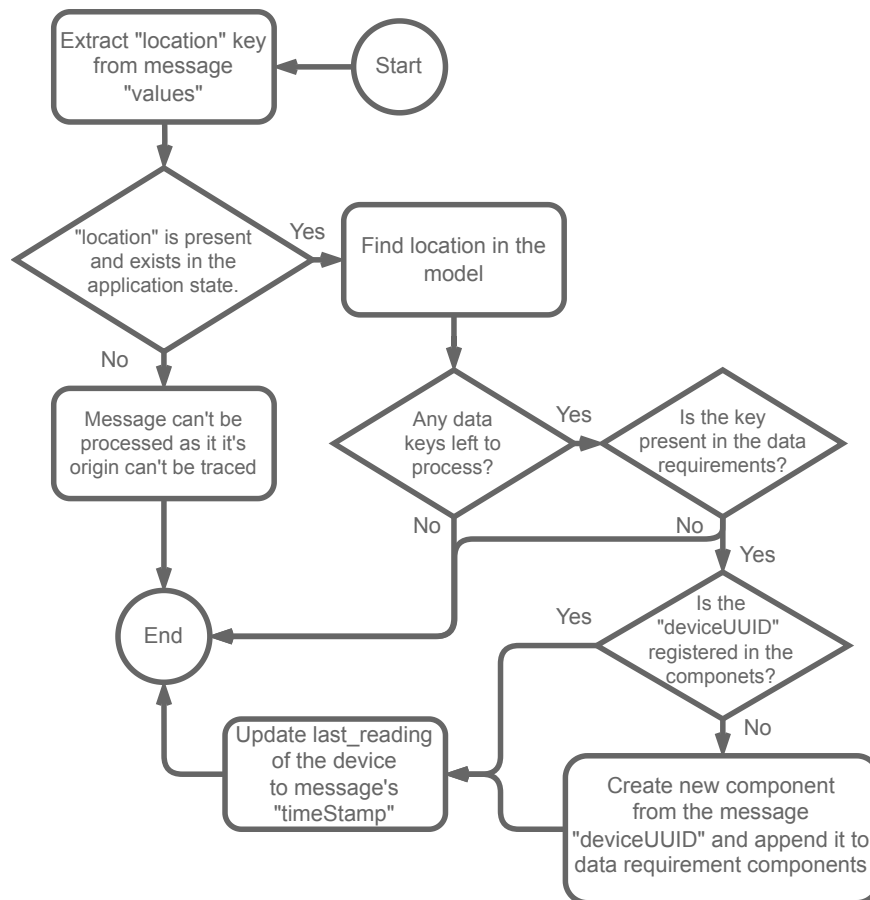


Este primer acercamiento, es bastante directo en cuanto a la implementación a realizar. Tras adquirir la definición del estado objetivo, se inicializa un cliente de MQTT, conectado el broker usado por Smart Campus UIS y suscrito al tópico usado por los dispositivos, `device-messages` (Jiménez Herrera, 2023), este empezaría a procesar los mensajes a medida que estos van llegando.

Cada mensaje recibido se somete a un conjunto de pasos para actualizar el estado de la aplicación. La figura 18 describe el proceso que el observador debe realizar con el fin de procesar los mensajes recibidos, dando como resultado, una versión actualizada del estado de la aplicación.

**Figura 18:**

Proceso realizado por el observador durante el consumo de los mensajes enviados por los dispositivos



Lo primero a realizar, es ubicarse en el ubicación origen del mensaje. Esto se realiza buscando en las locaciones registradas en la aplicación, usando la llave (Ver figura 4) presente en el mensaje enviado por el dispositivo. En caso que esta, no esté presente, o la locación no haya sido referenciada en la declaración del estado objetivo, el mensaje será descartado. Se debe resaltar que, este puede ser uno de los principales puntos de falla debido a los posibles errores que se puedan presentar al usar nombres diferentes para una locación, sea en los

mensajes enviados por problemas en la configuración; o en el momento de la creación del estado de referencia.

Una vez localizado el origen geográfico del mensaje, se recorrerán los datos reportados por los dispositivos. Estos se cruzarán contra los requerimientos de datos, actualizando los tiempos de los componentes; o registrándolos en caso de que sea el primer mensaje publicado por ellos. Esto se realizará para todas los datos enviados por el dispositivo.

## 7.4 Comparando Arquitecturas

Ahora que se conoce el estado actual del sistema, al igual que estado de referencia; lo siguiente a realizar era la comparación de las arquitecturas. El resultado de esta, daría la base para poder decidir el qué hacer para adaptar el sistema y el cómo hacerlo.

Es necesario definir el como se realizará la evaluación del sistema. Esto no es tan sencillo como hacer una comparación usando un igual ( $A == B$ ), ya que, el realizar esto, no resultaría realmente información sobre, qué, internamente, presenta problemas.

Ahora, la propiedades implementadas, definidas en los estados objetivo; son la manera las características principales por las cuales es posible realizar la evaluación del sistema. *Timeout*, cumpliendo la función de establecer el tiempo máximo entre los reportes de dispositivos, definiendo cuando estos pueden considerarse como "vencidos"; y *count*, como la cantidad de dispositivos en estado *Coherent* mínima para el desarrollo de la aplicación.

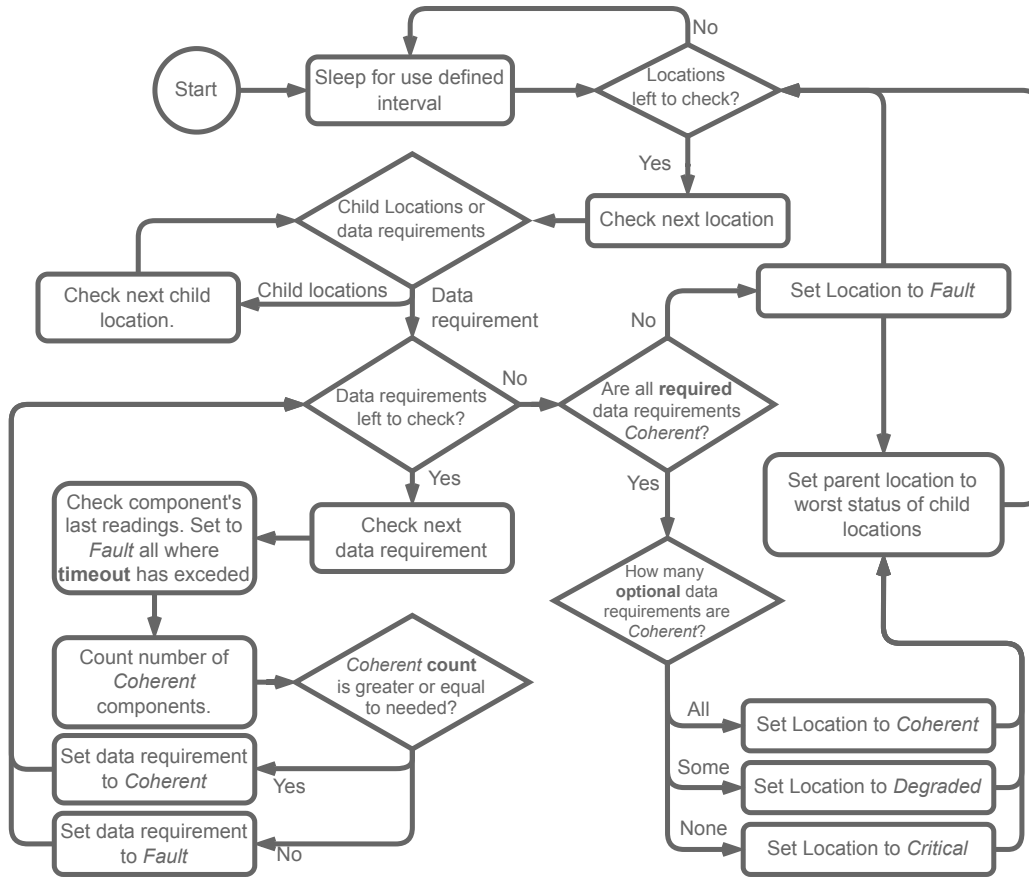
Siendo así, se definió un proceso de comparación consta de evaluar cada componente del sistema en función de las propiedades. A partir de estas, es posible el evaluar el estado de la aplicación. Este proceso es descrito por la figura 19.

El proceso, apodado reloj, debido a su ejecución dado un intervalo de tiempo definido por el usuario; realiza la evaluación de la aplicación, recorriendo la estructura locación por locación. Siendo así, y con el fin de evitar una dispersión grande de los lugares de procesamiento, se implementó en *Looker*.

Los componentes, pueden existir únicamente en un estado binario, sea *Coherent*, en el caso

**Figura 19:**

Proceso reloj realizado por el observador para la actualización del estado de la aplicación



de que cumplan con las condiciones establecidas en el requerimiento; o *Fault* de lo contrario. Estos se basan únicamente en la propiedad *timeout*, marcándolos de manera correspondiente a partir de su último reporte registrado.

Así mismo, los requerimientos de datos, con la misma característica de su estado binario; usan propiedad *count* para definir su condición. En caso de que la cantidad total de componentes sea igual a la requerida, se marcará como *Coherent*; de lo contrario, estará en *Fault*. Se ha de resaltar que, debido a que estas propiedades son opcionales durante la declaración, podrán presentarse casos en los que, dada una configuración específica, con un único reporte (o incluso ninguno) estos podrán considerarse válidos durante todo el de ejecución de la aplicación.

Una vez evaluadas los componentes y los requerimientos de datos, lo siguiente es evaluar

la locación. Esto dependerá del tipo de la locación. Si es un agrupador, su estado será el peor de los estados de las hijas. De lo contrario, si tiene requerimientos de datos, su estado depende de sus requerimientos de datos y del tipo de cada uno de ellos.

El estado de estas locaciones se determina en dos pasos. En el primer paso, se verifica si alguno de sus requerimientos obligatorios está en estado *Fault*. En ese caso, la locación se considera en estado *Fault*, ya que no puede cumplir con los requisitos mínimos de la aplicación. Para las locaciones sin requerimientos opcionales, este será el final del proceso.

El segundo paso, en el caso de locaciones con requerimientos opcionales, el factor determinante será la cantidad de estados *Coherent*. La locación tendrá un estado *Critical*, *Degraded*, o *Coherent* según si todos, algunos o ninguno de estos requerimientos está en estado *Fault*, respectivamente.

Esta manera de evaluar las locaciones permite el tener una idea más clara del estado de cada una de las locaciones, y por consiguiente, el estado de la aplicación. A partir de esta evaluación, se realizarán las diferentes adaptaciones necesarias con el fin de acercar el estado de la aplicación al estado de referencia esperado.

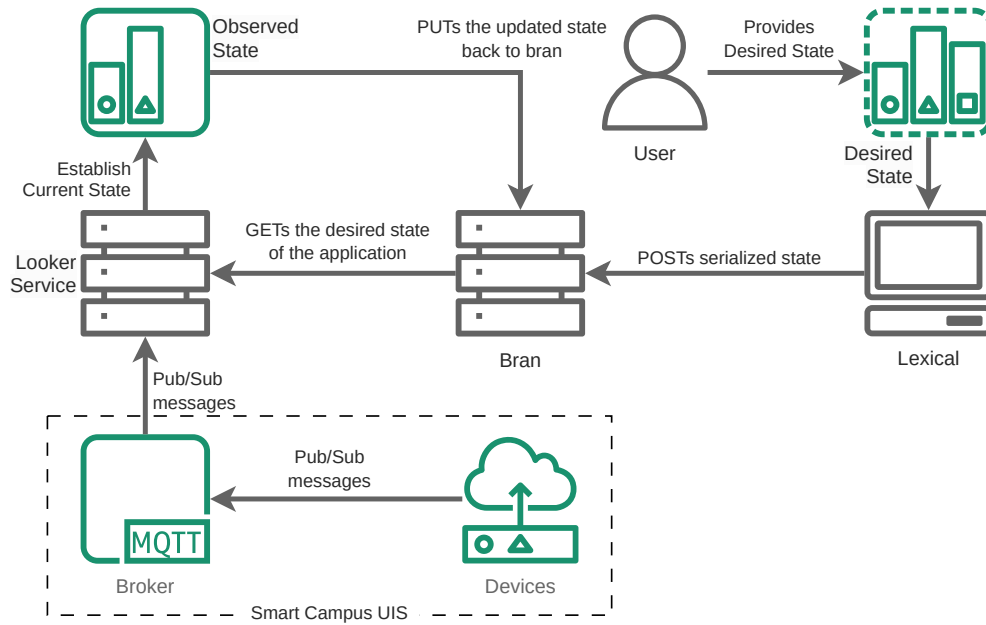
Una vez actualizado, y evaluado el estado; se tendría que reportar al agregador el nuevo estado de la aplicación. Para esto, se agregó un paso final tras la ejecución del proceso reloj, en la cual, *Looker*, realiza una petición tipo PUT al endpoint designado en *Bran*, con el fin de actualizar el registro. La arquitectura del proyecto, hasta el momento, se puede ver en la figura 20.

## 8 Adaptando la Arquitectura

### 8.1 Identificando Los Problemas, Estableciendo Acciones

Ya con una manera de evaluar los estados observados de las aplicaciones definidas, lo siguiente a realizar era el establecer un proceso por el cual se pudieran identificar los problemas en la aplicación, y a partir de esto, establecer las acciones a realizar con el fin de modificar la arquitectura de la aplicación hacia el estado de referencia definido.

**Figura 20:**  
Arquitectura actual del proyecto



Como se estableció durante la sección 7.4, el estado de la aplicación, en términos generales, es determinada por los estados de sus requerimientos de datos. Siendo así, se definió un proceso por el cual, a partir de las condiciones en las que se encuentren estos requerimientos, se determinen las acciones a realizar con el fin de adaptar la arquitectura.

Partiendo de esto, se definió el proceso, descrito por la figura 21, en dos partes. La primera, está encargada de la búsqueda e identificación de los problemas dentro de la aplicación; y la segunda, de manera interna, el determinar las acciones a seguir para adaptar la arquitectura.

La manera en la de que se identifican los problemas es similar a la vista en la figura 19. Se recorren las locaciones, buscando requerimientos de datos. Una vez en una de estas locaciones, se validan los requerimientos y se definen las acciones.

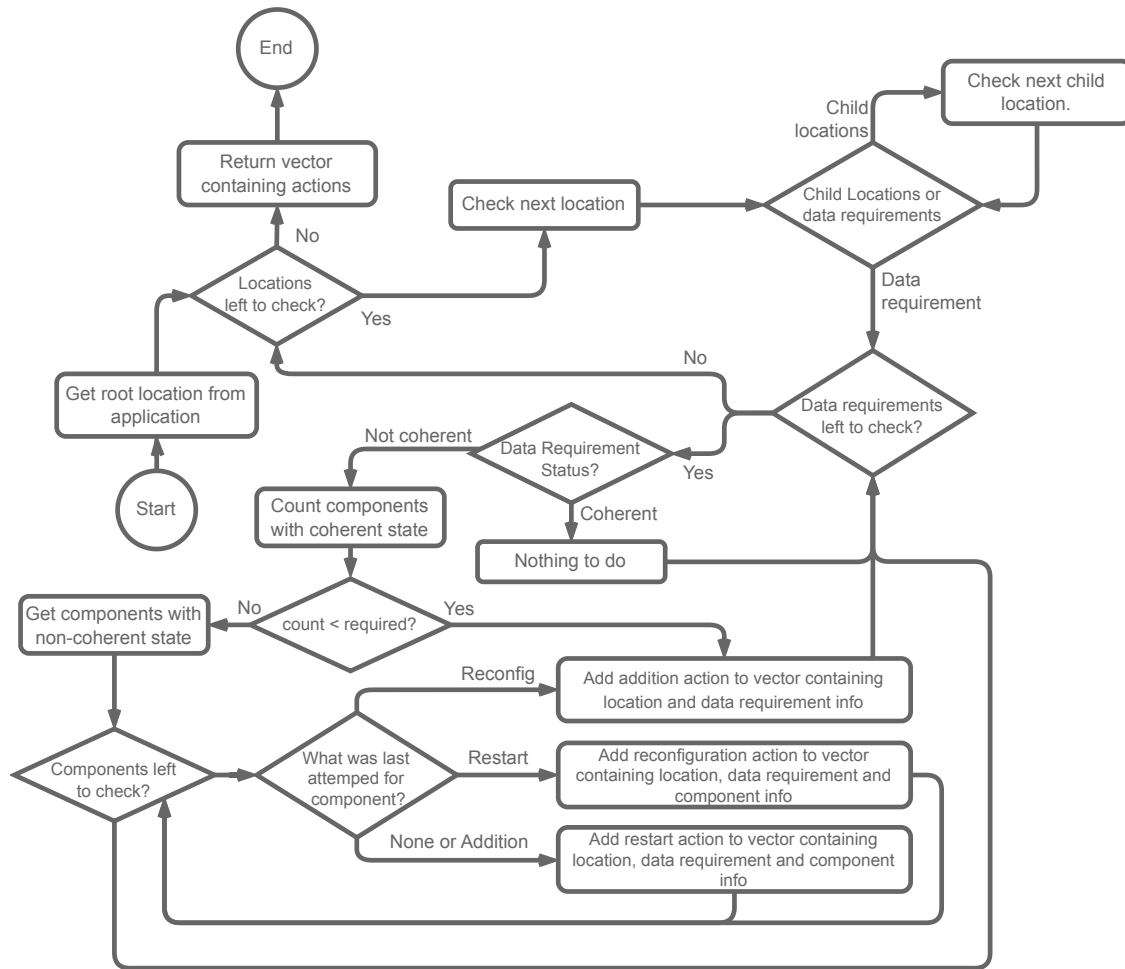
Ahora, las posibles acciones a tomar para la arquitectura de la aplicación, como se vió durante la sección 8.1, dependen de los objetivos y los requerimientos del sistema. Siendo así, se tomaron 3 tipos de acciones por las cuales se puede modificar el estado del sistema: *Addition*, *Restart* y *Reconfigure*.

Todos los requerimientos en un estado diferente a *Coherent*, son evaluados con el fin de



**Figura 21:**

Proceso para la identificación de problemas de los requerimientos de datos



establecer la acción a realizar. Lo primero, es revisar la cantidad de componentes registrados en el requerimiento; si se tiene menos de los requeridos, se realizará un acción tipo *Addition* con el fin de cumplir con el numero de fuentes de datos esperada.

En el caso de tener la cantidad requerida de dispositivos, pero estado aún en estado *Fault*; la siguiente opción es reiniciar el servicio. Esto busca poner nuevamente en un estado válido el componente, en caso de que haya quedado en un estado inválido; o poner nuevamente en ejecución el servicio en el escenario de que este haya muerto.

Si el reiniciar no tiene efecto en el estado del componente, la siguiente opción es reconfigurar el componente. Esto busca la modificación de algún aspecto, sea la actualización una

variable de ambiente o parámetro, con el fin de retornarlo a un estado válido.

Finalmente, si todo falla y no es posible recuperar el dispositivo; la última opción es iniciar un nuevo servicio el cual pueda suplir las condiciones de su predecesor. De esta manera, aún es posible retornar la aplicación hacia un estado válido, independiente de los componentes en un estado irrecuperable.

La implementación de este proceso, se realizó en el agregador *Bran*. Esto debido a que, al tener el estado de las aplicaciones, cortesía de lo observado por *Looker*; deja a este servicio como el punto medio de la aplicación, encargado de definir las acciones a realizar.

## 8.2 De Acciones, a Instrucciones

Ahora, se tienen las acciones a realizar para poder adaptar la arquitectura de la aplicación hacia el estado de referencia; sin embargo, estas acciones no contienen la información necesaria para ejecutar ninguno de los procesos necesarios.

Partiendo de esto, se determinó la necesidad de una colección estructuras de datos, que contuvieran las instrucciones a seguir, para cada una de las acciones definidas. Esta, debe especificar la locación de la aplicación en la cual se ejecutan, con el fin de tener una mejor granularidad y control sobre la manera en la que se realizan las modificaciones en la arquitectura.

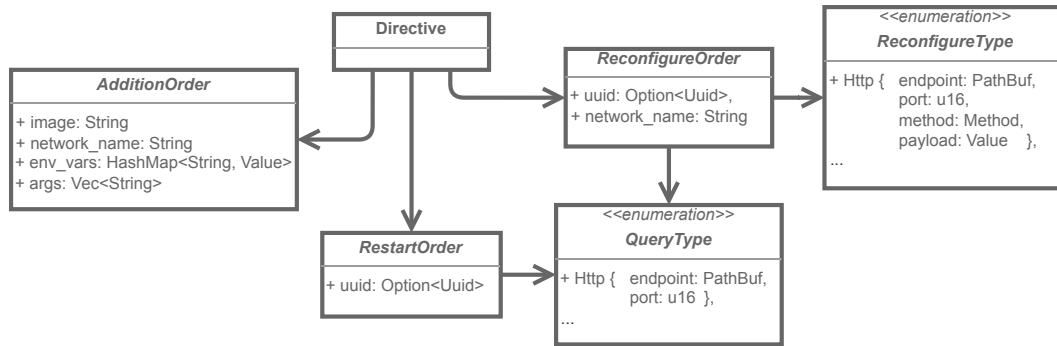
En consecuencia, como se observa en la figura 22, se definió una estructura de datos llamada *Directive*. Esta contiene tres campos adicionales para las estructuras *AdditionOrder*, *RestartOrder*, y *ReconfigureOrder*; que corresponden con las acciones *Addition*, *Restart* y *Reconfigure*, respectivamente.

Cada una estas directivas, y ordenes, se agregan a un registro en *Bran*, usando el nombre de la aplicación y la locación a la pertenecen para su indexación. Cada una se registra usando un endpoint dedicado a la recepción de las órdenes.

La orden de adición contiene el nombre de la imagen del servicio a desplegar, la red a la cual debe conectarse para poder interactuar con los demás servicios, y los argumentos que

**Figura 22:**

UML de la estructura de directivas y ordenes



se deben pasar sea como variables de ambiente o argumentos de línea de comando.

Las órdenes de reinicio y reconfiguración poseen el *UUID* del dispositivo que buscan afectar. Siendo así, al ser necesario tener una manera por la cual identificar qué servicio es qué componente, se implementó el enum *QueryType*. Este, contiene la información de como debe buscarse el contenedor en la red. Para esta primera implementación, su única variante es una búsqueda *Http* hacia un endpoint y un puerto.

Finalmente, la orden de reconfiguración tiene como parte extra la red en la que se busca el servicio a reiniciar y el tipo de reconfiguración (*ReconfigureType*) a realizar. Este funciona de manera similar a *QueryType*, en tanto indica la manera en la que se aplicará la reconfiguración. En este caso, para la variante *Http* implementada, indica el endpoint y el puerto a usar, junto con el método y payload que deben ir en la petición para hacer la reconfiguración efectiva.

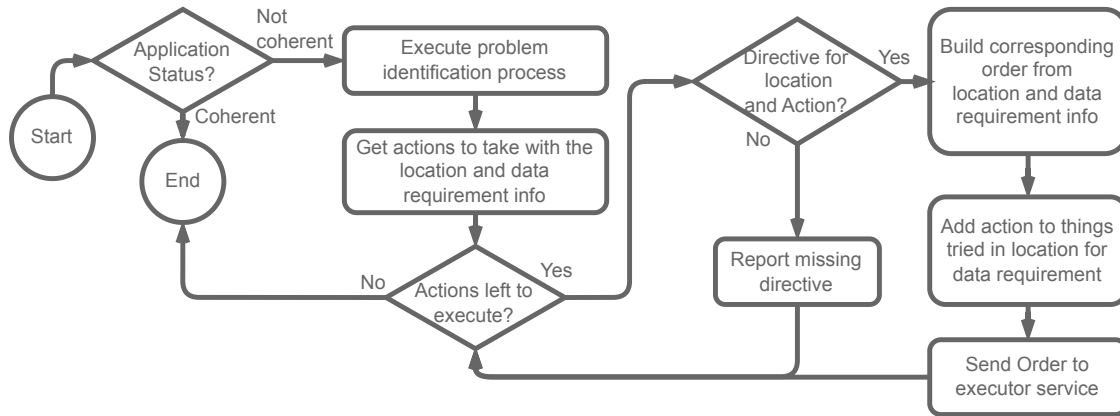
Se ha de resaltar que, estas ordenes, aunque contienen las instrucciones para poder realizar modificaciones a la arquitectura, en el momento de su declaración, no están completas y requieren de información extra para poder ejecutarse.

El proceso de la figura 23, muestra el proceso por el cual las órdenes presentes en el registro de directivas de *Bran*, son usadas como base para la construcción de las órdenes finales, y son enviadas para su posterior ejecución.

Su ejecución inicia validando el estado de la aplicación reportada. En caso de este sea

**Figura 23:**

Proceso de decisión de acciones base realizado por Bran



diferente a *Coherent*, se llamará al proceso de identificación de problemas para obtener las acciones a realizar para alterar el estado de aplicación.

Una vez recibido el vector conteniendo las acciones, y la información asociada correspondiente, se realizará la construcción de cada una de las órdenes a ejecutar. Esta construcción se realiza a partir de las instrucciones bases establecidas en las directivas usando el UUID para las acciones de *Restart* y *Reconfigure*; y la información de la locación y el requerimiento de dato en las acciones de *Addition*<sup>13</sup>.

Ya con las ordenes creadas, estas se enviaron al servicio encargado de ejecutar las instrucciones, dando como resultado, modificaciones en la arquitectura.

### 8.3 De Instrucciones a Acciones

Ya con las instrucciones necesarias para poder realizar cambios en la arquitectura de las aplicaciones, se realizó la implementación de un servicio encargado de ejecutar las ordenes recibidas desde *Bran*.

Ahora, Smart Campus UIS, como plataforma, se ejecuta sobre contenedores de docker; y para efectos de esta primera versión del proyecto, toda la arquitectura de presente, se ejecuta

<sup>13</sup>Es necesario resaltar que estas construcciones, en especial para el caso de las acciones de *Addition*, fueron implementadas para funcionar con los servicios *Mocker* desarrollados específicamente para el proyecto. Esto se explorará más a fondo durante la sección 9 y 11.

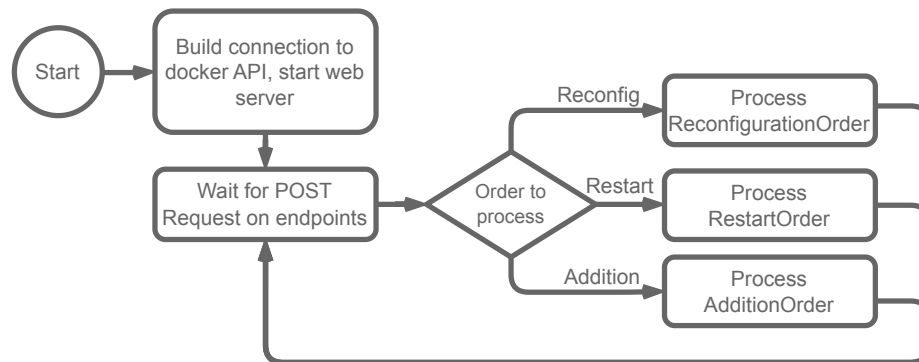
sobre contenedores. Como consecuencia, era necesario que el servicio implementado, tuviera la de ejecutar comandos de docker con el fin de poder llevar a cabo las acciones.

Esto se solucionó usando la Docker Engine API como el canal de comunicación entre el servicio, y el resto de la arquitectura. Esto se debe a que, esta API REST, permite la interacción directa con el daemon de Docker (Docker Inc, 2023), posibilitando la ejecución de todos los comandos relacionados con docker, desde el iniciar y detener servicios hasta crear y eliminar contenedores.

El servicio, apodado *DoThing*, realiza visto en la figura 24. Este está orientado principalmente al procesamiento de las peticiones realizadas, por lo que es relativamente sencillo en cuanto a las acciones que este realiza.

**Figura 24:**

Proceso base de DoThing



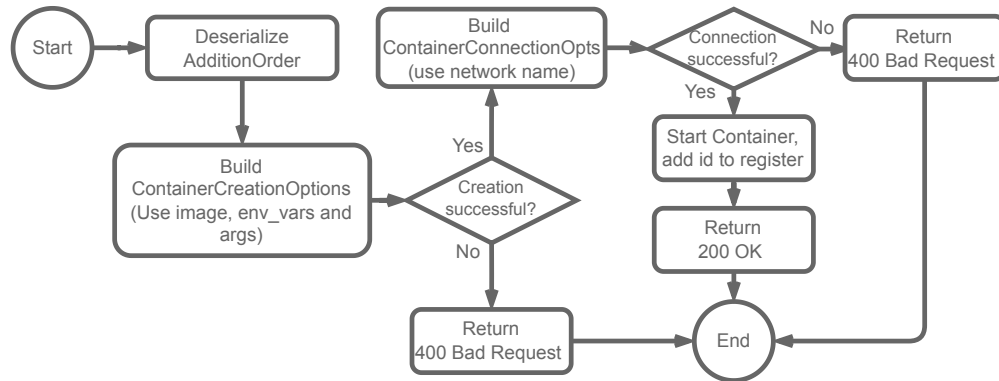
Tras inicializar la conexión con la API de Docker, iniciará un servidor web por el cual estará recibiendo las órdenes a ejecutar. Cada tipo de acción tiene su propio endpoint donde se realiza un proceso específico para poder ejecutarla. Cada acción realizada, registrará los contenedores afectados en un registro interno, con el fin de facilitar el procesamiento de múltiples acciones sobre un servicio.

Las ordenes de *Addition*, como se referencia en la figura 25, realizan el proceso a partir de la `image`, `env_vars` y `args`, declarados en la orden, para construir el contenedor con su respectivo servicio.

Una vez se tenga el contenedor creado, usando `network_name`, se conectará el contene-

**Figura 25:**

Proceso para la ejecución de ordenes de adición

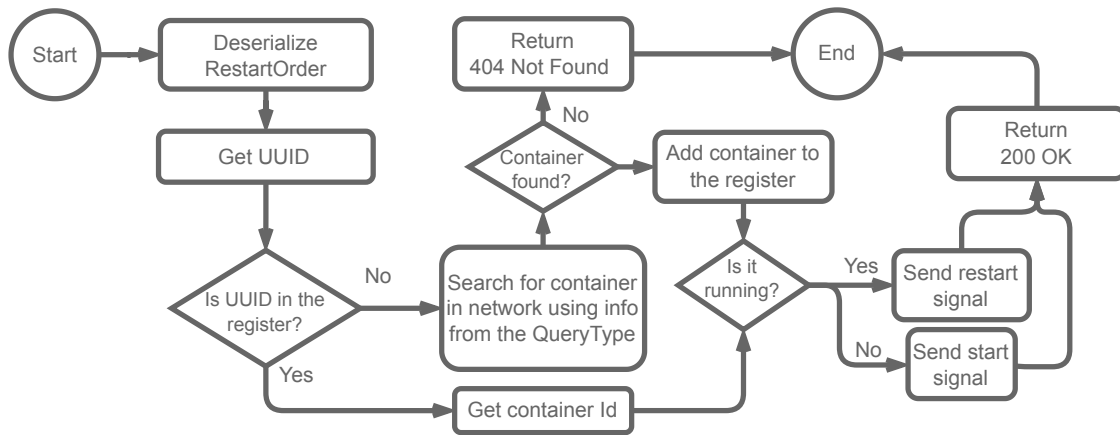


dor creado a la red indicada. De no presentarse errores durante este proceso, se iniciará el contenedor, se agrega al registro y retorna 200 OK.

Las órdenes de tipo *Restart*, referenciadas en la figura 26 pueden ejecutarse de una de dos maneras, dependiendo del estado del servicio a afectar.

**Figura 26:**

Proceso para la ejecución de ordenes de reinicio



Primero, se realiza la búsqueda del contenedor en el registro; de no estar presente, se realizará la búsqueda de este usando la información proveída en el *QueryType* de la orden.

Una vez identificado el servicio, si está corriendo<sup>14</sup>, la orden de reinicio será equivalente al comando `docker restart <container>`, esto con el objetivo de eliminar *cuelgues* en

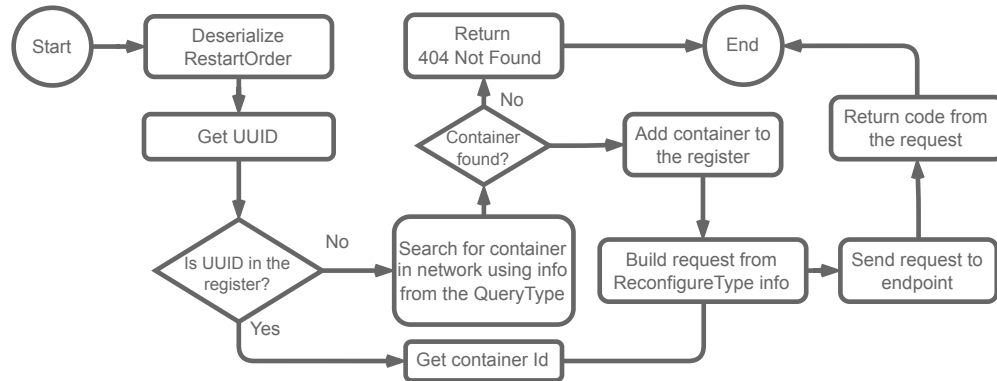
<sup>14</sup>Referido en la documentación de Docker Engine API como *Running*

el servicio. Por el contrario, si el contenedor no está en ejecución, la orden ejecutará el equivalente al comando `docker start <container>`, para así iniciar nuevamente el servicio.

Finalmente, las órdenes de tipo *Reconfigure*, cuyo proceso puede verse en la figura 27, es similar al realizado para las órdenes de tipo *Restart*.

**Figura 27:**

Proceso para la ejecución de ordenes de reconfiguración



Una vez ubicado el servicio, se construirá, a partir de los datos reportados en *ReconfigureType*, la petición a realizar en el servicio. Esta, en consecuencia, se enviará usando un cliente *Http* y esperará a la respuesta de esta. El resultado, de esta transacción, será el mismo reportado por *DoThing* hacia *Bran*.

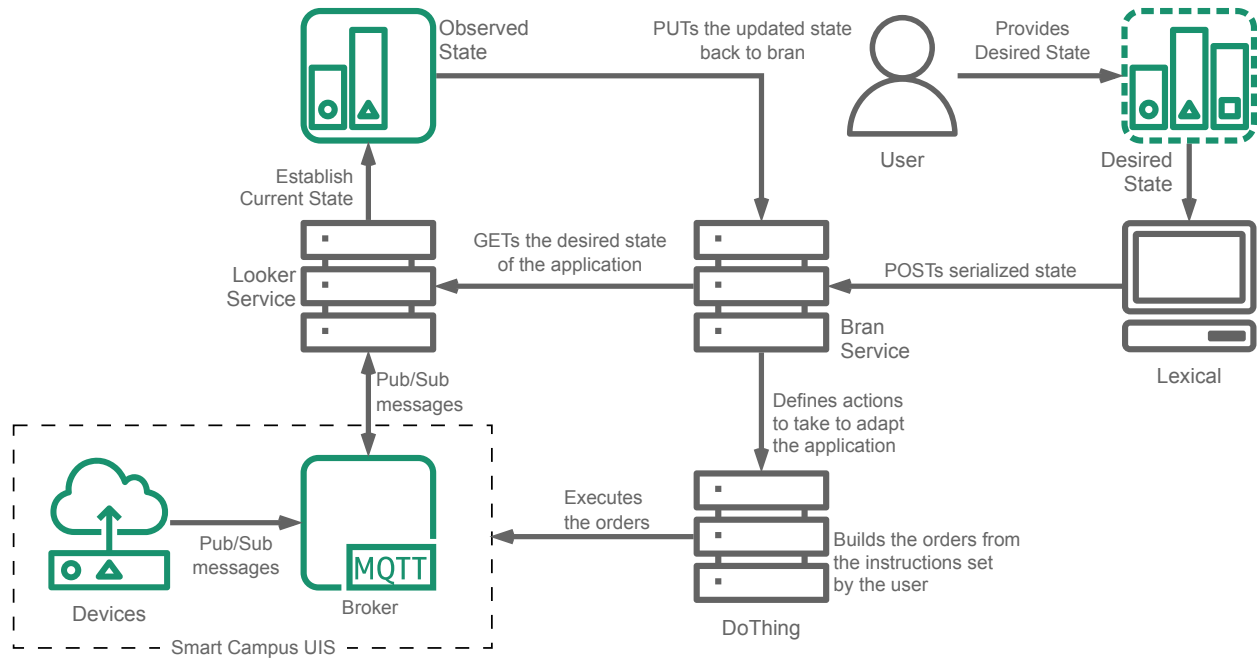
Ahora, con la implementación de este servicio, encargado de ejecutar las acciones requeridas para la adaptación de las arquitecturas de las aplicaciones, completa el ciclo autonómico MAPE-K implementado para Smart Campus UIS.

Una vez ejecutadas las órdenes; el efecto de estas, serían captadas por el observador, evaluando nuevamente el estado de la aplicación, e, idealmente, reportando un cambio positivo hacia la coherencia esperada con el estado de referencia definido en un principio.

La arquitectura final del presente proyecto, con toda la implementación de los servicios, puede ver se en la figura 28.

**Figura 28:**

Arquitectura completa del proyecto



## 9 Validación de la Implementación

Lo último a realizar fue la validación de las funcionalidades de la aplicación desarrollada. Para ello, se establecieron escenarios de prueba por los cuales se pueda comprobar el funcionamiento de aplicación. Estos, se definieron con el fin de probar la capacidad y la efectividad de los mecanismos implementados de modificar el estado inicial de la aplicación hacia el estado de referencia, al igual que la identificación de algunas de las falencias de esta.

Ahora, se debe resaltar que, las pruebas se realizaron usando un servicio genérico, apodado Mocker, encargado de la generación de datos, especialmente desarrollado para funcionar en condiciones ideales.

Para cada una de estas pruebas, se definió un estado de referencia, dadas unas condiciones iniciales, y una serie de eventos los cuales modificarán nuevamente el estado de la aplicación, las cuales tendrán que ser nuevamente manejadas.



## 9.1 Escenario Experimental A

El primer escenario, busca el evaluar la capacidad de la implementación realizada de iniciar toda una aplicación desde cero. Partiendo de esto, se definió la aplicación vista en la figura 29. Esta tiene dos locaciones raíz, con requerimientos de datos iguales en ambas.

**Figura 29:**

Diagrama del escenario experimental A

| Location - North Sector                                                                                                                                                                                                 | Location - South Sector                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Required:<br>+ 1 temperature component<br>reporting every 30 seconds<br>+ 3 wind-speed component<br>reporting every 60 seconds<br><br>Optional:<br><br>+ 3 Particulate Matter components<br>reporting every 120 seconds | Required:<br>+ 1 temperature component<br>reporting every 30 seconds<br>+ 3 wind-speed component<br>reporting every 60 seconds<br><br>Optional:<br><br>+ 3 Particulate Matter components<br>reporting every 120 seconds |

Inicialmente, no habrán dispositivos presentes en ejecución, por lo que el sistema tendrá que identificar la falla, definir las acciones de adición correspondientes, y ejecutarlas para poder suplir con los requerimientos de datos.

Para ello, tomando como referencia la figura 29, se realizó la declaración del estado de referencia, presente en el anexo 1, y, como observa en la figura 30, se validó usando *Lexical*.

**Figura 30:**

Validación de la declaración de referencia realizada para el escenario experimental A

```
INFO lexical] Lexical started
INFO lexical::parsing::parser] Loading file
INFO lexical::parsing::parser] YAML File loaded
INFO lexical::parsing::parser] Creating Application instance
INFO lexical::parsing::parser] Application instance created
INFO lexical::parsing::parser] Creating Locations
INFO lexical::location::location] Processing temperature in Sector Norte
INFO lexical::location::location] Processing wind-speed in Sector Norte
INFO lexical::location::location] Processing particulate-matter in Sector Norte
INFO lexical::location::location] Processing wind-speed in Sector Sur
INFO lexical::location::location] Processing particulate-matter in Sector Sur
INFO lexical::location::location] Processing temperature in Sector Sur
INFO lexical::parsing::parser] Locations created
INFO lexical] Application Malaga definition was validated!
```

Ya con las condiciones iniciales, y el estado de referencia definido, se estructuraron, las directivas, vistas en el anexo 2, que permitirían realizar la adaptación de la arquitectura

hacia el estado de objetivo.

Partiendo de todo lo anterior, se definió la serie de pasos a realizar para este primer escenario experimental:

1. Desplegar los servicios de Smart Campus UIS
2. Desplegar los servicios *Bran* y *DoThing*.
3. Usando *Lexical*, establecer el estado de referencia de la aplicación.
4. Declarar en los endpoints de *Bran*, las directivas definidas.
5. Desplegar el servicio *Looker* para la aplicación.
6. A la nivelación del estado de la aplicación.

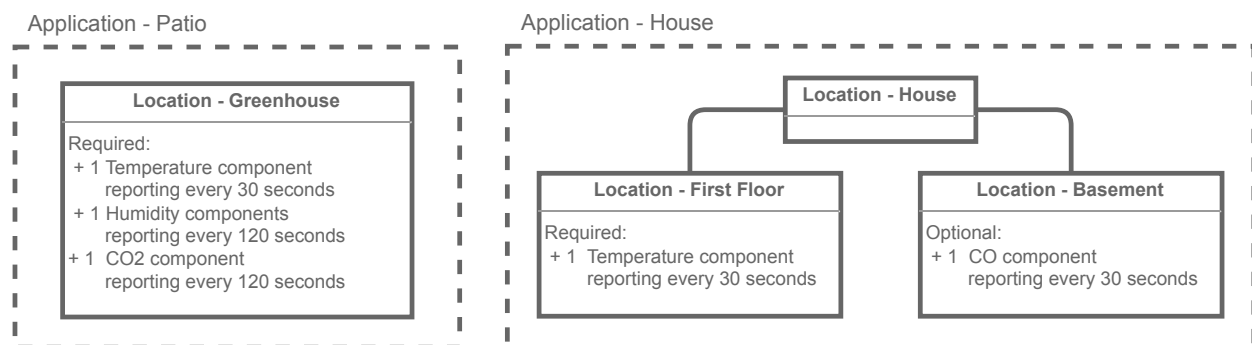
Al final de la simulación de este escenario, se eliminarán todos los servicios desplegados con el fin de asegurar una ejecución limpia de futuros procesos.

## 9.2 Escenario Experimental B

El segundo escenario, busca el evaluar la capacidad de la implementación en donde ya existen componentes en ejecución, al igual que el manejo de múltiples aplicaciones. Para esto, se definieron dos aplicaciones, como se observa en la figura 31,

**Figura 31:**

Aplicaciones definidas para el desarrollo del escenario experimental B



Para esta simulación, todos los componentes de la aplicación ya estarán ejecutándose, con el problema de que no todos estarán dentro de los parámetros establecidos. Esto requerirá que la aplicación identifique el servicio correspondiente, y realice la reconfiguración de sus parámetros.

De la misma manera que se realizó en escenario experimental A, se realizó la declaración de los estados de referencia de cada una de las aplicaciones, al igual que las directivas para su ejecución. Los YAML de esta declaración, y las directivas definidas, se encuentran en los apéndices 3, 4, 5 y 6.

Partiendo de todo lo anterior, se definió la serie de pasos a realizar para este primer escenario experimental:

1. Desplegar los servicios de Smart Campus UIS
2. Desplegar los servicios *Bran* y *DoThing*.
3. Usando *Lexical*, establecer el estado de referencia de la aplicación Patio y House.
4. Declarar en los endpoints de *Bran*, las directivas definidas.
5. Desplegar los servicios de simulación de datos *Mocker* respecto a las expectativas, tal que el estado sea coherente.
6. Desplegar los servicios *Looker* para cada una de las aplicaciones.
7. Matar los servicios desplegados y eliminar uno de los contenedores creados.
8. Esperar a ejecución de las órdenes *Restart* y *Addition*.

## 10 Análisis de Resultados

### 10.1 Resultados De Escenario Experimental A

El objetivo de las pruebas realizadas en el presente escenario experimental, era validar que, los procesos implementados, efectivamente realizaran la adaptación de las aplicacio-

nes desde un estado de completa falla, en donde ninguno de los datos necesarios estuviera presente.

Siguiendo los pasos definidos en la sección 9.1, se realizó el despliegue de los servicios base de Smart Campus UIS, al igual que *Bran* y *DoThing*. Como se presenta en la figura 33, se registró el estado de referencia al agregador tras validarlo usando *Lexical*, y se fijaron las directivas a usar para la adaptación de la arquitectura.

**Figura 32:**

Aplicación y directivas registradas en Bran

```

$ docker run -d -p 8014:8014 --name=bran --network=smart_campus_production_smart_uis -e RUST_LOG=bran mrdal
d3d9ede81dbbb0a372bb79f2868dcb8f959b3af5331b23bb36f0c6701cb92ad5
$ docker logs -f bran
[2024-01-19T06:38:53Z INFO bran::planner::planner] Starting Planner Execution
[2024-01-19T06:38:53Z INFO bran] Initializing server at 0.0.0.0:8014
[2024-01-19T06:39:28Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53004
[2024-01-19T06:39:28Z INFO bran::endpoints::receptor] Malaga was added to the register
[2024-01-19T06:39:34Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:34Z INFO bran::endpoints::receptor] Added addition directive in north-sector in app Malaga
[2024-01-19T06:39:38Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:38Z INFO bran::endpoints::receptor] Added addition directive in south-sector in app Malaga
[2024-01-19T06:39:40Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:40Z INFO bran::endpoints::receptor] Updated restart directive in north-sector in app Malaga
[2024-01-19T06:39:45Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:45Z INFO bran::endpoints::receptor] Updated restart directive in south-sector in app Malaga
[2024-01-19T06:39:54Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:54Z INFO bran::endpoints::receptor] Updated reconfig directive in north-sector in app Malaga
[2024-01-19T06:39:57Z INFO bran::endpoints::receptor] POST for Malaga request from 172.19.0.1:53024
[2024-01-19T06:39:57Z INFO bran::endpoints::receptor] Updated reconfig directive in south-sector in app Malaga

```

Una vez se inició el servicio *Looker*, como se ve en la figura 33, este empezó a evaluar el estado de la aplicación; que al no tener ningún tipo de reporte de algún componente, entró directamente en estado de falla.

**Figura 33:**

Looker evalúa el estado de la aplicación

```

9aa81c397805164b178c8fecelal9c5dac97485fa49
ERROR looker::eyes::clock] Failed to fetch timeout check value: environment variable not found
INFO looker::eyes::mqtt] Using URL tcp://mqtt_broker:1883/
WARN looker::eyes::clock] Defaulting timeout_check to 10
INFO looker] Starting server at 0.0.0.0:3000
INFO looker::eyes::mqtt] Establisng connection...
INFO looker::eyes::mqtt] Created MQTT connection
INFO looker::eyes::clock] Timeout check at 2024-01-19 01:46:53.704225448
INFO looker::eyes::clock] Timeout check complete
INFO looker::axe::requester] Updated application Malaga has been POSTed to http://bran:8014/apps/Malaga
INFO looker::eyes::clock] Timeout check at 2024-01-19 01:47:03.705439088
INFO looker::eyes::clock] Timeout check complete
INFO looker::axe::requester] Updated application Malaga has been POSTed to http://bran:8014/apps/Malaga

```

El estado observado, se reportó a *Bran*, el cual, realizó el recorrido para identificar los puntos de falla de la aplicación. Como se observa en la figura 34, como era de esperar, los

tres requerimientos definidos en cada una de las locaciones estaba en estado de falla. Al no haber componentes reportando datos, como era de esperarse, se definieron acciones *Addition* para todos los componentes de la aplicación.

**Figura 34:**

Bran identifica los problemas y establece las acciones

```
bran::endpoints::receptor] Malaga's state was updated
bran::planner::planner] Starting Planner Execution
bran::planner::planner] Checking Application Malaga
bran::planner::planner] Found 3 data requirements with errors in south-sector
bran::planner::planner] Creating Addition Order for data requirement particulate-matter in south-sector
bran::planner::planner] Creating Addition Order for data requirement wind-speed in south-sector
bran::planner::planner] Creating Addition Order for data requirement temperature in south-sector
bran::planner::planner] Found 3 data requirements with errors in north-sector
bran::planner::planner] Creating Addition Order for data requirement temperature in north-sector
bran::planner::planner] Creating Addition Order for data requirement particulate-matter in north-sector
bran::planner::planner] Creating Addition Order for data requirement wind-speed in north-sector
bran::planner::planner] Executing Addition order
bran::planner::planner] Building addition order 1 out of 2 from ProblemInfo { location_key: "south-sector",
ce_uuid: None }
bran::planner::planner] Executing order 1 out of 2
```

A partir de las acciones, se crearon las órdenes para ejecución usando las directivas como base. Como se puede ver en la figura 35, cada una de estas se envió a *DoThing*, el cual realizó el procesamiento requerido para la construcción y arranque de los contenedores. El resultado de este procesamiento se puede ver en la figura 36

**Figura 35:**

DoThing ejecuta las acciones establecidas por Bran

```
0 dothing::endpoints::director] Using ContainerCreateOpts { name: None, params: {"Env": Array [String("ip=mqtt_bro
device_uuid=ae33cddf-7f70-407f-878c-71d198dc9934")], "Image": String("mrdahaniel/mocker:latest"), "Cmd": Array [St
cation:north-sector"), String("topic:wind-speed")]} } as creation options
0 dothing::endpoints::director] Sending create signal to docker socket
0 dothing::endpoints::director] Container created with name: pedantic-elgamal
0 dothing::endpoints::director] Building network connection options...
0 dothing::endpoints::director] Built network connection options
0 dothing::endpoints::director] Connecting container to network smart_campus_production_smart_uis
0 dothing::endpoints::director] Connected container to network smart_campus_production_smart_uis
0 dothing::endpoints::director] Starting container pedantic-elgamal
0 dothing::endpoints::director] Started container pedantic-elgamal
0 dothing::endpoints::director] Process successful!
0 dothing::endpoints::director] POST request from 172.19.0.6:54336 to add container
0 dothing::endpoints::director] Building container...
0 dothing::actions::addition] Processing env_vars from Hash to Vec
0 dothing::actions::addition] env_vars processed
0 dothing::actions::addition] Building ContainerCreationOptions...
0 dothing::actions::addition] Built ContainerCreationOptions
0 dothing::endpoints::director] Using ContainerCreateOpts { name: None, params: {"Image": String("mrdahaniel/mock
uid=fb422c7a-6255-42a9-b48e-38ccefcd259e"), String("ip=mqtt_broker"), String("RUST_LOG=mockerr")], "Cmd": Array [St
cation:north-sector"), String("topic:wind-speed")]} } as creation options
0 dothing::endpoints::director] Sending create signal to docker socket
```

En total, se crearon 12 contenedores, que corresponden a los 12 componentes requeridos para el desarrollo de la aplicación. Cada uno de ellos configurado para suplir las necesidades de operación establecidas en el estado de referencia. Estos despliegues, como era de esperarse, fueron detectado por *looker*, el cual realizó el nuevas valoraciones del estado.

**Figura 36:**

Contenedores creados por DoThing en respuesta a las órdenes recibidas

```
Every 1.0s: docker container ls -a
```

| CONTAINER ID | IMAGE                                     | COMMAND                 | CREATED        | STATUS        |
|--------------|-------------------------------------------|-------------------------|----------------|---------------|
| be10fac91e31 | mrdaaniel/mocker:latest                   | "/mocker wind-speed..." | 7 seconds ago  | Up 6 seconds  |
| 87be21eb8a92 | mrdaaniel/mocker:latest                   | "/mocker wind-speed..." | 7 seconds ago  | Up 6 seconds  |
| 1493f42f41c0 | mrdaaniel/mocker:latest                   | "/mocker wind-speed..." | 8 seconds ago  | Up 7 seconds  |
| elbe9dc6ce92 | mrdaaniel/mocker:latest                   | "/mocker particulat..." | 8 seconds ago  | Up 7 seconds  |
| 2fe1387443f9 | mrdaaniel/mocker:latest                   | "/mocker particulat..." | 8 seconds ago  | Up 8 seconds  |
| 6f33df9571d7 | mrdaaniel/mocker:latest                   | "/mocker temperatur..." | 9 seconds ago  | Up 8 seconds  |
| 8ac5e9d279a1 | mrdaaniel/mocker:latest                   | "/mocker temperatur..." | 9 seconds ago  | Up 8 seconds  |
| 249e342bfce9 | mrdaaniel/mocker:latest                   | "/mocker wind-speed..." | 10 seconds ago | Up 9 seconds  |
| 190d3fe013c4 | mrdaaniel/mocker:latest                   | "/mocker wind-speed..." | 10 seconds ago | Up 9 seconds  |
| 447a4ed21267 | mrdaaniel/mocker:latest                   | "/mocker wind-speed..." | 10 seconds ago | Up 10 seconds |
| 765944fa10fd | mrdaaniel/mocker:latest                   | "/mocker particulat..." | 11 seconds ago | Up 10 seconds |
| 7d6cf546e876 | mrdaaniel/mocker:latest                   | "/mocker particulat..." | 11 seconds ago | Up 10 seconds |
| cd1718c40388 | mrdaaniel/looker:latest                   | "/looker"               | 2 minutes ago  | Up 2 minutes  |
| 9a4eb8c81017 | mrdaaniel/dotthing:latest                 | "/dotthing"             | 10 minutes ago | Up 10 minutes |
| d3d9ede81dbb | mrdaaniel/bran:latest                     | "/bran -e watcher_i..." | 10 minutes ago | Up 10 minutes |
| 80f4928c8d58 | smart-campus_production-data_microservice | "java -jar /app.jar"    | 6 days ago     | Up 12 hours   |

Los valores por defecto de los servicios usados para estas pruebas es un intervalo de tiempo entre mensajes de un minuto. Este valor por defecto, aunque para los requerimientos de datos cuyos *timeout*, sean superiores, supliría las necesidades de la aplicación. Sin embargo, en el marco de esta simulación, esta frecuencia de reportes no es suficiente para cumplir con el estado de referencia de los requerimiento de temperatura de las locaciones.

Como consecuencia, el estado de la aplicación regresó a estado *Fault*, y *Bran*, al detectar los problemas, emitió las acciones tipo *Restart* con el fin de traer los contenedores a un estado válido. Esto se puede ver en la figura 37.

**Figura 37:**

Bran identifica otros problemas en la aplicación

```

anner::planner] Starting Planner Execution
anner::planner] Checking Application Malaga
anner::planner] Found 1 data requirements with errors in south-sector
anner::planner] Creating Restart Order for component d8ef9ba1-1a9a-4e08-a0e6-e2414
anner::planner] Found 1 data requirements with errors in north-sector
anner::planner] Creating Restart Order for component cc81a342-41f4-4301-9c14-33287
anner::planner] Executing Restart order
anner::planner] Executing order: RestartOrder { uuid: Some(d8ef9ba1-1a9a-4e08-a0e6
points::receptor] PUT for Malaga request from 172.19.0.8:53374
points::receptor] Malaga's state was updated
anner::planner] Executing Restart order
anner::planner] Executing order: RestartOrder { uuid: Some(cc81a342-41f4-4301-9c14

```

Este intento de reiniciar los servicios, como era de esperarse, no fue realmente efectiva debido a que la falla se debe a un problema en la configuración del servicio. Siendo así, la aplicación se mantuvo en estado de falla. Se esperaba que, ambos componentes encargados de la temperatura, al ya haber descartado el reinicio como un mecanismo efectivo para esta situación, fueran sometidos a una acción de reconfiguración, sin embargo, como se observa en la figura 38, en ese instante, sólo el componente de `north-sector` fue afectado.

**Figura 38:**

Bran establece acciones de reconfiguración para adaptar la aplicación

```

receptor] PUT for Malaga request from 172.19.0.8:50694
receptor] Malaga's state was updated
receptor] PUT for Malaga request from 172.19.0.8:49084
receptor] Malaga's state was updated
anner] Starting Planner Execution
anner] Checking Application Malaga
anner] Found 1 data requirements with errors in north-sector
anner] Creating Reconfigure Order for component cc81a342-41f4-4301-9c14-33
anner] Found 0 data requirements with errors in south-sector
anner] Executing Reconfigure order
anner] Executing order: ReconfigureOrder { uuid: Some(cc81a342-41f4-4301-9
endpoint: "/get/device", port: 8000 }, reconfig: Http { endpoint: "/updat
Number(25)} } }
receptor] PUT for Malaga request from 172.19.0.8:55762
receptor] Malaga's state was updated

```

Esta reconfiguración fue ejecutada por *DoThing*, y tomó efecto, alterando el valor por



defecto entre mensajes, pasando de un reporte cada minuto, a uno cada 25 segundos<sup>15</sup>. Con el nuevo valor, los componentes de temperatura, empezaban a reportar con mayor frecuencia, cumpliendo con las expectativas.

Poco tiempo después, el componente de temperatura de `south-sector`, reportó el estado de falla, por lo se ejecutó la misma acción de reconfiguración que con el componente de la otra locación, llevando el estado de la aplicación a *Coherent*.

A partir de esta simulación, se pudo validar que el proceso implementado, tiene la capacidad de realizar la adaptación desde el peor escenario, hacia un esto de coherencia con la aplicación declarada inicialmente. Todas las acciones definidas por *Bran*, permitieron modificar el estado de manera positiva.

Sin embargo, se identificaron algunas limitaciones de esta implementación. Como se observa en la figura 38, se muestra que `south-sector` no presenta problemas en sus requerimientos, cuando, debido a su configuración, debía estar en estado de falla.

Esto se debe a un *quirk* de la implementación realizada. Al manejar los procesos cada cierto intervalo de tiempo, es posible que un componente se encuentren en estado de falla, pero alcance a pasar a coherente justo antes de la evaluación de las acciones a tomar. Eventualmente, el componente fallaría en reportar antes de una una de las evaluaciones desencadenando en una acción, como fue el caso con la tardía reconfiguración del componente de `south-sector`.

## 10.2 Resultados De Escenario Experimental B

El objetivo principal del escenario experimental B, era evaluar la capacidad de la implementación realizada de recuperar una aplicación en estado de coherencia, en donde se presenta una falla masiva, nuevamente a un estado válido; al igual que la capacidad de *Bran* de manejar más de una aplicación, como se había nombrado en la sección 7.2.

Igual que durante el escenario experimental A, se realizó el despliegue de los servicios

---

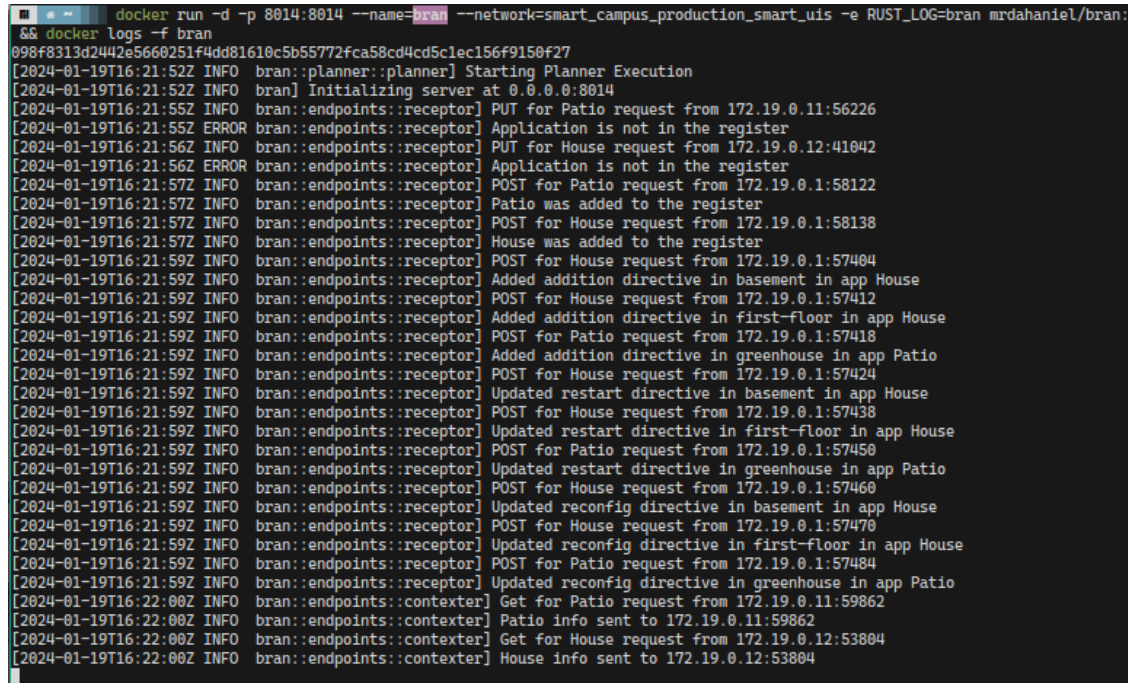
<sup>15</sup>Según lo definido en las directivas presentes en el apéndice 2



*Bran* y *DoThing*. Como se observa en la figura 39, se definieron las arquitecturas objetivo de las dos aplicaciones a monitorear, al igual que las directivas a usar durante este proceso.

**Figura 39:**

Aplicaciones y directivas registradas en Bran



```

$ docker run -d -p 8014:8014 --name=bran --network=smart_campus_production_smart_uis -e RUST_LOG=bran mrdahaniel/bran:
$ docker logs -f bran
098f8313d2442e5660251f4dd81610c5b55772fca58cd4cd5c1ec156f9150f27
[2024-01-19T16:21:52Z INFO bran::planner::planner] Starting Planner Execution
[2024-01-19T16:21:52Z INFO bran] Initializing server at 0.0.0.0:8014
[2024-01-19T16:21:55Z INFO bran::endpoints::receptor] PUT for Patio request from 172.19.0.11:56226
[2024-01-19T16:21:55Z ERROR bran::endpoints::receptor] Application is not in the register
[2024-01-19T16:21:56Z INFO bran::endpoints::receptor] PUT for House request from 172.19.0.12:41042
[2024-01-19T16:21:56Z ERROR bran::endpoints::receptor] Application is not in the register
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:58122
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] Patio was added to the register
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:58138
[2024-01-19T16:21:57Z INFO bran::endpoints::receptor] House was added to the register
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57404
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Added addition directive in basement in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57412
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Added addition directive in first-floor in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:57418
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Added addition directive in greenhouse in app Patio
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57424
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated restart directive in basement in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57438
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated restart directive in first-floor in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:57450
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated restart directive in greenhouse in app Patio
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57460
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated reconfig directive in basement in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for House request from 172.19.0.1:57470
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated reconfig directive in first-floor in app House
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] POST for Patio request from 172.19.0.1:57484
[2024-01-19T16:21:59Z INFO bran::endpoints::receptor] Updated reconfig directive in greenhouse in app Patio
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] Get for Patio request from 172.19.0.11:59862
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] Patio info sent to 172.19.0.11:59862
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] Get for House request from 172.19.0.12:53804
[2024-01-19T16:22:00Z INFO bran::endpoints::contexter] House info sent to 172.19.0.12:53804

```

Seguidamente, como se observa en la figura 40, se desplegaron los servicios de *Looker* para cada una de las aplicaciones a monitorear. Ya con esta base lista, y como se definió en los pasos a seguir, se desplegaron de manera manual, un total de 3 servicios *Mocker* con configuraciones específicas (tanto de los datos que estos reportaban, como de los tiempos de reporte) con el fin de cumplir con los requerimientos de datos definidos para las dos aplicaciones.

**Figura 40:**

Dos instancias de *looker* monitoreando las aplicaciones declaradas

```

looker::eyes::clock] Defaulting timeout_check to 10
looker::eyes::mqtt] Using URL tcp://mqtt_broker:1883/
looker] Starting server at 0.0.0.0:3000
looker::eyes::mqtt] Establishing connection...
looker::eyes::mqtt] Created MQTT connection
looker::app::application] message received from topic: 'device-messages'
looker::app::application] device_uuid = dact68bb-18ad-4ef2-82b8-4abee5742bd2
looker::app::application] topic = temperatura
looker::app::application] values = {"location":"entrance","temperature":28}
looker::app::application] No location was found for key 'entrance'
looker::app::application] message received from topic: 'device-messages'
looker::app::application] device_uuid = 7b5185e3-6181-452e-a647-fb6957a76425
looker::app::application] topic = temperatura
looker::app::application] values = {"co2":44,"location":"basement"}
looker::app::application] No location was found for key 'basement'
looker::app::application] message received from topic: 'device-messages'
looker::app::application] device_uuid = 15f66f9f-bda5-4e4b-844c-fe687bb12979
looker::app::application] topic = temperatura
looker::app::application] values = {"co2":185,"humidity":50,"location":"gree
looker::app::application] Updated state.
looker::eyes::clock] Timeout check at 2024-01-19 11:22:10.325554880
looker::eyes::clock] Timeout check complete
looker::axe::requester] Updated application Patio has been POSTed to http://
looker::eyes::clock] Timeout check at 2024-01-19 11:22:20.326572796
looker::eyes::clock] Timeout check complete
looker::axe::requester] Updated application Patio has been POSTed to http://

[2024-01-19T16:22:00Z INFO looker::eyes::mqtt] Using URL tcp://mqtt_broker:1883/
[2024-01-19T16:22:00Z WARN looker::eyes::clock] Defaulting timeout_check to 10
[2024-01-19T16:22:00Z INFO looker] Starting server at 0.0.0.0:3000
[2024-01-19T16:22:00Z INFO looker::eyes::mqtt] Establishing connection...
[2024-01-19T16:22:00Z INFO looker::eyes::mqtt] Created MQTT connection
[2024-01-19T16:22:01Z INFO looker::app::application] message received from topic:
[2024-01-19T16:22:01Z INFO looker::app::application] device_uuid = dact68bb-18ad-4
[2024-01-19T16:22:01Z INFO looker::app::application] topic = temperatura
[2024-01-19T16:22:01Z INFO looker::app::application] values = {"location":"entranc
[2024-01-19T16:22:01Z ERROR looker::app::application] No location was found for key
[2024-01-19T16:22:01Z INFO looker::app::application] message received from topic:
[2024-01-19T16:22:01Z INFO looker::app::application] device_uuid = 7b5185e3-6181-4
[2024-01-19T16:22:01Z INFO looker::app::application] topic = temperatura
[2024-01-19T16:22:01Z INFO looker::app::application] values = {"co2":44,"location":
[2024-01-19T16:22:01Z INFO looker::app::application] Updated state.
[2024-01-19T16:22:02Z INFO looker::app::application] message received from topic:
[2024-01-19T16:22:02Z INFO looker::app::application] device_uuid = 15f66f9f-bda5-4
[2024-01-19T16:22:02Z INFO looker::app::application] topic = temperatura
[2024-01-19T16:22:02Z INFO looker::app::application] values = {"co2":185,"humidity
[2024-01-19T16:22:02Z ERROR looker::app::application] No location was found for key
[2024-01-19T16:22:10Z INFO looker::eyes::clock] Timeout check at 2024-01-19 11:22:
[2024-01-19T16:22:10Z INFO looker::eyes::clock] Timeout check complete
[2024-01-19T16:22:10Z INFO looker::axe::requester] Updated application House has b
[2024-01-19T16:22:20Z INFO looker::eyes::clock] Timeout check at 2024-01-19 11:22:
[2024-01-19T16:22:20Z INFO looker::eyes::clock] Timeout check complete
[2024-01-19T16:22:20Z INFO looker::axe::requester] Updated application House has b

```

Como era de esperarse, con las configuraciones hechas a la medida de las aplicaciones, el estado de estas era *Coherent*, sin la necesidad de ningún tipo de cambios en los dispositivos. Siendo así, como se ve en la figura 41, se matan los 3 contenedores desplegados, y se elimina uno de ellos.

**Figura 41:**

Se matan los contenedores con los servicios iniciales

```

Every 1.0s: docker container ls -a

```

| CONTAINER ID     | IMAGE NAMES                               | COMMAND                  | CREATED       | STATUS                    |
|------------------|-------------------------------------------|--------------------------|---------------|---------------------------|
| 013ed7ae8477     | mrdaaniel/mockers:latest                  | "/mockers co2:random..." | 3 minutes ago | Exited (137) 1 second ago |
| da06a86bc3c0     | mrdaaniel/mockers:latest                  | "/mockers temperatur..." | 3 minutes ago | Exited (137) 1 second ago |
| e5958ed4acac     | mrdaaniel/looker:latest                   | "/looker"                | 3 minutes ago | Up 3 minutes              |
| 01->3000/tcp     | house-looker                              |                          |               |                           |
| 4864c04bc106     | mrdaaniel/looker:latest                   | "/looker"                | 3 minutes ago | Up 3 minutes              |
| 00->3000/tcp     | patio-looker                              |                          |               |                           |
| 025c7b091086     | mrdaaniel/doing:latest                    | "/doing"                 | 3 minutes ago | Up 3 minutes              |
| 50->8050/tcp     | doing                                     |                          |               |                           |
| c70fec2f5626     | mrdaaniel/bran:latest                     | "/bran -e watcher_i..."  | 4 minutes ago | Up 4 minutes              |
| 14->8014/tcp     | bran                                      |                          |               |                           |
| 80f4928c8d58     | smart_campus_production-data_microservice | "java -jar /app.jar"     | 7 days ago    | Up 25 hours               |
| 91->8091/tcp     | data_microservice                         |                          |               |                           |
| b8888aee8a63     | mongo-express                             | "/sbin/tini -- /dock..." | 7 days ago    | Up 25 hours               |
| 81->8081/tcp     | smart_campus_production-mongo-express-1   |                          |               |                           |
| a8d97ecef21      | mongo                                     | "docker-entrypoint.s..." | 7 days ago    | Up 25 hours               |
| 27017->27017/tcp | smart_campus_production-mongo-1           |                          |               |                           |
| e3a7ce6eb973     | eclipse-mosquitto                         | "/docker-entrypoint..."  | 7 days ago    | Up 25 hours               |
| 83->1883/tcp     | mqtt_broker                               |                          |               |                           |

La figura 42, muestra el como *Bran*, define las órdenes para reiniciar los servicios para traer la aplicación a un estado nuevamente. En condiciones normales, se esperaría que esto sea lo único a realizar para adaptar la arquitectura, sin embargo, este no fue el caso.

**Figura 42:**

Bran intenta reiniciar los contenedores

```

r] Starting Planner Execution
r] Checking Application Patio
r] Found 1 data requirements with errors in greenhouse
r] Creating Restart Order for component 0b60df47-b823-4c92-b065-a17696c140f6 data requirement temperature in
r] Executing Restart order
r] Executing order: RestartOrder { uuid: Some(0b60df47-b823-4c92-b065-a17696c140f6), query_type: Http { endp
r] Checking Application House
r] Found 1 data requirements with errors in first-floor
r] Creating Restart Order for component 4f8c7b1b-3682-4734-82ca-33af6909c652 data requirement temperature in
r] Found 1 data requirements with errors in basement
r] Creating Restart Order for component 6e33c6a3-d942-4e0d-8daf-e0f967d74b90 data requirement co in basement
r] Executing Restart order
r] Executing order: RestartOrder { uuid: Some(4f8c7b1b-3682-4734-82ca-33af6909c652), query_type: Http { endp
r] Executing Restart order
r] Executing order: RestartOrder { uuid: Some(6e33c6a3-d942-4e0d-8daf-e0f967d74b90), query_type: Http { endp
r] PUT for Patio request from 172.19.0.8:42000

```

Ya que los contenedores no fueron desplegados, o ha sido modificados, de ninguna manera por *DoThing*, no estos no están mapeados. Siendo así, como se ve en la figura 43, es necesario buscar los contenedores en la red, pero, al estar los servicios muertos, y depender de la consulta *Http* para poder identificarlos, no puede encontrarlos.

**Figura 43:**

DoThing no puede encontrar los contenedores

```

:restart] Making get request to http://house-looker:8000/get/device
:restart] Got no response from http://house-looker:8000/get/device
:restart] Checking container with id 4864c04bc10660d3ce16384bde5c173cladbb495c1146068fe9c93deaaeb37a2
:restart] Making get request to http://patio-looker:8000/get/device
:restart] Got no response from http://patio-looker:8000/get/device
:restart] Checking container with id 025c7b091086226a66a8251b80451c70b9aec99137ac45802f6a1339217a295b
:restart] Making get request to http://dothing:8000/get/device
:restart] Got no response from http://dothing:8000/get/device
:restart] Checking container with id c70fec2f56261c8e3e572c83173cdcbf708eac0b30f1acd256dae1e7ac23c1b9
:restart] Making get request to http://bran:8000/get/device
:restart] Got no response from http://bran:8000/get/device
:restart] Checking container with id 80f4928c8d5886801e2a91a77d5d1043ca43c8d3d308badfcae10546b441f6bc9
:restart] Making get request to http://data_microservice:8000/get/device
:restart] Got no response from http://data_microservice:8000/get/device
:restart] Checking container with id b8888aee8a63ffbc1f6ffbb09f06cbe09af770de511bb16bf417233eac7b2572
:restart] Making get request to http://smart_campus_production-mongo-express-1:8000/get/device
:restart] Got no response from http://smart_campus_production-mongo-express-1:8000/get/device
:restart] Checking container with id a8d97ecef219fc6e79f009a3a398351224671e4e724c02b2296aaba395a83b6
:restart] Making get request to http://smart_campus_production-mongo-1:8000/get/device
:restart] Got no response from http://smart_campus_production-mongo-1:8000/get/device
:restart] Checking container with id e3a7ce6eb973ebf6c95b955db1787d8aa430335a3d53c1554f967b95025ba51a
:restart] Making get request to http://mqtt_broker:8000/get/device
:restart] Got no response from http://mqtt_broker:8000/get/device
:director] Could not find container

```

Esto demuestra otra de las falencias con la implementación realizada, que es la búsqueda del crecimiento de la fase de conocimiento. El haber identificado los componentes, antes de que se presentaran problemas, hubiera posibilitado la ejecución de órdenes cuando los servi-

cios estén caídos. Así mismo, se hubiera podido establecer una búsqueda que no dependiera del estado de los componentes, quizás usando tags.

El resultado final, es que *Bran*, al no poder actuar de otra manera, como se observa en la figura 44, ordena acciones de adición para suplir los componentes. Esto, eventualmente, resulta en el regreso de la aplicación a un estado válido, pero sigue sin ser lo ideal.

**Figura 44:**

Bran crea órdenes de adicción

```
[2024-01-19T19:21:38Z INFO bran:endpoints:receptor] PUT for House request from 172.19.0.9:57962
[2024-01-19T19:21:38Z INFO bran:endpoints:receptor] House's state was updated
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:21:38Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "greenhouse", data_requirement_key: "
temperature", device_uuid: None }
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing order 1 out of 1
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing Restart order
[2024-01-19T19:21:38Z INFO bran:planner:planner] Executing order: RestartOrder { uuid: Some(a22dadc5-e63b-4052-9266-4445aa19bf37), query_type: Http { endp
oint: "/get/device", port: 8000 } }
[2024-01-19T19:21:47Z INFO bran:endpoints:receptor] PUT for Patio request from 172.19.0.8:43622
[2024-01-19T19:21:47Z INFO bran:endpoints:receptor] Patio's state was updated
[2024-01-19T19:21:48Z INFO bran:endpoints:receptor] PUT for House request from 172.19.0.9:41636
[2024-01-19T19:21:48Z INFO bran:endpoints:receptor] House's state was updated
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:21:49Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "greenhouse", data_requirement_key: "
co2", device_uuid: None }
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing order 1 out of 1
[2024-01-19T19:21:49Z INFO bran:planner:planner] Checking Application House
[2024-01-19T19:21:49Z WARN bran:planner:planner] Found 1 data requirements with errors in first-floor
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Addition Order for data requirement temperature in first-floor
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Restart Order for component b05984f0-ad28-46d7-a052-9eb14b7a7a15 data requirement temperature in
first-floor
[2024-01-19T19:21:49Z WARN bran:planner:planner] Found 1 data requirements with errors in basement
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Addition Order for data requirement co in basement
[2024-01-19T19:21:49Z INFO bran:planner:planner] Creating Restart Order for component 198a2e59-b52f-4be5-b02c-30b05a28130b data requirement co in basement
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:21:49Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "first-floor", data_requirement_key:
"temperature", device_uuid: None }
[2024-01-19T19:21:49Z INFO bran:planner:planner] Executing order 1 out of 1
[2024-01-19T19:21:50Z INFO bran:planner:planner] Executing Restart order
[2024-01-19T19:21:50Z INFO bran:planner:planner] Executing order: RestartOrder { uuid: Some(b05984f0-ad28-46d7-a052-9eb14b7a7a15), query_type: Http { endp
oint: "/get/device", port: 8000 } }
[2024-01-19T19:21:57Z INFO bran:endpoints:receptor] PUT for Patio request from 172.19.0.8:50608
[2024-01-19T19:21:57Z INFO bran:endpoints:receptor] Patio's state was updated
[2024-01-19T19:21:58Z INFO bran:endpoints:receptor] PUT for House request from 172.19.0.9:51078
[2024-01-19T19:21:58Z INFO bran:endpoints:receptor] House's state was updated
[2024-01-19T19:22:01Z INFO bran:planner:planner] Executing Addition order
[2024-01-19T19:22:01Z INFO bran:planner:planner] Building addition order 1 out of 1 from ProblemInfo { location_key: "basement", data_requirement_key: "co
", device_uuid: None }
[2024-01-19T19:22:01Z INFO bran:planner:planner] Executing order 1 out of 1
```

## 11 Conclusiones y Trabajo Futuro

El presente trabajo buscaba realizar la implementación de mecanismos de adaptación de arquitecturas a la plataforma Smart Campus UIS, que permitiera alterar el estado de una aplicación hacia un estado de referencia, de manera autonómica.

Para ello, se desarrolló e implementó un metamodelo capaz representar dichas arquitecturas, al igual que una manera de declararla tomando como foco los datos requeridos para su correcto funcionamiento; el cual cumple con el objetivo de establecer un estado de referencia.

También se diseñó, e implementó, un servicio capaz de interpretar los mensajes de los dispositivos y evaluar el estado de la aplicación; al igual que una proceso de identificar los problemas y definir las acciones a tomar; cumpliendo con el objetivo de determinar una manera por la cual se pudieran realizar dichas comparaciones.

Así mismo, se diseñaron mecanismos que dieran la capacidad de modificar la arquitectura, hacia el estado de referencia definido; cumpliendo con el objetivo de reducir la cantidad de diferencias entre el estado observado, y el deseado.

Finalmente, se validaron las implementaciones realizadas, al igual que se identificaron las falencias de las mismas, dando paso a un nuevo foco de trabajo, ajustando, afinando y extendiendo las maneras en las que se pueden adaptar las arquitecturas de manera autonómica, al igual que estandarizando y simplificando la manera en la que se trabajan con estas herramientas.

De esta primera implementación, rara vez se sale de una línea de comando, por lo que el desarrollo de interfases para los diversos servicios implementados, con el fin de facilitar la accesibilidad a la plataforma.

Así mismo, la extensión de manera de evaluar los estados del sistema, a partir de diferentes datos definidos por el usuario, como uso de promedios para flexibilizar las fallas o contadores, antes de tomar acciones para ignorar fallos temporales, son puntos importantes a mejorar en futuras versiones.

También se ha de considerar la reimplementación de algunos de los mecanismos, con el fin de aumentar su eficacia en modificar los estados. Al igual que la expansión de la base de conocimiento, con el fin de aprovechar todos los recursos disponibles, independiente del estado de algunos componentes.

Se recomienda el buscar formas de permitir, al usuario, definir procesamientos de datos extra para la evaluación de el estado de referencia. Idealmente de manera agnóstica al lenguaje de programación usado. Lo cual daría un valor extra a la flexibilidad de la plataforma.

Finalmente, se recomienda la realización de pruebas más extensas, con dispositivos no simulados, que permitan evolucionar este primer prototipo de plugin para Smart Campus UIS, a algo que se pueda desplegar con relativa facilidad, independientemente del la aplicación que se esté desarrollando.

## Referencias

- Anagnostopoulos, T. (2023). Smart campus. En *Iot-enabled unobtrusive surveillance systems for smart campus safety* (p. 17-25). doi: 10.1002/9781119903932.ch3
- Andre, C. (2007). *Uml extensions: the sysml profile*. Descargado 2023-05-19, de [https://www.i3s.unice.fr/~map/Cours/MASTER\\_TSM/Cours7\\_SysML.pdf](https://www.i3s.unice.fr/~map/Cours/MASTER_TSM/Cours7_SysML.pdf) (Presentation on UML extensions)
- Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J. C., Barends, R., ... Martinis, J. M. (2019). Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779), 505–510. Descargado de <https://www.nature.com/articles/s41586-019-1666-5> doi: 10.1038/s41586-019-1666-5
- Boixo, S., Isakov, S. V., Smelyanskiy, V. N., y Neven, H. (2017). Simulation of low-depth quantum circuits as complex undirected graphical models. *arXiv preprint arXiv:1712.05384*. Descargado de <https://arxiv.org/abs/1712.05384>
- Burtscher, M., y Ratanaworabhan, P. (2009). Fpc: A high-speed compressor for double-precision floating-point data. *IEEE Transactions on Computers*, 58(1), 18–31. Descargado de <https://userweb.cs.txstate.edu/~mb92/papers/tc09.pdf> doi: 10.1109/TC.2008.131
- Carnegie Mellon University. (2017). *Aadl and osate: A tool kit to support model-based engineering*. Online. Carnegie Mellon University. Descargado de [https://resources.sei.cmu.edu/asset\\_files/FactSheet/2017\\_010\\_001\\_506838.pdf](https://resources.sei.cmu.edu/asset_files/FactSheet/2017_010_001_506838.pdf)
- Carnegie Mellon University. (2022). *Architecture analysis and design language*. Carnegie Mellon University. Descargado de [https://www.sei.cmu.edu/our-work/projects/display.cfm?customel\\_datapageid\\_4050=191439%2C191439](https://www.sei.cmu.edu/our-work/projects/display.cfm?customel_datapageid_4050=191439%2C191439)
- Chen, J., Zhang, F., Huang, C., Newman, M., y Shi, Y. (2018). *Classical simulation of intermediate-size quantum circuits*. Descargado de <https://arxiv.org/abs/1805.01450>



- Costa, B., Pires, P. F., y Delicato, F. C. (2016). Modeling iot applications with sysml4iot. En *2016 42th euromicro conference on software engineering and advanced applications (seaa)* (p. 157-164). doi: 10.1109/SEAA.2016.19
- Docker Inc. (2023). *Develop with docker engine api*. Online. Docker. Descargado de <https://docs.docker.com/engine/api/>
- Díaz, G., Steffemel, L. A., Barrios, C. J., y Couturier, J.-F. (2024). *How to build a software quantum simulator*. Preprints. Descargado de <https://doi.org/10.20944/preprints202409.1497.v1> doi: 10.20944/preprints202409.1497.v1
- Díaz Toro, G. J. (2025). *Gestión de memoria en simuladores de computación cuántica de software* (Tesis doctoral, Universidad Industrial de Santander). Descargado 2025-05-13, de <https://noesis.uis.edu.co/items/357883df-3223-45b0-9848-f7681c5b0511>
- Eclipse Foundation. (2018). *Eclipse mita*. Eclipse Foundation. Descargado 2023-05-13, de <https://www.eclipse.org/mita/>
- Harrand, N., Fleurey, F., Morin, B., y Husa, K. (2016, 10). Thingml: a language and code generation framework for heterogeneous targets. En (p. 125-135). doi: 10.1145/2976767.2976812
- Huffman, N., Pavlichin, D., y Weissman, T. (2024). *Lossy compression for schrödinger-style quantum simulations*. Descargado de <https://arxiv.org/abs/2401.11088>
- IoT-A. (2014). *Internet of things architecture*. Online. Descargado 2023-05-20, de <https://www.iot-a.eu/public/>
- Jamalidinan, S., y Cheshmi, K. (2025). *Floating-point data transformation for lossless compression*. Descargado de <https://arxiv.org/abs/2506.18062> doi: 10.48550/arXiv.2506.18062
- Jaques, S., y Häner, T. (2021). *Leveraging state sparsity for more efficient quantum simulations*. Descargado de <https://arxiv.org/abs/2105.01533>
- Jiménez Herrera, H. A. (2022). *Mecanismos autónomos para la autodescripción de arquitecturas iot distribuidas*. Descargado de <https://noesis.uis.edu.co/items/>



c5d1cce7-3f8f-4e3e-a083-66bd3c4745f3

- Jiménez Herrera, H. A. (2023). *Smart campus uis production*. Online. GitHub. Descargado de [https://github.com/UIS-IoT-Smart-Campus/smart\\_campus\\_production](https://github.com/UIS-IoT-Smart-Campus/smart_campus_production)
- Khaled, A. E., Helal, A., Lindquist, W., y Lee, C. (2018). Iot-ddl—device description language for the “t” in iot. *IEEE Access*, 6, 24048-24063. doi: 10.1109/ACCESS.2018.2825295
- Kirchhof, J. C., Rumpe, B., Schmalzing, D., y Wortmann, A. (2022a). *Montithings*. Online. Descargado 2023-05-13, de <https://github.com/MontiCore/montithings>
- Kirchhof, J. C., Rumpe, B., Schmalzing, D., y Wortmann, A. (2022b). Montithings: Model-driven development and deployment of reliable iot applications. *Journal of Systems and Software*, 183, 111087. Descargado de <https://www.sciencedirect.com/science/article/pii/S0164121221001849> doi: <https://doi.org/10.1016/j.jss.2021.111087>
- Lalanda, P., Diaconescu, A., y McCann, J. A. (2014). *Autonomic computing: Principles, design and implementation*. Springer.
- Li, S., Kimura, Y., Sato, H., Yu, J., y Fujita, M. (2023). *Parallelizing quantum simulation with decision diagrams*. Descargado de <https://arxiv.org/abs/2312.01570>
- Lindstrom, P., y Isenburg, M. (2006). Fast and efficient compression of floating-point data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5), 1245–1250. Descargado de <https://pubmed.ncbi.nlm.nih.gov/17080858/> doi: 10.1109/TVCG.2006.143
- Markov, I. L., Fatima, A., Isakov, S. V., y Boixo, S. (2018). *Quantum supremacy is both closer and farther than it appears*. Descargado de <https://arxiv.org/abs/1807.10749>
- Markov, I. L., y Shi, Y. (2008). Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3), 963–981. Descargado de <https://arxiv.org/abs/quant-ph/0511069> doi: 10.1137/050644756
- Masui, K., Amiri, M., Connor, L., Deng, M., Fandino, M., Hoefer, C., ... Vanderlinde, K. (2015). A compression scheme for radio data in high performance computing. *arXiv preprint arXiv:1503.00638*. Descargado de <https://arxiv.org/abs/1503.00638>

- (Describes the Bitshuffle filter and HDF5 plugin) doi: 10.48550/arXiv.1503.00638
- Miller, D. M., y Thornton, M. A. (2006). QMDD: A decision diagram structure for reversible and quantum circuits. En *Proceedings of the 36th international symposium on multiple-valued logic (ISMVL'06)* (p. 30). Descargado de <https://ieeexplore.ieee.org/document/1623982/> doi: 10.1109/ISMVL.2006.35
- Mozilla Foundation. (2023). *State machine*. Online. Descargado de [https://developer.mozilla.org/en-US/docs/Glossary/State\\_machine](https://developer.mozilla.org/en-US/docs/Glossary/State_machine)
- Object Management Group. (2015). *What is sysml?* Online. Descargado de <https://www.omg-sysml.org/what-is-sysml.htm>
- Pednault, E., Gunnels, J. A., Nannicini, G., Horesh, L., Magerlein, T., Solomonik, E., ... Wisnieff, R. (2020). *Pareto-efficient quantum circuit simulation using tensor contraction deferral*. Descargado de <https://arxiv.org/abs/1710.05867>
- Red Hat Foundation. (2023). *What is yaml?* Descargado 2023-5-27, de <https://www.redhat.com/en/topics/automation/what-is-yaml>
- Shvets, A. (2019). *Dive into design patterns* (R. S. Andrew Wetmore, Ed.). Refactoring Guru.
- Song, Y., Li, Z., y Wu, J. (2023). *Mera: Memory reduction and acceleration for quantum circuit simulation via redundancy exploration*. Descargado de <https://arxiv.org/abs/2411.15332>
- Viamontes, G. F., Markov, I. L., y Hayes, J. P. (2005). Graph-based simulation of quantum computation in the density matrix representation. *Quantum Information & Computation*, 5(2), 113–130. Descargado de <https://arxiv.org/abs/quant-ph/0403114>
- Vidal, G. (2003). Efficient classical simulation of slightly entangled quantum computations. *Phys. Rev. Lett.*, 91(14), 147902. doi: 10.1103/PhysRevLett.91.147902
- Vinkhuijzen, L., Coopmans, T., Elkouss, D., Dunjko, V., y Laarman, A. (2023, septiembre). LIMDD: A Decision Diagram for Simulation of Quantum Computing Including Stabilizer States. *Quantum*, 7, 1108. Descargado de <https://quantum-journal.org/>

[papers/q-2023-09-11-1108/](#) doi: 10.22331/q-2023-09-11-1108

- Wu, X.-C., Di, S., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2018). Amplitude-aware lossy compression for quantum circuit simulation. En *Proceedings of the 4th international workshop on data reduction for big scientific data (drbsd-4) at sc18*. IEEE.
- Wu, X.-C., Di, S., Dasgupta, E. M., Cappello, F., Finkel, H., Alexeev, Y., y Chong, F. T. (2019). Full-state quantum circuit simulation by using data compression. En *Proceedings of the international conference for high performance computing, networking, storage and analysis (sc'19)*. IEEE/ACM. doi: 10.1145/3295500.3356155
- Zhang, B., Fang, B., Ye, F., Gu, Y., Tallent, N., Tan, G., y Tao, D. (2024a). *Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework*. Descargado de <https://arxiv.org/abs/2410.14088>
- Zhang, B., Fang, B., Ye, F., Gu, Y., Tallent, N., Tan, G., y Tao, D. (2024b). *Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework*. Descargado de <https://arxiv.org/abs/2410.14088>

## Apéndices

### 1 Directivas declaradas la aplicación Malaga

```
application:
  name: Malaga
  description: "Test Scenario A"

locations:
  malaga:
    name: "Sede Malaga"
    locations:
      north-sector:
        name: "Sector Norte"
        data-requirements:
          temperature:
            output: number
            required: true
            count: 1
            timeout: 30
          wind-speed:
            output: number
            required: true
            count: 3
            timeout: 60
          particulate-matter:
            output: number
            required: false
            count: 2
            timeout: 120
      south-sector:
        name: "Sector Sur"
        data-requirements:
          wind-speed:
            output: number
            required: true
            count: 3
            timeout: 60
          particulate-matter:
            output: number
            required: false
            count: 2
            timeout: 120
          temperature:
            output: number
            required: true
            count: 1
            timeout: 30
```

## 2 Directivas de la aplicación Malaga

```
{
  "north-sector": {
    "addition": {
      "image": "mrdahaniel/mocker:latest",
      "network_name": "smart_campus_production_smart_uis",
      "env_vars": {
        "RUST_LOG": "mock",
        "ip": "mqtt_broker"
      },
      "args": [
        "key:random(5,35)"
      ]
    },
    "reconfig": {
      "uuid": null,
      "network": "smart_campus_production_smart_uis",
      "query_type": {
        "Http": {
          "endpoint": "/get/device",
          "port": 8000
        }
      },
      "reconfig": {
        "Http": {
          "endpoint": "/update/interval",
          "port": 8000,
          "method": "PUT",
          "payload": {
            "nanos": 0,
            "secs": 25
          }
        }
      }
    },
    "restart": {
      "uuid": null,
      "query_type": {
        "Http": {
          "endpoint": "/get/device",
          "port": 8000
        }
      }
    }
  },
  "south-sector": {
    "addition": {
      "image": "mrdahaniel/mocker:latest",
      "network_name": "smart_campus_production_smart_uis",
      "env_vars": {
        "ip": "mqtt_broker",
        "RUST_LOG": "mock"
      },
      "args": [
        "key:random(5,35)"
      ]
    },
    "reconfig": {
      "uuid": null,
      "network": "smart_campus_production_smart_uis",
      "query_type": {
        "Http": {

```

```

        "endpoint": "/get/device",
        "port": 8000
    },
    "reconfig": {
        "Http": {
            "endpoint": "/update/interval",
            "port": 8000,
            "method": "PUT",
            "payload": {
                "nanos": 0,
                "secs": 25
            }
        }
    },
    "restart": {
        "uuid": null,
        "query_type": {
            "Http": {
                "endpoint": "/get/device",
                "port": 8000
            }
        }
    }
}

```

### 3 YAML de la aplicación Patio

```
application:
  name: Patio
  description: "Patio App"

locations:
  greenhouse:
    name: "Greenhouse"
    data-requirements:
      temperature:
        output: number
        required: true
        count: 1
        timeout: 30
      co2:
        output: number
        required: true
        count: 1
        timeout: 120
      humidity:
        output: number
        required: true
        count: 1
        timeout: 120
```

### 4 YAML la aplicación House

```
application:
  name: House
  description: "House App"

locations:
  first-floor:
    name: "First Floor"
    data-requirements:
      temperature:
        output: number
        required: true
        count: 1
        timeout: 30
  basement:
    name: "Basement"
    data-requirements:
      co:
        output: number
        required: false
        count: 1
        timeout: 30
```

## 5 Directivas de la aplicación Patio

```
{
  "greenhouse": {
    "addition": {
      "image": "mrdahaniel/mocker:latest",
      "network_name": "smart_campus_production_smart_uis",
      "env_vars": {
        "RUST_LOG": "mock",
        "ip": "mqtt_broker"
      },
      "args": [
        "key:random(5,35)"
      ]
    },
    "reconfig": {
      "uuid": null,
      "network": "smart_campus_production_smart_uis",
      "query_type": {
        "Http": {
          "endpoint": "/get/device",
          "port": 8000
        }
      },
      "reconfig": {
        "Http": {
          "endpoint": "/update/interval",
          "port": 8000,
          "method": "PUT",
          "payload": {
            "nanos": 0,
            "secs": 25
          }
        }
      }
    },
    "restart": {
      "uuid": null,
      "query_type": {
        "Http": {
          "endpoint": "/get/device",
          "port": 8000
        }
      }
    }
  }
}
```



## 6 Directivas de la aplicación House

```
{
  "basement": {
    "addition": {
      "image": "mrdahaniel/mocker:latest",
      "network_name": "smart_campus_production_smart_uis",
      "env_vars": {
        "RUST_LOG": "mock",
        "ip": "mqtt_broker"
      },
      "args": [
        "key:random(5,35)"
      ]
    },
    "reconfig": {
      "uuid": null,
      "network": "smart_campus_production_smart_uis",
      "query_type": {
        "Http": {
          "endpoint": "/get/device",
          "port": 8000
        }
      },
      "reconfig": {
        "Http": {
          "endpoint": "/update/interval",
          "port": 8000,
          "method": "PUT",
          "payload": {
            "nanos": 0,
            "secs": 25
          }
        }
      }
    },
    "restart": {
      "uuid": null,
      "query_type": {
        "Http": {
          "endpoint": "/get/device",
          "port": 8000
        }
      }
    }
  },
  "first-floor": {
    "addition": {
      "image": "mrdahaniel/mocker:latest",
      "network_name": "smart_campus_production_smart_uis",
      "env_vars": {
        "ip": "mqtt_broker",
        "RUST_LOG": "mock"
      },
      "args": [
        "key:random(5,35)"
      ]
    },
    "reconfig": {
      "uuid": null,
      "network": "smart_campus_production_smart_uis",
      "query_type": {
        "Http": {
```

```
        "endpoint": "/get/device",
        "port": 8000
    },
    "reconfig": {
        "Http": {
            "endpoint": "/update/interval",
            "port": 8000,
            "method": "PUT",
            "payload": {
                "nanos": 0,
                "secs": 25
            }
        }
    },
    "restart": {
        "uuid": null,
        "query_type": {
            "Http": {
                "endpoint": "/get/device",
                "port": 8000
            }
        }
    }
}
```